Project Context: Mobility_Control_Tower

Project root: /mnt/d/wsl/projects/Mobility_Control_Tower
This PDF is structured for LLM ingestion: file paths are explicit and text blocks are wrapped in code fences.

1. Project Tree

```
Mobility_Control_Tower/
??? config/
?   ??? sources.yml
??? data/
??? docs/
?   ??? api.md
?   ??? data_quality.md
?   ??? data_source.md
?   ??? final_demo_guide.md
?   ??? kpi_definitions.md
?   ??? phase_1_decision.md
?   ??? project_summary.md
?   ??? realtime_exploration.md
?   ??? realtime_snapshot_kpis.md
?   ??? runbook.md
?   ??? serving_layer.md
?   ??? static_mvp_explanation.md
?   ??? teacher_presentation_notes.md
??? scripts/
?   ??? run_full_demo.sh
?   ??? run_phase_2_demo.sh
??? src/
?   ??? mobility_control_tower/
?   ?   ??? api/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? app.py
?   ?   ?   ??? db.py
?   ?   ?   ??? report.py
?   ?   ?   ??? routes.py
?   ?   ??? dashboard/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? api_client.py
?   ?   ?   ??? app.py
?   ?   ??? ingestion/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? gtfs_raw.py
?   ?   ??? metrics/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? gtfs_kpis.py
?   ?   ??? profiling/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? gtfs_profile.py
?   ?   ??? quality/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? gtfs_quality.py
?   ?   ??? realtime/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? gtfs_rt_charts.py
?   ?   ?   ??? gtfs_rt_compatibility.py
?   ?   ?   ??? gtfs_rt_kpis.py
?   ?   ?   ??? gtfs_rt_parser.py
?   ?   ?   ??? gtfs_rt_raw.py
?   ?   ?   ??? gtfs_rt_report.py
?   ?   ?   ??? gtfs_rt_snapshot_report.py
?   ?   ??? reporting/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? charts.py
?   ?   ?   ??? demo_report.py
?   ?   ?   ??? final_report.py
?   ?   ??? serving/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? duckdb_loader.py
```

```
?   ?   ?   ??? serving_report.py
?   ?   ?   ??? sql_views.py
?   ?   ??? transformations/
?   ?   ?   ??? __init__.py
?   ?   ?   ??? gtfs_bronze.py
?   ?   ?   ??? gtfs_silver.py
?   ?   ??? __init__.py
?   ?   ??? cli.py
?   ?   ??? config.py
?   ??? mobility_control_tower.egg-info/
?       ??? dependency_links.txt
?       ??? entry_points.txt
?       ??? PKG-INFO
?       ??? requires.txt
?       ??? SOURCES.txt
?       ??? top_level.txt
??? tests/
?   ??? test_api.py
?   ??? test_final_demo.py
?   ??? test_gtfs_kpis.py
?   ??? test_gtfs_pipeline.py
?   ??? test_gtfs_profile.py
?   ??? test_gtfs_realtime.py
?   ??? test_gtfs_rt_gold.py
?   ??? test_raw_metadata.py
?   ??? test_serving_layer.py
??? .gitignore
??? pyproject.toml
??? README.md

2. Source And Text Files

.gitignore
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/.gitignore
FILE: .gitignore
‘‘‘text
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
*.egg-info/

# Generated or externally supplied data
data/raw/*
!data/raw/manual/
!data/raw/manual/.gitkeep
!data/raw/tisseo/
!data/raw/tisseo/.gitkeep
data/bronze/*
!data/bronze/.gitkeep
data/silver/*
!data/silver/.gitkeep
data/gold/*
!data/gold/.gitkeep
data/reports/*
!data/reports/.gitkeep
# Generated chart files are covered by data/reports/*; keep the rule explicit for clarity.
data/reports/figures/*
data/raw_realtime/*
!data/raw_realtime/.gitkeep
data/realtime/*
!data/realtime/.gitkeep
data/realtime_gold/*
!data/realtime_gold/.gitkeep
data/serving/*
!data/serving/.gitkeep
*.pb
*.zip
*.duckdb
```

````
*.duckdb.wal
*.db
```

pyproject.toml
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/pyproject.toml
FILE: pyproject.toml
```toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "mobility-control-tower"
version = "0.1.0"
description = "Local public transport data engineering platform"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
  "duckdb>=1.0,<2",
  "fastapi>=0.115,<1",
  "gtfs-realtime-bindings>=1.0,<2",
  "matplotlib>=3.8,<4",
  "pandas>=2.0,<3",
  "PyYAML>=6.0,<7",
  "requests>=2.31,<3",
  "streamlit>=1.40,<2",
  "uvicorn>=0.30,<1",
]

[project.optional-dependencies]
dev = ["pytest>=8,<9"]

[project.scripts]
mobility-control-tower = "mobility_control_tower.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

README.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/README.md
FILE: README.md
```markdown
# Mobility Control Tower

Local Data Engineering platform for public transport data.

Mobility Control Tower processes Tisseo Toulouse GTFS Schedule data and bounded
GTFS-Realtime snapshots into validated, queryable, teacher-friendly data products. It is a
local academic MVP, not an enterprise monitoring platform.

## Architecture Overview

```mermaid
flowchart LR
    A[Official GTFS Schedule] --> B[raw]
    B --> C[bronze]
    C --> D[silver]
    D --> E[gold static KPIs]
    E --> F[DuckDB serving]
    F --> G[FastAPI read-only API]
    G --> H[Streamlit local dashboard]

    RT[GTFS-Realtime snapshot] --> RTRaw[raw feed.pb]
````
````

````
    RTRaw --> RTParsed[parsed realtime CSV]
    RTParsed --> RTGold[realtime snapshot KPIs]
    RTGold --> F
```

## Data Source

- Static GTFS: Tisseo Toulouse public transport schedule.
- GTFS-Realtime: one saved snapshot at a time.
- Licence: ODbL for the static source.
- Official dataset page:
https://transport.data.gouv.fr/datasets/tisseo-reseau-transport-urbain-toulousain

## Implemented Features

- Static GTFS ingestion with immutable raw ZIP preservation.
- Metadata capture with SHA256 checksums.
- Bronze extraction and silver cleaned GTFS tables.
- GTFS quality validation reports.
- Gold static planning KPIs and charts.
- GTFS-Realtime snapshot fetch, raw protobuf preservation, and parsing.
- Static/live identifier compatibility checks.
- Realtime snapshot delay indicators.
- Local DuckDB serving database.
- Read-only FastAPI API.
- Read-only Streamlit dashboard.
- Teacher-facing reports and documentation.
- Pytest coverage for core behavior.

## Current Pipeline

Static:

```text
GTFS ZIP -> raw -> bronze -> silver -> validation -> gold KPIs -> charts/reports
```

Realtime snapshot:

```text
GTFS-RT feed.pb -> raw_realtime -> realtime parsed CSV -> realtime_gold KPIs ->
charts/reports
```

Serving and demo:

```text
gold + realtime_gold -> DuckDB -> FastAPI -> Streamlit dashboard
```

## Quickstart

```bash
python -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev]'
pytest
```

## Full Workflow

Replace placeholders with the run IDs printed by the commands.

```bash
python -m mobility_control_tower.cli ingest-gtfs --source tisseo --download
python -m mobility_control_tower.cli profile-gtfs --raw-run data/raw/tisseo/<static_run_id>
python -m mobility_control_tower.cli build-bronze --raw-run data/raw/tisseo/<static_run_id>
python -m mobility_control_tower.cli build-silver --bronze-run
data/bronze/tisseo/<static_run_id>
````

```
python -m mobility_control_tower.cli validate-gtfs --silver-run
data/silver/tisseo/<static_run_id>
python -m mobility_control_tower.cli build-gold --silver-run
data/silver/tisseo/<static_run_id>
python -m mobility_control_tower.cli generate-static-charts --gold-run
data/gold/tisseo/<static_run_id>
python -m mobility_control_tower.cli generate-demo-report --gold-run
data/gold/tisseo/<static_run_id>
python -m mobility_control_tower.cli generate-static-mvp-report --gold-run
data/gold/tisseo/<static_run_id>

python -m mobility_control_tower.cli fetch-gtfs-rt --source tisseo --feed-type trip_updates
python -m mobility_control_tower.cli parse-gtfs-rt --raw-rt-run
data/raw_realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli report-gtfs-rt --rt-run
data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli check-rt-compatibility --silver-run
data/silver/tisseo/<static_run_id> --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli build-rt-gold --silver-run
data/silver/tisseo/<static_run_id> --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-rt-charts --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-rt-snapshot-report --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>

python -m mobility_control_tower.cli build-serving-db --gold-run
data/gold/tisseo/<static_run_id> --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-serving-report --serving-run
data/serving/tisseo/<static_run_id>
```

API Usage

Start the local read-only API:

```bash
python -m mobility_control_tower.cli serve-api \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb \
  --host 127.0.0.1 \
  --port 8000
```

Example URLs:

- `http://127.0.0.1:8000/health`
- `http://127.0.0.1:8000/metadata`
- `http://127.0.0.1:8000/static/top-routes?limit=5`
- `http://127.0.0.1:8000/realtime/feed-health`
- `http://127.0.0.1:8000/docs`

Generate the API report:

```bash
python -m mobility_control_tower.cli generate-api-report \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb
```

Dashboard Usage

Start the API first, then run:

```bash
streamlit run src/mobility_control_tower/dashboard/app.py
```

Optional CLI helper:

```bash
```

```
python -m mobility_control_tower.cli serve-dashboard --api-url http://127.0.0.1:8000
```

The dashboard is read-only and consumes the FastAPI endpoints. It shows static planning KPIs, GTFS-Realtime snapshot indicators, compatibility, and limitations.

Generated Reports

Reports are generated under `data/reports/`:

- GTFS profile report.
- GTFS quality report.
- Static demo report.
- Static MVP evidence report.
- GTFS-Realtime snapshot report.
- Realtime compatibility report.
- Realtime snapshot KPI report.
- Serving layer report.
- API report.
- Final project report.

Generate the final report:

```bash
python -m mobility_control_tower.cli generate-final-report \
  --serving-run data/serving/tisseo/<static_run_id>
```

Testing

```bash
pytest
```

Tests use fake GTFS, fake GTFS-Realtime protobuf messages, temporary DuckDB databases, and mocked API responses. They do not require internet access.

Limitations

- Local academic MVP, not production software.
- GTFS-Realtime is processed as snapshots, not streaming.
- No dashboard writes, authentication, deployment, Docker, cloud, Kafka, Spark, Airflow, PostgreSQL, ML, or RAG.
- Delay indicators from one snapshot are not route reliability metrics.
- Trip-level real-time compatibility may be imperfect.

Future Roadmap

- Collect repeated GTFS-Realtime snapshots.
- Improve trip-level matching.
- Add a richer dashboard once the API contract is stable.
- Add scheduling/orchestration only after the local MVP is accepted.
- Consider PostgreSQL or production deployment only in a later phase.

Academic Positioning

This project demonstrates Data Engineering fundamentals: ingestion, raw preservation, layered transformations, validation, KPI design, realtime snapshot parsing, local serving, API exposure, dashboard consumption, testing, and documentation.

It is intentionally scoped so a student can explain every layer clearly during a PFA presentation.
```

config/sources.yml
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/config/sources.yml
FILE: config/sources.yml
```yaml
sources:
```
```

```
  tisseo:
    name: "TissØo ? RØseau transport urbain toulousain"
    source_page_url:
"https://transport.data.gouv.fr/datasets/tisseo-reseau-transport-urbain-toulousain"
    data_gouv_url:
"https://www.data.gouv.fr/datasets/tisseo-reseau-transport-urbain-toulousain"
    download_url:
"https://data.toulouse-metropole.fr/explore/dataset/tisseo-gtfs/files/fc1dda89077cf37e4f7521
760e0ef4e9/download/"
    licence: "ODbL"
    output_filename: "Tisseo_GTFS.zip"
    gtfs_realtime:
      trip_updates_url: "https://api.tisseo.fr/opendata/gtfsrt/GtfsRt.pb"
      vehicle_positions_url: null
      service_alerts_url: "https://api.tisseo.fr/opendata/gtfsrt/Alert.pb"
'''

docs/api.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/api.md
FILE: docs/api.md
'''markdown
# Local read-only API

## Purpose

The API exposes the DuckDB serving database through simple JSON endpoints. It proves that
the project can serve trusted data products to future consumers such as a dashboard,
notebook, external application, or teacher demo.

This is a local read-only API. It does not modify data.

## Start the API

'''bash
python -m mobility_control_tower.cli serve-api \
  --db data/serving/tisseo/<run_id>/mobility_control_tower.duckdb \
  --host 127.0.0.1 \
  --port 8000
'''

Interactive FastAPI documentation is available at:

'http://127.0.0.1:8000/docs'

## Basic endpoints

### GET /health

Returns service status and whether the DuckDB database can be opened.

### GET /metadata

Returns the database path, available tables, available views, and whether static and
real-time snapshot data are present.

## Static planning endpoints

### GET /static/network-overview

Reads 'v_network_overview'.

Query parameters:

- 'limit': default 20, max 100.

### GET /static/top-routes

Reads 'v_top_routes_static'.
```

Query parameters:

- ‘limit‘: default 10, max 100.

GET /static/hourly-headway

Reads ‘v_route_hourly_headway‘.

Query parameters:

- ‘route_id‘: optional.
- ‘service_date‘: optional.
- ‘limit‘: default 50, max 500.

Example:

‘/static/hourly-headway?route_id=line:69&service_date=2026-07-31&limit=10‘

GET /static/route-types

Reads ‘v_route_type_daily_summary‘.

Query parameters:

- ‘service_date‘: optional.
- ‘limit‘: default 50, max 500.

Real-time snapshot endpoints

These endpoints work only when the DuckDB file contains real-time snapshot views. If not, they return HTTP 404 with a clear message.

GET /realtime/feed-health

Reads ‘v_rt_feed_health‘.

GET /realtime/compatibility

Reads ‘v_rt_identifier_compatibility‘.

GET /realtime/top-delayed-routes

Reads ‘v_rt_top_delayed_routes_snapshot‘.

Query parameters:

- ‘limit‘: default 10, max 100.

GET /realtime/top-delayed-stops

Reads ‘v_rt_top_delayed_stops_snapshot‘.

Query parameters:

- ‘limit‘: default 10, max 100.

Response format

Most endpoints return:

‘‘‘json
{
  "data": [],
  "count": 0,
  "source": "view_name",
  "notes": []
}
‘‘‘

Limitations

- This is not a production API.
- It has no authentication because it is local and read-only.
- It does not expose arbitrary SQL.
- It does not implement a dashboard.
- It does not implement continuous streaming.
- Real-time endpoints are snapshot-based.

Why this prepares a future dashboard

A dashboard can later consume these endpoints instead of reading local CSV files directly. The API creates a clean contract between data products and future user interfaces.
‘‘‘

docs/data_quality.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/data_quality.md
FILE: docs/data_quality.md
‘‘‘markdown
Basic GTFS data quality

The silver validator checks whether core tables and columns exist, reports row counts, and detects:

- missing key values and duplicate stop, route, or trip identifiers;
- invalid stop latitude and longitude;
- uncommon or invalid obvious ‘route_type‘ values;
- invalid arrival and departure time syntax, including support for valid hours above 23;
- stop times referencing unknown trips or stops;
- trips referencing unknown routes or service identifiers.

Status meanings

- **PASS**: the check found no problems.
- **WARN**: the data deserves review, but the condition may be an extension or a usable non-critical value. The route-type range check uses this status.
- **FAIL**: a required structure, value, format, uniqueness rule, or relationship has a detected problem.

The report’s overall status is the most severe individual status. Validation reports findings without modifying silver data or stopping merely because a check fails.

Limitations

This is educational, basic validation rather than complete GTFS certification. It does not implement the full GTFS Schedule specification, conditional field requirements, geographic plausibility within Toulouse, ordering rules, schedule consistency, shape validation, translated feeds, or every extended route type. It counts problems but does not include potentially sensitive or very large lists of affected rows. A dedicated standards validator would still be appropriate before production publication.
‘‘‘

docs/data_source.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/data_source.md
FILE: docs/data_source.md
‘‘‘markdown
TissØo data source

The selected dataset is **TissØo ? RØseau transport urbain toulousain**. TissØo Voyageurs produces transport data for the Toulouse urban network. The static GTFS describes the theoretical public-transport offer: stops, schedules, and lines. It is published daily and covers approximately the next three weeks.

Official pages:

- [French National Access Point for transport data](https://transport.data.gouv.fr/datasets/tisseo-reseau-transport-urbain-toulousain)
- [data.gouv.fr dataset page](https://www.data.gouv.fr/datasets/tisseo-reseau-transport-urbain-toulousain)

The dataset page identifies the licence as the **Open Data Commons Open Database License (ODbL)**. This project records that licence identifier in ingestion metadata; it does not reinterpret or add licence terms.

Why start with static GTFS?

Static GTFS is finite, file-based, reproducible, and easy to inspect. It gives this first phase a clear foundation for raw-data preservation, metadata, and data-quality profiling before any continuously changing inputs are introduced.

TissØo also publishes GTFS-RT real-time data. It is postponed because real-time ingestion introduces polling, changing state, and operational concerns that are outside this phase. No GTFS-RT data is downloaded or processed here.
'''

docs/final_demo_guide.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/final_demo_guide.md
FILE: docs/final_demo_guide.md
'''markdown
Final demo guide

Full demo sequence

1. Install:

   '''bash
   python -m venv .venv
   source .venv/bin/activate
   python -m pip install -e '.[dev]'
   '''

2. Run tests:

   '''bash
   pytest
   '''

3. Download and ingest static GTFS:

   '''bash
   python -m mobility_control_tower.cli ingest-gtfs --source tisseo --download
   '''

4. Build static layers:

   '''bash
   python -m mobility_control_tower.cli profile-gtfs --raw-run
data/raw/tisseo/<static_run_id>
   python -m mobility_control_tower.cli build-bronze --raw-run
data/raw/tisseo/<static_run_id>
   python -m mobility_control_tower.cli build-silver --bronze-run
data/bronze/tisseo/<static_run_id>
   python -m mobility_control_tower.cli validate-gtfs --silver-run
data/silver/tisseo/<static_run_id>
   python -m mobility_control_tower.cli build-gold --silver-run
data/silver/tisseo/<static_run_id>
   '''

5. Generate static evidence:

   '''bash
   python -m mobility_control_tower.cli generate-static-charts --gold-run
data/gold/tisseo/<static_run_id>
   python -m mobility_control_tower.cli generate-demo-report --gold-run
data/gold/tisseo/<static_run_id>
   '''

6. Fetch and parse one GTFS-Realtime snapshot:

   ‘‘‘bash
   python -m mobility_control_tower.cli fetch-gtfs-rt --source tisseo --feed-type
trip_updates
   python -m mobility_control_tower.cli parse-gtfs-rt --raw-rt-run
data/raw_realtime/tisseo/trip_updates/<rt_run_id>
   ‘‘‘

7. Build realtime snapshot KPIs:

   ‘‘‘bash
   python -m mobility_control_tower.cli build-rt-gold --silver-run
data/silver/tisseo/<static_run_id> --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
   ‘‘‘

8. Build DuckDB serving database:

   ‘‘‘bash
   python -m mobility_control_tower.cli build-serving-db --gold-run
data/gold/tisseo/<static_run_id> --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
   ‘‘‘

9. Start API:

   ‘‘‘bash
   python -m mobility_control_tower.cli serve-api --db
data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb
   ‘‘‘

10. Start dashboard in another terminal:

    ‘‘‘bash
    streamlit run src/mobility_control_tower/dashboard/app.py
    ‘‘‘

11. Generate final report:

    ‘‘‘bash
    python -m mobility_control_tower.cli generate-final-report --serving-run
data/serving/tisseo/<static_run_id>
    ‘‘‘

Five-minute demo script

1. "This project turns official Tisseo transport data into trusted local data products."
2. "The static GTFS pipeline preserves raw data, cleans it, validates it, and builds
planning KPIs."
3. "The realtime part is snapshot-based. It parses one GTFS-Realtime feed and compares IDs
with static GTFS."
4. "DuckDB makes the outputs queryable with SQL."
5. "FastAPI exposes read-only JSON endpoints."
6. "The Streamlit dashboard consumes the API and shows the final local demo."
7. "This is not production monitoring; it is a complete academic MVP."
‘‘‘

docs/kpi_definitions.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/kpi_definitions.md
FILE: docs/kpi_definitions.md
‘‘‘markdown
Static GTFS KPI definitions

These gold indicators describe the **planned transport offer** in TissØo’s static schedule.

Scheduled trips by route and day

One row represents one route on one service date. ‘scheduled_trips_count‘ counts trips whose
‘service_id‘ is active that day. This is the main first-version KPI because it makes planned
route supply easy to compare across dates.

Scheduled departures by route and hour

One row represents one route, service date, and service-day hour. Only the minimum ‘stop_sequence‘ for each trip is counted, so the value represents trip starts rather than all station calls. GTFS hours after midnight remain 24, 25, and so on.

Scheduled stop departures by day

One row represents one stop on one service date. Every stop-time departure with a usable departure time is counted, giving the planned activity at that stop.

Network daily summary

One row represents a service date and reports active routes, scheduled trips, scheduled stop departures, and active stops. It is a compact overview of the scheduled network supply.

Route period summary

One row represents one route over the full GTFS service period. It reports active service days, total scheduled trips, average trips per active day, maximum daily trips, and the first and last service dates. Use this table when saying "top routes over the GTFS service period".

Planned hourly headway

One row represents one route, service date, and service-day hour. ‘planned_headway_minutes‘ is ‘60 / scheduled_departures_count‘. This is an approximate planned headway from the static schedule, not real passenger waiting time.

Route type daily summary

One row represents one service date and GTFS route type. It reports active routes, scheduled trips, and scheduled stop departures for labels such as Tram, Subway/Metro, Bus, and other standard GTFS route types. Unknown route types are labelled Unknown.

Busiest route/day

This table keeps the top 50 individual route/day combinations by scheduled trips. It is useful for evidence because it avoids mixing a per-day peak with a full-period total.

Busiest stop/day

This table keeps the top 50 individual stop/day combinations by scheduled departures. It identifies high-activity planned stop days, not observed passenger demand.

Service calendars

Regular weekday service is expanded between ‘start_date‘ and ‘end_date‘. A ‘calendar_dates‘ exception type 1 adds service; type 2 removes it. If only ‘calendar_dates‘ exists, additions define the available dates.

Why these are not reliability KPIs

Static GTFS says what should run, not what actually happened. These KPIs measure planned supply and establish a comparison baseline. Future real-time observations could add actual arrivals, delays, cancellations, headway regularity, and planned-versus-observed reliability. Those concerns are intentionally outside the current phase.
‘‘‘

docs/phase_1_decision.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/phase_1_decision.md
FILE: docs/phase_1_decision.md
‘‘‘markdown
Phase 1 decision

The first implementation slice uses only TissØo static GTFS. Each source archive is copied unchanged into a timestamped raw run, accompanied by provenance metadata and a SHA256 checksum. Profiling reads directly from that preserved ZIP and writes human- and

machine-readable reports.

This deliberately establishes reproducibility and basic observability before adding transformations or infrastructure. Generated data is ignored by Git.
'''

docs/project_summary.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/project_summary.md
FILE: docs/project_summary.md
'''markdown
Project summary

Title

Mobility Control Tower

Objective

Build a local Data Engineering platform that processes public transport data and produces reliability-oriented evidence for an academic PFA demo.

Data source

Tisseo Toulouse static GTFS and GTFS-Realtime snapshot feeds.

Implemented components

- Static GTFS ingestion
- Raw, bronze, silver, gold layers
- Quality validation
- Static planning KPIs
- GTFS-Realtime snapshot parsing
- Static/live compatibility
- Realtime snapshot KPIs
- DuckDB serving database
- FastAPI read-only API
- Streamlit local dashboard
- Reports and documentation
- Automated tests

Generated outputs

CSV tables, JSON manifests, Markdown reports, PNG charts, DuckDB database, API responses, and dashboard views.

Technical skills demonstrated

Python, pandas, GTFS, GTFS-Realtime protobuf, data validation, layered data architecture, KPI design, DuckDB SQL, FastAPI, Streamlit, pytest, documentation.

Current limitations

Realtime is snapshot-based, not streaming. The dashboard is local. No production deployment or authentication is included.

Future work

Repeated snapshot collection, improved trip matching, richer visual analytics, and production architecture evaluation.

Final status

Ready for academic demonstration.
'''

docs/realtime_exploration.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/realtime_exploration.md
FILE: docs/realtime_exploration.md
'''markdown

GTFS-Realtime exploration

What GTFS-Realtime is

GTFS-Realtime is the live companion to static GTFS. Static GTFS describes the planned offer. GTFS-Realtime describes operational information observed around fetch time.

Common GTFS-Realtime feed entity types are:

- Trip Updates: changes or predictions for trips and stop times.
- Vehicle Positions: vehicle location and status information.
- Service Alerts: disruptions, messages, and informed routes, stops, or trips.

Tisseo context

The official transport.data.gouv.fr dataset page states that Tisseo publishes GTFS-RT data and lists a combined ‘GtfsRt.pb‘ resource containing TripUpdate and Alert entities, plus an ‘Alert.pb‘ resource containing alerts. Vehicle Positions are not listed as available there, so the project leaves the vehicle positions URL unset unless a reliable official endpoint is provided later.

Why snapshots first

This phase fetches one protobuf file and saves it unchanged as ‘feed.pb‘. That file is a deterministic local snapshot. It can be reused for:

- parser tests;
- replay experiments;
- demos without live internet;
- static/live compatibility checks.

This is intentionally not continuous streaming. There is no Kafka, Spark, Airflow, database, API, dashboard, or cloud service in this phase.

Why static/live compatibility matters

Future real-time reliability indicators need joins between live observations and static schedule data. Before building those indicators, the project must answer whether live ‘route_id‘, ‘trip_id‘, and ‘stop_id‘ values can be matched to the silver static GTFS tables.

The compatibility report measures how many identifiers match and how many are unmatched. Perfect matching is not required during exploration, but the result shows whether the feed is a candidate for later real-time phases.

What the real-time gold layer adds

The real-time gold layer combines one parsed Trip Updates snapshot with the static silver GTFS tables. It creates:

- feed health indicators;
- enriched trip update rows with static route and trip match flags;
- route delay snapshot indicators;
- stop delay snapshot indicators;
- identifier compatibility percentages;
- static PNG charts and a teacher-facing Markdown report.

These are snapshot indicators observed at fetch time, not continuous real-time reliability metrics.

Why trip ID mismatch can happen

Trip IDs can fail to match when the selected static GTFS run does not exactly correspond to the real-time publication, when operational trips are represented differently in real-time, or when the real-time feed exposes partial or modified trip descriptors. This is why the project keeps unmatched trip rows instead of dropping them.

Why route IDs and stop IDs still matter

Route and stop identifiers are often more stable than trip identifiers. If ‘route_id‘ and

‘stop_id‘ compatibility are strong, the snapshot can still support route-level and stop-level exploration while trip-level joins remain under review.

Why this does not justify Kafka or Spark yet

One snapshot proves parsing and compatibility, not sustained volume, velocity, or production operations. Before introducing streaming tools, the project should collect multiple snapshots, compare freshness, inspect identifier stability, and confirm that delay indicators remain interpretable.

What results are needed before streaming later

Before any streaming phase, the project should have evidence that:

- a GTFS-Realtime snapshot can be fetched and preserved;
- the protobuf can be parsed into simple tables;
- feed age can be estimated from the header timestamp;
- useful identifiers are present;
- identifiers mostly match static silver GTFS;
- saved snapshots can reproduce the same parser and compatibility outputs offline.
‘‘‘

docs/realtime_snapshot_kpis.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/realtime_snapshot_kpis.md
FILE: docs/realtime_snapshot_kpis.md
‘‘‘markdown
Real-time snapshot KPI definitions

These indicators are computed from one parsed GTFS-Realtime Trip Updates snapshot and one static silver GTFS run.

Feed health

‘rt_feed_health_snapshot.csv‘ describes the snapshot itself: feed type, fetch time, header timestamp, feed age, entity counts, identifier counts, and whether delay fields are present.

Freshness status:

- PASS: feed age is 90 seconds or less;
- WARN: feed age is over 90 seconds and up to 300 seconds;
- FAIL: feed age is over 300 seconds;
- UNKNOWN: feed age cannot be computed.

This is a snapshot health indicator, not a service-level guarantee.

Identifier compatibility

‘rt_identifier_compatibility_snapshot.csv‘ compares distinct ‘route_id‘, ‘trip_id‘, and ‘stop_id‘ values from the real-time snapshot with static silver GTFS tables.

Status:

- PASS: match percentage is at least 95%;
- WARN: match percentage is at least 50% and below 95%;
- FAIL: match percentage is below 50%;
- NOT_APPLICABLE: no values are available.

This helps explain whether static/live joins are safe enough for later phases.

Enriched trip updates

‘rt_trip_update_enriched.csv‘ keeps every parsed trip update row and adds route names, route match flags, trip match flags, and a compatibility note. Rows with unmatched ‘trip_id‘ values are preserved for diagnosis.

Route delay snapshot

‘rt_route_delay_snapshot.csv‘ groups stop-time updates by route. Delay uses ‘arrival_delay‘

when available, otherwise ‘departure_delay‘. It reports average, median, min, max, five-minute delay counts, early update counts, and missing-delay counts.

This is not route reliability. It is only delay observed in one snapshot.

Stop delay snapshot

‘rt_stop_delay_snapshot.csv‘ groups stop-time updates by stop and adds static stop names when available. It reports the same delay logic at stop level.

Limitations of single-snapshot metrics

A single snapshot cannot prove recurring delay patterns, punctuality, cancellation behavior, or passenger waiting time. It can only show what was visible at one fetch time.

What continuous reliability would need

Continuous reliability indicators would require repeated snapshots over time, stable scheduling of fetches, storage design, deduplication rules, temporal windows, stronger static/live matching, and clear definitions for delay, cancellation, and headway reliability.

Those concerns are intentionally outside this snapshot-only phase.
’’’

docs/runbook.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/runbook.md
FILE: docs/runbook.md
’’’markdown
Mobility Control Tower Runbook

Operational guide for the local academic MVP.

Setup

’’’bash
python -m venv .venv
source .venv/bin/activate
python -m pip install -e ’.[dev]’
pytest
’’’

Run all commands from the repository root.

Static Workflow

’’’bash
python -m mobility_control_tower.cli ingest-gtfs --source tisseo --download
python -m mobility_control_tower.cli profile-gtfs --raw-run data/raw/tisseo/<static_run_id>
python -m mobility_control_tower.cli build-bronze --raw-run data/raw/tisseo/<static_run_id>
python -m mobility_control_tower.cli build-silver --bronze-run
data/bronze/tisseo/<static_run_id>
python -m mobility_control_tower.cli validate-gtfs --silver-run
data/silver/tisseo/<static_run_id>
python -m mobility_control_tower.cli build-gold --silver-run
data/silver/tisseo/<static_run_id>
python -m mobility_control_tower.cli generate-static-charts --gold-run
data/gold/tisseo/<static_run_id>
python -m mobility_control_tower.cli generate-demo-report --gold-run
data/gold/tisseo/<static_run_id>
python -m mobility_control_tower.cli generate-static-mvp-report --gold-run
data/gold/tisseo/<static_run_id>
’’’

Manual fallback: place ‘Tisseo_GTFS.zip‘ in ‘data/raw/manual/‘ and use ‘--local-zip data/raw/manual/Tisseo_GTFS.zip‘.

Realtime Snapshot Workflow

```bash
python -m mobility_control_tower.cli fetch-gtfs-rt --source tisseo --feed-type trip_updates
python -m mobility_control_tower.cli parse-gtfs-rt --raw-rt-run
data/raw_realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli report-gtfs-rt --rt-run
data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli check-rt-compatibility --silver-run
data/silver/tisseo/<static_run_id> --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli build-rt-gold --silver-run
data/silver/tisseo/<static_run_id> --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-rt-charts --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-rt-snapshot-report --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
```

Use careful wording: this is a GTFS-Realtime snapshot observed at fetch time, not streaming.

Serving Workflow

```bash
python -m mobility_control_tower.cli build-serving-db \
  --gold-run data/gold/tisseo/<static_run_id> \
  --rt-gold-run data/realtime_gold/tisseo/trip_updates/<rt_run_id>

python -m mobility_control_tower.cli query-serving-db \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb \
  --query-name top-routes \
  --limit 10

python -m mobility_control_tower.cli generate-serving-report \
  --serving-run data/serving/tisseo/<static_run_id>
```

Useful query names: ‘network-overview‘, ‘top-routes‘, ‘hourly-headway‘, ‘route-types‘, ‘rt-feed-health‘, ‘rt-compatibility‘, ‘rt-top-delayed-routes‘, ‘rt-top-delayed-stops‘.

API Workflow

```bash
python -m mobility_control_tower.cli serve-api \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb \
  --host 127.0.0.1 \
  --port 8000
```

Open:

- ‘http://127.0.0.1:8000/health‘
- ‘http://127.0.0.1:8000/metadata‘
- ‘http://127.0.0.1:8000/static/top-routes?limit=5‘
- ‘http://127.0.0.1:8000/realtime/feed-health‘
- ‘http://127.0.0.1:8000/docs‘

Generate report:

```bash
python -m mobility_control_tower.cli generate-api-report \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb
```

Dashboard Workflow

Start the API first.

```bash
streamlit run src/mobility_control_tower/dashboard/app.py
```

Or:

```bash
python -m mobility_control_tower.cli serve-dashboard --api-url http://127.0.0.1:8000
```

The dashboard is read-only. It consumes API endpoints and does not write data.

Final Report

```bash
python -m mobility_control_tower.cli generate-final-report \
  --serving-run data/serving/tisseo/<static_run_id>
```

Teacher Demo Sequence

1. Show the architecture diagram in the README.
2. Run `pytest`.
3. Show raw/bronze/silver/gold folders.
4. Open quality and KPI reports.
5. Show realtime snapshot compatibility.
6. Query DuckDB with `top-routes`.
7. Start API and open `/docs`.
8. Start dashboard and explain each section.
9. Open the final project report.

Troubleshooting

- API down in dashboard: start `serve-api` first.
- Missing realtime endpoints: rebuild serving DB with `--rt-gold-run`.
- Missing DuckDB file: run `build-serving-db`.
- Trip compatibility WARN: selected static GTFS may not match all realtime trip IDs.
- Download failure: use manual static ZIP placement or saved realtime snapshots.

Clean Generated Data Safely

Generated data is ignored by git. To clean local outputs, remove specific generated subfolders only, for example:

```bash
rm -r data/reports/<specific_file_or_folder>
rm -r data/serving/tisseo/<static_run_id>
```

Do not delete source code, docs, config, or tests.

What Not To Commit

Do not commit:

- downloaded GTFS ZIP files;
- GTFS-Realtime `feed.pb` snapshots;
- generated CSV data under `data/`;
- DuckDB files;
- generated PNG charts;
- generated reports from real data unless explicitly approved.
```

docs/serving_layer.md
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/docs/serving_layer.md
FILE: docs/serving_layer.md
```markdown
# Local DuckDB serving layer

## What a serving layer is

A serving layer is the place where generated data products become easy to query. In this project, it means loading static gold KPIs and optional real-time snapshot KPIs into a local

DuckDB file.

## Why CSV files alone are not ideal

CSV files are simple and transparent, but analysis across several CSV files quickly becomes inconvenient. SQL views make it easier to demonstrate the project, compare indicators, and answer teacher questions without manually opening many files.

## Why DuckDB is used locally

DuckDB is local and embedded. It does not require a database server, user accounts, networking, Docker, or cloud services. It works well with CSV files and is easy to run from Python.

## Tables and views

The serving database loads available static gold tables such as ‘route_daily_trips‘, ‘network_daily_summary‘, ‘route_period_summary‘, and ‘route_hourly_headway‘.

If a real-time snapshot gold run is provided, it also loads tables such as ‘rt_feed_health_snapshot‘, ‘rt_route_delay_snapshot‘, and ‘rt_identifier_compatibility_snapshot‘.

Main SQL views include:

- ‘v_network_overview‘;
- ‘v_top_routes_static‘;
- ‘v_route_hourly_headway‘;
- ‘v_route_type_daily_summary‘;
- ‘v_rt_feed_health‘;
- ‘v_rt_identifier_compatibility‘;
- ‘v_rt_top_delayed_routes_snapshot‘;
- ‘v_rt_top_delayed_stops_snapshot‘.

## How this prepares future API or dashboard work

The serving layer gives future API or dashboard code a clear source to query. Instead of reading many CSV files directly, a later phase could query the same SQL views.

## Why PostgreSQL is not introduced yet

PostgreSQL would add server setup, administration, credentials, and deployment choices. DuckDB is enough for the current academic MVP because the goal is local SQL demonstration, not production database operations.
‘‘‘

docs/static_mvp_explanation.md
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/docs/static_mvp_explanation.md
FILE: docs/static_mvp_explanation.md
‘‘‘markdown
# Static MVP explanation

## What the static MVP proves

The static MVP proves that the project can take an official public-transport GTFS ZIP and turn it into reproducible Data Engineering outputs:

- immutable raw data preservation with metadata and checksum;
- bronze extraction of source GTFS files;
- cleaned silver tables;
- basic quality validation;
- gold static planning KPIs;
- Markdown reports and PNG charts that can be regenerated.

The important point is repeatability. The same workflow can be run again on a new daily Tisseo GTFS publication and produce comparable outputs.

## What it does not prove

This phase does not prove real-time reliability. It does not process GTFS-RT, delays, cancellations, vehicle positions, passenger counts, or observed waiting time.

The KPIs are scheduled-service indicators:

- scheduled trips, not actual trips;
- scheduled departures, not observed departures;
- planned headway approximation, not real passenger waiting time.

## Why this is a valid first Data Engineering slice

A Data Engineering project should first make source data trustworthy and usable. This slice covers ingestion, raw preservation, metadata, layered transformation, validation, manifests, tests, and analytical outputs.

That foundation is useful before adding real-time data because future reliability indicators need a clean static schedule baseline.

## What real-time data will add later

Real-time data can later compare observed service against the planned schedule. It could add actual arrival times, delays, missed trips, vehicle positions, headway regularity, and planned-versus-observed reliability indicators.

Those features are intentionally postponed so the static pipeline remains simple, testable, and explainable.
'''

docs/teacher_presentation_notes.md
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/docs/teacher_presentation_notes.md
FILE: docs/teacher_presentation_notes.md
'''markdown
# Teacher presentation notes

## Simple explanation

Mobility Control Tower is a local Data Engineering platform for public transport data. It transforms official transport files into validated tables, KPIs, reports, an API, and a dashboard.

## Public transport data

Public transport data describes stops, routes, trips, schedules, and sometimes live service updates.

## GTFS Schedule

GTFS Schedule is the planned offer: where stops are, which routes exist, and what trips are scheduled.

## GTFS-Realtime

GTFS-Realtime is operational data observed around fetch time. In this project it is handled as saved snapshots, not streaming.

## Pipeline

The pipeline preserves raw files, creates bronze copies, cleans silver tables, validates quality, computes gold KPIs, serves data through DuckDB and FastAPI, then displays a local dashboard.

## Why raw, bronze, silver, gold

- Raw: immutable source evidence.
- Bronze: extracted source files.
- Silver: cleaned canonical tables.
- Gold: analytical data products.

## Why DuckDB and FastAPI

DuckDB provides local SQL without a server. FastAPI exposes the trusted DuckDB views as read-only JSON endpoints for future consumers.

## Dashboard

The dashboard shows API health, static planning KPIs, GTFS-Realtime snapshot feed health, compatibility, and observed delay indicators.

## Implemented

Ingestion, validation, KPI computation, realtime snapshot parsing, DuckDB serving, read-only API, dashboard, reports, docs, and tests.

## Not implemented

No Kafka, Spark, Airflow, Docker, cloud, ML, authentication, production API, or streaming monitoring system.

## Future extensions

Collect repeated realtime snapshots, improve trip matching, add richer dashboard views, and consider production infrastructure only after the academic MVP is accepted.
'''

scripts/run_full_demo.sh
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/scripts/run_full_demo.sh
FILE: scripts/run_full_demo.sh
'''bash
#!/usr/bin/env bash
set -euo pipefail

echo "Mobility Control Tower full demo guide"
echo
echo "Run these commands step by step. Replace <static_run_id> and <rt_run_id> with printed run IDs."
echo
echo "1. pytest"
echo "2. python -m mobility_control_tower.cli ingest-gtfs --source tisseo --download"
echo "3. python -m mobility_control_tower.cli profile-gtfs --raw-run data/raw/tisseo/<static_run_id>"
echo "4. python -m mobility_control_tower.cli build-bronze --raw-run data/raw/tisseo/<static_run_id>"
echo "5. python -m mobility_control_tower.cli build-silver --bronze-run data/bronze/tisseo/<static_run_id>"
echo "6. python -m mobility_control_tower.cli validate-gtfs --silver-run data/silver/tisseo/<static_run_id>"
echo "7. python -m mobility_control_tower.cli build-gold --silver-run data/silver/tisseo/<static_run_id>"
echo "8. python -m mobility_control_tower.cli fetch-gtfs-rt --source tisseo --feed-type trip_updates"
echo "9. python -m mobility_control_tower.cli parse-gtfs-rt --raw-rt-run data/raw_realtime/tisseo/trip_updates/<rt_run_id>"
echo "10. python -m mobility_control_tower.cli build-rt-gold --silver-run data/silver/tisseo/<static_run_id> --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>"
echo "11. python -m mobility_control_tower.cli build-serving-db --gold-run data/gold/tisseo/<static_run_id> --rt-gold-run data/realtime_gold/tisseo/trip_updates/<rt_run_id>"
echo "12. python -m mobility_control_tower.cli serve-api --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb"
echo "13. streamlit run src/mobility_control_tower/dashboard/app.py"
echo "14. python -m mobility_control_tower.cli generate-final-report --serving-run data/serving/tisseo/<static_run_id>"
'''

scripts/run_phase_2_demo.sh
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/scripts/run_phase_2_demo.sh

FILE: scripts/run_phase_2_demo.sh
```bash
#!/usr/bin/env bash
set -euo pipefail

python -m mobility_control_tower.cli ingest-gtfs \
  --source tisseo \
  --local-zip data/raw/manual/Tisseo_GTFS.zip

echo "Use the printed raw run path with:"
echo "python -m mobility_control_tower.cli profile-gtfs --raw-run data/raw/tisseo/<run_id>"
```

src/mobility_control_tower/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/__init__.py
FILE: src/mobility_control_tower/__init__.py
```python
"""Mobility Control Tower package."""

__version__ = "0.1.0"
```

src/mobility_control_tower/cli.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/cli.py
FILE: src/mobility_control_tower/cli.py
```python
"""Command-line interface for the Mobility Control Tower GTFS workflow."""

from __future__ import annotations

import argparse
import os
import subprocess
import sys
from pathlib import Path

import uvicorn

from mobility_control_tower.api.app import create_app
from mobility_control_tower.api.report import generate_api_report
from mobility_control_tower.config import load_source
from mobility_control_tower.ingestion.gtfs_raw import download_and_preserve_gtfs,
preserve_gtfs_zip
from mobility_control_tower.metrics.gtfs_kpis import build_gold
from mobility_control_tower.profiling.gtfs_profile import profile_raw_run
from mobility_control_tower.quality.gtfs_quality import validate_silver_run
from mobility_control_tower.realtime.gtfs_rt_compatibility import
check_realtime_compatibility
from mobility_control_tower.realtime.gtfs_rt_charts import generate_rt_charts
from mobility_control_tower.realtime.gtfs_rt_kpis import build_rt_gold
from mobility_control_tower.realtime.gtfs_rt_parser import parse_realtime_snapshot
from mobility_control_tower.realtime.gtfs_rt_raw import FEED_TYPES, fetch_realtime_snapshot
from mobility_control_tower.realtime.gtfs_rt_report import generate_realtime_report
from mobility_control_tower.realtime.gtfs_rt_snapshot_report import
generate_rt_snapshot_report
from mobility_control_tower.reporting.charts import generate_static_charts
from mobility_control_tower.reporting.demo_report import generate_demo_report,
generate_static_mvp_report
from mobility_control_tower.reporting.final_report import generate_final_report
from mobility_control_tower.serving.duckdb_loader import build_serving_database,
dataframe_to_text_table, query_serving_database
from mobility_control_tower.serving.serving_report import generate_serving_report
from mobility_control_tower.transformations.gtfs_bronze import build_bronze
from mobility_control_tower.transformations.gtfs_silver import build_silver


def build_parser() -> argparse.ArgumentParser:
```

```
    parser = argparse.ArgumentParser(prog="mobility-control-tower")
    commands = parser.add_subparsers(dest="command", required=True)
    ingest = commands.add_parser("ingest-gtfs", help="Preserve a static GTFS ZIP")
    ingest.add_argument("--source", required=True)
    mode = ingest.add_mutually_exclusive_group(required=True)
    mode.add_argument("--local-zip", type=Path)
    mode.add_argument("--download", action="store_true")
    ingest.add_argument("--config", type=Path, default=Path("config/sources.yml"))
    ingest.add_argument("--raw-root", type=Path, default=Path("data/raw"))

    profile = commands.add_parser("profile-gtfs", help="Profile a preserved raw GTFS run")
    profile.add_argument("--raw-run", type=Path, required=True)
    profile.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    bronze = commands.add_parser("build-bronze", help="Extract a raw GTFS run into bronze
files")
    bronze.add_argument("--raw-run", type=Path, required=True)
    bronze.add_argument("--bronze-root", type=Path, default=Path("data/bronze"))

    silver = commands.add_parser("build-silver", help="Clean a bronze GTFS run into silver
CSV tables")
    silver.add_argument("--bronze-run", type=Path, required=True)
    silver.add_argument("--silver-root", type=Path, default=Path("data/silver"))

    validate = commands.add_parser("validate-gtfs", help="Run basic checks on a silver GTFS
run")
    validate.add_argument("--silver-run", type=Path, required=True)
    validate.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    gold = commands.add_parser("build-gold", help="Build static-schedule KPI tables from
silver GTFS")
    gold.add_argument("--silver-run", type=Path, required=True)
    gold.add_argument("--gold-root", type=Path, default=Path("data/gold"))

    report = commands.add_parser("generate-demo-report", help="Generate a Markdown report
from a gold run")
    report.add_argument("--gold-run", type=Path, required=True)
    report.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    charts = commands.add_parser("generate-static-charts", help="Generate static PNG charts
from gold KPI tables")
    charts.add_argument("--gold-run", type=Path, required=True)
    charts.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    static_mvp = commands.add_parser("generate-static-mvp-report", help="Generate a concise
static MVP evidence report")
    static_mvp.add_argument("--gold-run", type=Path, required=True)
    static_mvp.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    fetch_rt = commands.add_parser("fetch-gtfs-rt", help="Fetch and preserve one
GTFS-Realtime protobuf snapshot")
    fetch_rt.add_argument("--source", required=True)
    fetch_rt.add_argument("--feed-type", choices=sorted(FEED_TYPES), required=True)
    fetch_rt.add_argument("--url")
    fetch_rt.add_argument("--config", type=Path, default=Path("config/sources.yml"))
    fetch_rt.add_argument("--raw-rt-root", type=Path, default=Path("data/raw_realtime"))

    parse_rt = commands.add_parser("parse-gtfs-rt", help="Parse a preserved GTFS-Realtime
snapshot into CSV tables")
    parse_rt.add_argument("--raw-rt-run", type=Path, required=True)
    parse_rt.add_argument("--rt-root", type=Path, default=Path("data/realtime"))

    report_rt = commands.add_parser("report-gtfs-rt", help="Generate a Markdown report for a
parsed GTFS-Realtime run")
    report_rt.add_argument("--rt-run", type=Path, required=True)
    report_rt.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    compat_rt = commands.add_parser("check-rt-compatibility", help="Compare parsed real-time
IDs with static silver GTFS")
```

```
    compat_rt.add_argument("--silver-run", type=Path, required=True)
    compat_rt.add_argument("--rt-run", type=Path, required=True)
    compat_rt.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    rt_gold = commands.add_parser("build-rt-gold", help="Build snapshot KPI tables from
parsed GTFS-Realtime and static silver GTFS")
    rt_gold.add_argument("--silver-run", type=Path, required=True)
    rt_gold.add_argument("--rt-run", type=Path, required=True)
    rt_gold.add_argument("--rt-gold-root", type=Path, default=Path("data/realtime_gold"))

    rt_charts = commands.add_parser("generate-rt-charts", help="Generate static PNG charts
from real-time gold tables")
    rt_charts.add_argument("--rt-gold-run", type=Path, required=True)
    rt_charts.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    rt_snapshot_report = commands.add_parser("generate-rt-snapshot-report", help="Generate a
Markdown report from real-time gold tables")
    rt_snapshot_report.add_argument("--rt-gold-run", type=Path, required=True)
    rt_snapshot_report.add_argument("--reports-dir", type=Path,
default=Path("data/reports"))

    serving = commands.add_parser("build-serving-db", help="Build a local DuckDB serving
database from gold outputs")
    serving.add_argument("--gold-run", type=Path, required=True)
    serving.add_argument("--rt-gold-run", type=Path)
    serving.add_argument("--serving-root", type=Path, default=Path("data/serving"))

    serving_query = commands.add_parser("query-serving-db", help="Run a predefined query
against the serving DuckDB database")
    serving_query.add_argument("--db", type=Path, required=True)
    serving_query.add_argument("--query-name", required=True)
    serving_query.add_argument("--limit", type=int, default=10)

    serving_report = commands.add_parser("generate-serving-report", help="Generate a
Markdown report for a serving database run")
    serving_report.add_argument("--serving-run", type=Path, required=True)
    serving_report.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    api = commands.add_parser("serve-api", help="Start the local read-only FastAPI server")
    api.add_argument("--db", type=Path, required=True)
    api.add_argument("--host", default="127.0.0.1")
    api.add_argument("--port", type=int, default=8000)

    api_report = commands.add_parser("generate-api-report", help="Generate a Markdown report
for the local API")
    api_report.add_argument("--db", type=Path, required=True)
    api_report.add_argument("--reports-dir", type=Path, default=Path("data/reports"))

    dashboard = commands.add_parser("serve-dashboard", help="Start the local read-only
Streamlit dashboard")
    dashboard.add_argument("--api-url", default="http://127.0.0.1:8000")

    final_report = commands.add_parser("generate-final-report", help="Generate the final
academic project report")
    final_report.add_argument("--serving-run", type=Path, required=True)
    final_report.add_argument("--reports-dir", type=Path, default=Path("data/reports"))
    return parser


def main() -> None:
    args = build_parser().parse_args()
    try:
        if args.command == "ingest-gtfs":
            source = load_source(args.source, args.config)
            if args.download:
                run_dir = download_and_preserve_gtfs(args.source, source, args.raw_root)
            else:
                run_dir = preserve_gtfs_zip(args.local_zip, args.source, source,
args.raw_root)
```

```
            print(f"Raw GTFS preserved in: {run_dir}")
            print(f"Metadata written to: {run_dir / 'metadata.json'}")
        elif args.command == "profile-gtfs":
            json_path, markdown_path = profile_raw_run(args.raw_run, args.reports_dir)
            print(f"JSON report written to: {json_path}")
            print(f"Markdown report written to: {markdown_path}")
        elif args.command == "build-bronze":
            run_dir = build_bronze(args.raw_run, args.bronze_root)
            print(f"Bronze GTFS written to: {run_dir}")
            print(f"Manifest written to: {run_dir / 'bronze_manifest.json'}")
        elif args.command == "build-silver":
            run_dir = build_silver(args.bronze_run, args.silver_root)
            print(f"Silver GTFS written to: {run_dir}")
            print(f"Manifest written to: {run_dir / 'silver_manifest.json'}")
        elif args.command == "validate-gtfs":
            json_path, markdown_path = validate_silver_run(args.silver_run,
args.reports_dir)
            print(f"JSON quality report written to: {json_path}")
            print(f"Markdown quality report written to: {markdown_path}")
        elif args.command == "build-gold":
            run_dir = build_gold(args.silver_run, args.gold_root)
            print(f"Gold KPI tables written to: {run_dir}")
            print(f"Manifest written to: {run_dir / 'gold_manifest.json'}")
        elif args.command == "generate-demo-report":
            report_path = generate_demo_report(args.gold_run, args.reports_dir)
            print(f"Demo report written to: {report_path}")
        elif args.command == "generate-static-charts":
            figures_dir = generate_static_charts(args.gold_run, args.reports_dir)
            print(f"Static charts written to: {figures_dir}")
        elif args.command == "generate-static-mvp-report":
            report_path = generate_static_mvp_report(args.gold_run, args.reports_dir)
            print(f"Static MVP evidence report written to: {report_path}")
        elif args.command == "fetch-gtfs-rt":
            source = load_source(args.source, args.config)
            run_dir = fetch_realtime_snapshot(args.source, source, args.feed_type, args.url,
args.raw_rt_root)
            print(f"Raw GTFS-Realtime snapshot preserved in: {run_dir}")
            print(f"Metadata written to: {run_dir / 'metadata.json'}")
        elif args.command == "parse-gtfs-rt":
            run_dir = parse_realtime_snapshot(args.raw_rt_run, args.rt_root)
            print(f"Parsed GTFS-Realtime tables written to: {run_dir}")
            print(f"Manifest written to: {run_dir / 'realtime_manifest.json'}")
        elif args.command == "report-gtfs-rt":
            report_path = generate_realtime_report(args.rt_run, args.reports_dir)
            print(f"GTFS-Realtime report written to: {report_path}")
        elif args.command == "check-rt-compatibility":
            json_path, markdown_path = check_realtime_compatibility(args.silver_run,
args.rt_run, args.reports_dir)
            print(f"JSON compatibility report written to: {json_path}")
            print(f"Markdown compatibility report written to: {markdown_path}")
        elif args.command == "build-rt-gold":
            run_dir = build_rt_gold(args.silver_run, args.rt_run, args.rt_gold_root)
            print(f"Real-time gold snapshot tables written to: {run_dir}")
            print(f"Manifest written to: {run_dir / 'rt_gold_manifest.json'}")
        elif args.command == "generate-rt-charts":
            figures_dir = generate_rt_charts(args.rt_gold_run, args.reports_dir)
            print(f"Real-time snapshot charts written to: {figures_dir}")
        elif args.command == "generate-rt-snapshot-report":
            report_path = generate_rt_snapshot_report(args.rt_gold_run, args.reports_dir)
            print(f"Real-time snapshot report written to: {report_path}")
        elif args.command == "build-serving-db":
            serving_run = build_serving_database(args.gold_run, args.rt_gold_run,
args.serving_root)
            print(f"Serving database written to: {serving_run /
'mobility_control_tower.duckdb'}")
            print(f"Manifest written to: {serving_run / 'serving_manifest.json'}")
        elif args.command == "query-serving-db":
            frame = query_serving_database(args.db, args.query_name, args.limit)
            print(dataframe_to_text_table(frame))
```

```
        elif args.command == "generate-serving-report":
            report_path = generate_serving_report(args.serving_run, args.reports_dir)
            print(f"Serving report written to: {report_path}")
        elif args.command == "serve-api":
            app = create_app(args.db)
            uvicorn.run(app, host=args.host, port=args.port)
        elif args.command == "generate-api-report":
            report_path = generate_api_report(args.db, args.reports_dir)
            print(f"API report written to: {report_path}")
        elif args.command == "serve-dashboard":
            env = dict(os.environ)
            env["MCT_API_URL"] = args.api_url
            dashboard_path = Path(__file__).parent / "dashboard" / "app.py"
            subprocess.run([sys.executable, "-m", "streamlit", "run", str(dashboard_path)],
check=True, env=env)
        else:
            report_path = generate_final_report(args.serving_run, args.reports_dir)
            print(f"Final project report written to: {report_path}")
    except (FileNotFoundError, ValueError, RuntimeError) as exc:
        print(f"Error: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


if __name__ == "__main__":
    main()
'''

src/mobility_control_tower/config.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/config.py
FILE: src/mobility_control_tower/config.py
'''python
"""Load source definitions from YAML configuration."""

from pathlib import Path
from typing import Any

import yaml


DEFAULT_CONFIG_PATH = Path("config/sources.yml")


def load_source(source_id: str, config_path: Path = DEFAULT_CONFIG_PATH) -> dict[str, Any]:
    if not config_path.is_file():
        raise FileNotFoundError(f"Source configuration not found: {config_path}")
    with config_path.open(encoding="utf-8") as handle:
        document = yaml.safe_load(handle) or {}
    sources = document.get("sources", {})
    if source_id not in sources:
        available = ", ".join(sorted(sources)) or "none"
        raise ValueError(f"Unknown source '{source_id}'. Available sources: {available}")
    return dict(sources[source_id])
'''

src/mobility_control_tower/api/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/api/__init__.py
FILE: src/mobility_control_tower/api/__init__.py
'''python
"""Read-only local API for DuckDB serving data products."""

'''

src/mobility_control_tower/api/app.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/api/app.py
FILE: src/mobility_control_tower/api/app.py
'''python
```

```
"""FastAPI application factory."""

from __future__ import annotations

from pathlib import Path

from fastapi import FastAPI

from mobility_control_tower.api.db import validate_database_path
from mobility_control_tower.api.routes import router


def create_app(db_path: str | Path) -> FastAPI:
    resolved = validate_database_path(db_path)
    app = FastAPI(
        title="Mobility Control Tower API",
        description="Local read-only API serving DuckDB data products.",
        version="0.1.0",
    )
    app.state.db_path = resolved
    app.include_router(router)
    return app
```

```
src/mobility_control_tower/api/db.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/api/db.py
FILE: src/mobility_control_tower/api/db.py
```python
"""DuckDB helpers for the read-only local API."""

from __future__ import annotations

from pathlib import Path
from typing import Any

import duckdb


def validate_database_path(db_path: str | Path) -> Path:
    path = Path(db_path)
    if not path.is_file():
        raise FileNotFoundError(f"DuckDB database not found: {path}")
    return path


def _connect(db_path: Path) -> duckdb.DuckDBPyConnection:
    return duckdb.connect(str(db_path), read_only=True)


def list_tables_and_views(db_path: Path) -> tuple[list[str], list[str]]:
    with _connect(db_path) as connection:
        rows = connection.execute(
            """
            SELECT table_name, table_type
            FROM information_schema.tables
            WHERE table_schema = 'main'
            ORDER BY table_name
            """
        ).fetchall()
    tables = [name for name, table_type in rows if table_type == "BASE TABLE"]
    views = [name for name, table_type in rows if table_type == "VIEW"]
    return tables, views


def database_connected(db_path: Path) -> bool:
    try:
        with _connect(db_path) as connection:
```
```

```
            connection.execute("SELECT 1").fetchone()
        return True
    except duckdb.Error:
        return False


def view_exists(db_path: Path, view_name: str) -> bool:
    _, views = list_tables_and_views(db_path)
    return view_name in views


def query_view(db_path: Path, view_name: str, limit: int, filters: dict[str, Any] | None =
None) -> list[dict[str, Any]]:
    filters = filters or {}
    where_clauses: list[str] = []
    params: list[Any] = []
    for column, value in filters.items():
        if value is not None:
            where_clauses.append(f"{column} = ?")
            params.append(value)
    where_sql = f" WHERE {' AND '.join(where_clauses)}" if where_clauses else ""
    params.append(limit)
    with _connect(db_path) as connection:
        try:
            rows = connection.execute(f"SELECT * FROM {view_name}{where_sql} LIMIT ?",
params).fetchdf()
        except duckdb.CatalogException as exc:
            raise ValueError(f"View '{view_name}' is not available in this serving
database") from exc
    return rows.where(rows.notna(), None).to_dict("records")

'''

src/mobility_control_tower/api/report.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/api/report.py
FILE: src/mobility_control_tower/api/report.py
'''python
"""Generate Markdown documentation for the local API."""

from __future__ import annotations

from pathlib import Path

from mobility_control_tower.api.db import list_tables_and_views, query_view,
validate_database_path, view_exists


def _markdown_table(rows: list[dict]) -> str:
    if not rows:
        return "No rows available."
    headers = list(rows[0].keys())
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join("---" for _ in headers) +
" |"]
    for row in rows:
        lines.append("| " + " | ".join(str(row.get(header, "")).replace("|", "\\|") for
header in headers) + " |")
    return "\n".join(lines)


def _sample(db_path: Path, view_name: str, limit: int = 2) -> str:
    if not view_exists(db_path, view_name):
        return "Unavailable in this serving database."
    return _markdown_table(query_view(db_path, view_name, limit))


def _run_id_from_db_path(db_path: Path) -> str:
    return db_path.parent.name
```

```
def generate_api_report(db_path: Path, reports_dir: Path = Path("data/reports")) -> Path:
    resolved = validate_database_path(db_path)
    tables, views = list_tables_and_views(resolved)
    static_endpoints = [
        "GET /static/network-overview",
        "GET /static/top-routes",
        "GET /static/hourly-headway",
        "GET /static/route-types",
    ]
    realtime_endpoints = [
        "GET /realtime/feed-health",
        "GET /realtime/compatibility",
        "GET /realtime/top-delayed-routes",
        "GET /realtime/top-delayed-stops",
    ]
    report = f"""# Mobility Control Tower - Local API Report

## 1. What the API exposes

The API is a local read-only API that serves DuckDB data products as JSON responses. It
exposes static planning views and, when available, GTFS-Realtime snapshot views.

## 2. Database used

- DuckDB path: ‘{resolved}‘
- Tables available: ‘{len(tables)}‘
- Views available: ‘{len(views)}‘

## 3. Available static endpoints

{chr(10).join(f’- ‘{endpoint}‘’ for endpoint in static_endpoints)}

## 4. Available real-time snapshot endpoints

{chr(10).join(f’- ‘{endpoint}‘’ for endpoint in realtime_endpoints)}

Real-time endpoints are snapshot-based and return 404 when the serving database does not
contain real-time snapshot views.

## 5. Example requests

- ‘GET http://127.0.0.1:8000/health‘
- ‘GET http://127.0.0.1:8000/metadata‘
- ‘GET http://127.0.0.1:8000/static/top-routes?limit=5‘
- ‘GET http://127.0.0.1:8000/static/hourly-headway?route_id=line:69&limit=10‘
- ‘GET http://127.0.0.1:8000/realtime/feed-health‘

FastAPI interactive documentation is available at ‘/docs‘ while the local server is running.

## 6. Example response snippets

### Top routes

{_sample(resolved, "v_top_routes_static", 3)}

### Real-time feed health

{_sample(resolved, "v_rt_feed_health", 1)}

### Real-time compatibility

{_sample(resolved, "v_rt_identifier_compatibility", 3)}

## 7. What this proves technically

The project can serve trusted local DuckDB data products through a read-only HTTP API. This
prepares future consumers such as a dashboard, notebook, teacher demo, or later monitoring
interface.
```

```
## 8. Current limitations

- This is not a production API.
- There is no authentication because the server is local-only and read-only in this phase.
- There is no dashboard yet.
- There is no streaming, database server, Docker, cloud service, or real-time monitoring
system.

## 9. Next project step

Use this API as the backend contract for a later dashboard or demonstration interface.
"""
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"api_report_{_run_id_from_db_path(resolved)}.md"
    output.write_text(report, encoding="utf-8")
    return output

'''

src/mobility_control_tower/api/routes.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/api/routes.py
FILE: src/mobility_control_tower/api/routes.py
'''python
"""FastAPI routes for serving DuckDB data products."""

from __future__ import annotations

from pathlib import Path
from typing import Any

from fastapi import APIRouter, HTTPException, Query, Request

from mobility_control_tower.api.db import database_connected, list_tables_and_views,
query_view, view_exists


router = APIRouter()


def _db_path(request: Request) -> Path:
    return request.app.state.db_path


def _limit(value: int, maximum: int) -> int:
    return max(1, min(value, maximum))


def _response(data: list[dict[str, Any]], source: str, notes: list[str] | None = None) ->
dict[str, Any]:
    return {"data": data, "count": len(data), "source": source, "notes": notes or []}


def _query_or_404(db_path: Path, view_name: str, limit: int, filters: dict[str, Any] | None
= None) -> dict[str, Any]:
    if not view_exists(db_path, view_name):
        if view_name.startswith("v_rt_"):
            raise HTTPException(status_code=404, detail="Realtime snapshot data is not
available in this serving database.")
        raise HTTPException(status_code=404, detail=f"Required view '{view_name}' is not
available in this serving database.")
    return _response(query_view(db_path, view_name, limit, filters), view_name)


@router.get("/health")
def health(request: Request) -> dict[str, Any]:
    db_path = _db_path(request)
    return {
```

```
        "status": "ok",
        "service": "Mobility Control Tower API",
        "database_connected": database_connected(db_path),
    }


@router.get("/metadata")
def metadata(request: Request) -> dict[str, Any]:
    db_path = _db_path(request)
    tables, views = list_tables_and_views(db_path)
    return {
        "database_path": str(db_path),
        "available_tables": tables,
        "available_views": views,
        "static_data_available": "v_network_overview" in views and "v_top_routes_static" in
views,
        "realtime_snapshot_available": any(view.startswith("v_rt_") for view in views),
    }


@router.get("/static/network-overview")
def static_network_overview(request: Request, limit: int = Query(default=20, ge=1, le=100))
-> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_network_overview", _limit(limit, 100))


@router.get("/static/top-routes")
def static_top_routes(request: Request, limit: int = Query(default=10, ge=1, le=100)) ->
dict[str, Any]:
    return _query_or_404(_db_path(request), "v_top_routes_static", _limit(limit, 100))


@router.get("/static/hourly-headway")
def static_hourly_headway(
    request: Request,
    route_id: str | None = None,
    service_date: str | None = None,
    limit: int = Query(default=50, ge=1, le=500),
) -> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_route_hourly_headway", _limit(limit, 500),
{"route_id": route_id, "service_date": service_date})


@router.get("/static/route-types")
def static_route_types(
    request: Request,
    service_date: str | None = None,
    limit: int = Query(default=50, ge=1, le=500),
) -> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_route_type_daily_summary", _limit(limit,
500), {"service_date": service_date})


@router.get("/realtime/feed-health")
def realtime_feed_health(request: Request) -> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_rt_feed_health", 10)


@router.get("/realtime/compatibility")
def realtime_compatibility(request: Request) -> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_rt_identifier_compatibility", 10)


@router.get("/realtime/top-delayed-routes")
def realtime_top_delayed_routes(request: Request, limit: int = Query(default=10, ge=1,
le=100)) -> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_rt_top_delayed_routes_snapshot",
_limit(limit, 100))
```

```
@router.get("/realtime/top-delayed-stops")
def realtime_top_delayed_stops(request: Request, limit: int = Query(default=10, ge=1,
le=100)) -> dict[str, Any]:
    return _query_or_404(_db_path(request), "v_rt_top_delayed_stops_snapshot", _limit(limit,
100))
```

```

src/mobility_control_tower/dashboard/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/dashboard/__init__.py
FILE: src/mobility_control_tower/dashboard/__init__.py
'''python
"""Streamlit dashboard for the local academic demo."""

```

```

src/mobility_control_tower/dashboard/api_client.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/dashboard/api_client.p
y
FILE: src/mobility_control_tower/dashboard/api_client.py
'''python
"""Small client for the local Mobility Control Tower API."""

from __future__ import annotations

from typing import Any

import requests


ENDPOINTS = {
    "health": "/health",
    "metadata": "/metadata",
    "network_overview": "/static/network-overview",
    "top_routes": "/static/top-routes",
    "hourly_headway": "/static/hourly-headway",
    "route_types": "/static/route-types",
    "rt_feed_health": "/realtime/feed-health",
    "rt_compatibility": "/realtime/compatibility",
    "rt_top_delayed_routes": "/realtime/top-delayed-routes",
    "rt_top_delayed_stops": "/realtime/top-delayed-stops",
}


def _url(api_url: str, endpoint: str) -> str:
    return api_url.rstrip("/") + endpoint


def get_json(api_url: str, endpoint: str, params: dict[str, Any] | None = None, timeout: int
= 5) -> dict[str, Any]:
    """Call one API endpoint and return either JSON data or a friendly error payload."""
    try:
        response = requests.get(_url(api_url, endpoint), params=params, timeout=timeout)
    except requests.RequestException as exc:
        return {
            "ok": False,
            "error": f"API is not reachable at {api_url}. Start it with 'python -m
mobility_control_tower.cli serve-api ...'. Details: {exc}",
            "data": [],
            "count": 0,
        }
    if response.status_code == 404:
        try:
            detail = response.json().get("detail", "Endpoint unavailable.")
        except ValueError:
            detail = "Endpoint unavailable."
```

```
        return {"ok": False, "error": detail, "data": [], "count": 0}
    if response.status_code >= 400:
        return {"ok": False, "error": f"API returned HTTP {response.status_code}.", "data":
[], "count": 0}
    try:
        payload = response.json()
    except ValueError:
        return {"ok": False, "error": "API returned a non-JSON response.", "data": [],
"count": 0}
    if isinstance(payload, dict):
        payload.setdefault("ok", True)
        return payload
    return {"ok": True, "data": payload, "count": len(payload) if isinstance(payload, list)
else 1}


def fetch_dashboard_data(api_url: str) -> dict[str, dict[str, Any]]:
    """Fetch the small set of payloads used by the dashboard."""
    return {
        "health": get_json(api_url, ENDPOINTS["health"]),
        "metadata": get_json(api_url, ENDPOINTS["metadata"]),
        "network_overview": get_json(api_url, ENDPOINTS["network_overview"], {"limit": 20}),
        "top_routes": get_json(api_url, ENDPOINTS["top_routes"], {"limit": 10}),
        "hourly_headway": get_json(api_url, ENDPOINTS["hourly_headway"], {"limit": 50}),
        "route_types": get_json(api_url, ENDPOINTS["route_types"], {"limit": 50}),
        "rt_feed_health": get_json(api_url, ENDPOINTS["rt_feed_health"]),
        "rt_compatibility": get_json(api_url, ENDPOINTS["rt_compatibility"]),
        "rt_top_delayed_routes": get_json(api_url, ENDPOINTS["rt_top_delayed_routes"],
{"limit": 10}),
        "rt_top_delayed_stops": get_json(api_url, ENDPOINTS["rt_top_delayed_stops"],
{"limit": 10}),
    }

'''

src/mobility_control_tower/dashboard/app.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/dashboard/app.py
FILE: src/mobility_control_tower/dashboard/app.py
'''python
"""Streamlit local demo dashboard."""

from __future__ import annotations

import os
from typing import Any

import pandas as pd
import streamlit as st

from mobility_control_tower.dashboard.api_client import fetch_dashboard_data


DEFAULT_API_URL = "http://127.0.0.1:8000"


def _rows(payload: dict[str, Any]) -> list[dict[str, Any]]:
    data = payload.get("data", [])
    return data if isinstance(data, list) else []


def _table(title: str, payload: dict[str, Any]) -> pd.DataFrame:
    st.subheader(title)
    if not payload.get("ok", True):
        st.warning(payload.get("error", "Data unavailable."))
        return pd.DataFrame()
    frame = pd.DataFrame(_rows(payload))
    if frame.empty:
        st.info("No rows available.")
```

```
    else:
        st.dataframe(frame, use_container_width=True)
    return frame


def main() -> None:
    st.set_page_config(page_title="Mobility Control Tower", layout="wide")
    st.title("Mobility Control Tower - Local Demo")
    st.caption("Local academic MVP: static planning KPIs, GTFS-Realtime snapshot indicators,
DuckDB serving, and read-only API.")

    api_url = st.sidebar.text_input("API URL", value=os.environ.get("MCT_API_URL",
DEFAULT_API_URL))
    st.sidebar.markdown("Start the API before using the dashboard.")

    data = fetch_dashboard_data(api_url)

    st.header("1. Project overview")
    st.write(
        "This dashboard consumes the local FastAPI API and shows queryable data products
from the Mobility Control Tower. "
        "It is a local academic MVP, not a production monitoring system."
    )

    st.header("2. API health")
    health = data["health"]
    if health.get("ok", True):
        st.json(health)
    else:
        st.error(health["error"])

    st.header("3. Static network overview")
    network = _table("Network daily summary", data["network_overview"])
    if not network.empty and "scheduled_trips_count" in network.columns:
        st.line_chart(network.set_index("service_date")["scheduled_trips_count"])

    st.header("4. Top routes by scheduled trips")
    top_routes = _table("Top routes over the static GTFS service period",
data["top_routes"])
    if not top_routes.empty and {"route_short_name",
"total_scheduled_trips"}.issubset(top_routes.columns):
        st.bar_chart(top_routes.set_index("route_short_name")["total_scheduled_trips"])

    st.header("5. Route type summary")
    _table("Route type daily summary sample", data["route_types"])

    st.header("6. Planned hourly headway sample")
    _table("Planned headway approximation by route/hour", data["hourly_headway"])

    st.header("7. Realtime snapshot feed health")
    st.write("These GTFS-Realtime snapshot values were observed at fetch time.")
    _table("Feed health", data["rt_feed_health"])

    st.header("8. Realtime identifier compatibility")
    compatibility = _table("Static/live identifier compatibility", data["rt_compatibility"])
    if not compatibility.empty and {"identifier_type",
"match_percentage"}.issubset(compatibility.columns):
        st.bar_chart(compatibility.set_index("identifier_type")["match_percentage"])

    st.header("9. Top delayed routes snapshot")
    delayed_routes = _table("Routes with highest average observed delay in one snapshot",
data["rt_top_delayed_routes"])
    if not delayed_routes.empty and {"route_short_name",
"avg_delay_seconds"}.issubset(delayed_routes.columns):
        st.bar_chart(delayed_routes.set_index("route_short_name")["avg_delay_seconds"])

    st.header("10. Limitations and next steps")
    st.write(
        "- These are static planning KPIs and GTFS-Realtime snapshot indicators.\n"
```

```
        "- The dashboard is read-only and local.\n"
        "- It is not a production monitoring system and does not implement streaming.\n"
        "- A future phase could add a richer dashboard or repeated snapshot collection."
    )


if __name__ == "__main__":
    main()

'''

src/mobility_control_tower/ingestion/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/ingestion/__init__.py
FILE: src/mobility_control_tower/ingestion/__init__.py
'''python
"""Raw data ingestion."""
'''

src/mobility_control_tower/ingestion/gtfs_raw.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/ingestion/gtfs_raw.py
FILE: src/mobility_control_tower/ingestion/gtfs_raw.py
'''python
"""Preserve a GTFS archive and capture its provenance metadata."""

from __future__ import annotations

import hashlib
import json
import shutil
import tempfile
import zipfile
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import requests


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def _validate_zip(path: Path) -> None:
    if not path.is_file():
        raise FileNotFoundError(f"GTFS ZIP not found: {path}")
    if not zipfile.is_zipfile(path):
        raise ValueError(f"Input is not a valid ZIP archive: {path}")


def preserve_gtfs_zip(
    input_zip: Path,
    source_id: str,
    source: dict[str, Any],
    raw_root: Path = Path("data/raw"),
    ingestion_method: str = "local",
    ingested_at: datetime | None = None,
) -> Path:
    """Copy an archive unchanged to a new raw run and write metadata beside it."""
    _validate_zip(input_zip)
    timestamp = ingested_at or datetime.now(timezone.utc)
    timestamp = timestamp.astimezone(timezone.utc)
    run_id = timestamp.strftime("%Y-%m-%d_%H%M%S")
    run_dir = raw_root / source_id / run_id
```

```
    run_dir.mkdir(parents=True, exist_ok=False)

    output_name = str(source.get("output_filename", input_zip.name))
    raw_zip = run_dir / output_name
    try:
        shutil.copyfile(input_zip, raw_zip)
        metadata = {
            "source_id": source_id,
            "source_name": source["name"],
            "source_page_url": source["source_page_url"],
            "licence": source["licence"],
            "ingestion_timestamp": timestamp.isoformat(),
            "ingestion_method": ingestion_method,
            "original_filename": input_zip.name,
            "raw_filename": raw_zip.name,
            "file_size_bytes": raw_zip.stat().st_size,
            "sha256": sha256_file(raw_zip),
        }
        (run_dir / "metadata.json").write_text(
            json.dumps(metadata, indent=2, ensure_ascii=False) + "\n", encoding="utf-8"
        )
    except Exception:
        shutil.rmtree(run_dir)
        raise
    return run_dir


def download_and_preserve_gtfs(
    source_id: str,
    source: dict[str, Any],
    raw_root: Path = Path("data/raw"),
) -> Path:
    """Download the configured archive to a temporary file, then preserve it."""
    download_url = source.get("download_url")
    if not download_url:
        raise ValueError(f"No download URL configured for source '{source_id}'")
    with tempfile.TemporaryDirectory() as temp_dir:
        temporary_zip = Path(temp_dir) / str(source.get("output_filename", "gtfs.zip"))
        with requests.get(str(download_url), stream=True, timeout=(10, 120)) as response:
            response.raise_for_status()
            with temporary_zip.open("wb") as handle:
                for chunk in response.iter_content(chunk_size=1024 * 1024):
                    if chunk:
                        handle.write(chunk)
        return preserve_gtfs_zip(
            temporary_zip, source_id, source, raw_root, ingestion_method="download"
        )
'''

src/mobility_control_tower/metrics/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/metrics/__init__.py
FILE: src/mobility_control_tower/metrics/__init__.py
'''python
"""Gold metrics derived from static GTFS schedules."""
'''

src/mobility_control_tower/metrics/gtfs_kpis.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/metrics/gtfs_kpis.py
FILE: src/mobility_control_tower/metrics/gtfs_kpis.py
'''python
"""Build explainable planning KPIs from silver GTFS CSV tables."""

from __future__ import annotations

import json
import shutil
from datetime import datetime, timezone
```

```
from pathlib import Path
from typing import Any

import pandas as pd

from mobility_control_tower.transformations.gtfs_silver import parse_gtfs_time


WEEKDAYS = ("monday", "tuesday", "wednesday", "thursday", "friday", "saturday", "sunday")
OUTPUT_COLUMNS = {
    "route_daily_trips": ["service_date", "route_id", "route_short_name", "route_long_name",
"route_type", "scheduled_trips_count"],
    "route_hourly_departures": ["service_date", "route_id", "route_short_name",
"route_long_name", "departure_hour", "scheduled_departures_count"],
    "stop_daily_departures": ["service_date", "stop_id", "stop_name",
"scheduled_departures_count"],
    "network_daily_summary": ["service_date", "active_routes_count",
"scheduled_trips_count", "scheduled_stop_departures_count", "active_stops_count"],
    "route_period_summary": ["route_id", "route_short_name", "route_long_name",
"route_type", "active_service_days", "total_scheduled_trips",
"average_trips_per_active_day", "max_daily_trips", "first_service_date",
"last_service_date"],
    "route_hourly_headway": ["service_date", "route_id", "route_short_name",
"route_long_name", "departure_hour", "scheduled_departures_count",
"planned_headway_minutes"],
    "route_type_daily_summary": ["service_date", "route_type", "route_type_label",
"active_routes_count", "scheduled_trips_count", "scheduled_stop_departures_count"],
    "busiest_route_day": ["service_date", "route_id", "route_short_name", "route_long_name",
"scheduled_trips_count", "rank"],
    "busiest_stop_day": ["service_date", "stop_id", "stop_name",
"scheduled_departures_count", "rank"],
}

ROUTE_TYPE_LABELS = {
    "0": "Tram",
    "1": "Subway/Metro",
    "2": "Rail",
    "3": "Bus",
    "4": "Ferry",
    "5": "Cable tram",
    "6": "Aerial lift",
    "7": "Funicular",
}


def _read_csv(path: Path) -> pd.DataFrame:
    return pd.read_csv(path, dtype=str, keep_default_na=False)


def _parse_date_series(values: pd.Series) -> pd.Series:
    compact = values.astype(str).str.strip()
    parsed = pd.to_datetime(compact, format="%Y%m%d", errors="coerce")
    iso_parsed = pd.to_datetime(compact.where(parsed.isna()), format="%Y-%m-%d",
errors="coerce")
    return parsed.fillna(iso_parsed)


def expand_service_dates(
    calendar: pd.DataFrame | None,
    calendar_dates: pd.DataFrame | None,
) -> pd.DataFrame:
    """Expand regular service and apply additions/removals, returning unique service
dates."""
    services: set[tuple[str, str]] = set()
    if calendar is not None and not calendar.empty:
        required = {"service_id", "start_date", "end_date", *WEEKDAYS}
        missing = sorted(required - set(calendar.columns))
        if missing:
            raise ValueError(f"calendar.csv is missing columns required for date expansion:
```

```
{', '.join(missing)}")
        for row in calendar.to_dict("records"):
            start = pd.to_datetime(str(row["start_date"]), format="%Y%m%d", errors="coerce")
            end = pd.to_datetime(str(row["end_date"]), format="%Y%m%d", errors="coerce")
            if pd.isna(start) or pd.isna(end) or end < start:
                continue
            active = {index for index, weekday in enumerate(WEEKDAYS) if
str(row.get(weekday, "0")) == "1"}
            for date in pd.date_range(start, end, freq="D"):
                if date.weekday() in active:
                    services.add((str(row["service_id"]), date.date().isoformat()))

    if calendar_dates is not None and not calendar_dates.empty:
        required = {"service_id", "date", "exception_type"}
        missing = sorted(required - set(calendar_dates.columns))
        if missing:
            raise ValueError(f"calendar_dates.csv is missing columns required for date
expansion: {', '.join(missing)}")
        dates = _parse_date_series(calendar_dates["date"])
        for row, parsed_date in zip(calendar_dates.to_dict("records"), dates):
            if pd.isna(parsed_date):
                continue
            key = (str(row["service_id"]), parsed_date.date().isoformat())
            if str(row["exception_type"]) == "1":
                services.add(key)
            elif str(row["exception_type"]) == "2":
                services.discard(key)

    return pd.DataFrame(sorted(services), columns=["service_id", "service_date"])


def _require_columns(table: str, frame: pd.DataFrame, columns: set[str]) -> None:
    missing = sorted(columns - set(frame.columns))
    if missing:
        raise ValueError(f"Silver table {table}.csv is missing required columns: {',
'.join(missing)}")


def _route_details(routes: pd.DataFrame) -> pd.DataFrame:
    details = routes.copy()
    for column in ("route_short_name", "route_long_name", "route_type"):
        if column not in details.columns:
            details[column] = ""
    return details[["route_id", "route_short_name", "route_long_name", "route_type"]]


def compute_gold_tables(tables: dict[str, pd.DataFrame]) -> dict[str, pd.DataFrame]:
    """Compute KPI tables from already loaded silver tables."""
    calendar = tables.get("calendar")
    calendar_dates = tables.get("calendar_dates")
    if calendar is None and calendar_dates is None:
        raise ValueError("Gold KPIs require calendar.csv or calendar_dates.csv")
    for table in ("routes", "trips", "stop_times", "stops"):
        if table not in tables:
            raise ValueError(f"Gold KPIs require missing silver table: {table}.csv")

    routes, trips, stop_times, stops = (tables[name].copy() for name in ("routes", "trips",
"stop_times", "stops"))
    _require_columns("routes", routes, {"route_id"})
    _require_columns("trips", trips, {"trip_id", "route_id", "service_id"})
    _require_columns("stop_times", stop_times, {"trip_id", "stop_id", "departure_time",
"stop_sequence"})
    _require_columns("stops", stops, {"stop_id", "stop_name"})
    service_dates = expand_service_dates(calendar, calendar_dates)
    if service_dates.empty and (len(trips) or len(stop_times)):
        raise ValueError("Service-date expansion produced no dates for non-empty trip data")

    route_info = _route_details(routes)
    trip_base = trips[["trip_id", "route_id", "service_id"]]
```

```
    trips_by_service = trip_base.groupby(["service_id", "route_id"],
as_index=False).size().rename(columns={"size": "scheduled_trips_count"})
    route_daily = service_dates.merge(trips_by_service, on="service_id",
how="inner").merge(route_info, on="route_id", how="left")
    route_daily = route_daily.groupby(["service_date", "route_id", "route_short_name",
"route_long_name", "route_type"], as_index=False,
dropna=False)["scheduled_trips_count"].sum()
    route_daily =
route_daily[OUTPUT_COLUMNS["route_daily_trips"]].sort_values(["service_date",
"route_id"]).reset_index(drop=True)

    stop_times["_stop_sequence"] = pd.to_numeric(stop_times["stop_sequence"],
errors="coerce")
    first_stops = stop_times.sort_values(["trip_id", "_stop_sequence"],
na_position="last").drop_duplicates("trip_id", keep="first")
    if "departure_time_seconds" in first_stops.columns:
        seconds = pd.to_numeric(first_stops["departure_time_seconds"], errors="coerce")
    else:
        seconds = first_stops["departure_time"].map(parse_gtfs_time)
    first_stops = first_stops.assign(departure_hour=(seconds // 3600).astype("Int64"))
    first_stops = first_stops.dropna(subset=["departure_hour"])
    hourly_base = first_stops[["trip_id", "departure_hour"]].merge(trip_base, on="trip_id",
how="inner")
    hourly_service = hourly_base.groupby(["service_id", "route_id", "departure_hour"],
as_index=False).size().rename(columns={"size": "scheduled_departures_count"})
    route_hourly = service_dates.merge(hourly_service, on="service_id",
how="inner").merge(route_info, on="route_id", how="left")
    route_hourly["departure_hour"] = route_hourly["departure_hour"].astype(int)
    route_hourly = route_hourly.groupby(["service_date", "route_id", "route_short_name",
"route_long_name", "departure_hour"], as_index=False,
dropna=False)["scheduled_departures_count"].sum()
    route_hourly =
route_hourly[OUTPUT_COLUMNS["route_hourly_departures"]].sort_values(["service_date",
"route_id", "departure_hour"]).reset_index(drop=True)

    departures = stop_times.loc[stop_times["departure_time"].str.strip().ne(""), ["trip_id",
"stop_id"]]
    stop_service = departures.merge(trip_base[["trip_id", "service_id"]], on="trip_id",
how="inner").groupby(["service_id", "stop_id"],
as_index=False).size().rename(columns={"size": "scheduled_departures_count"})
    stop_names = stops[["stop_id", "stop_name"]].drop_duplicates("stop_id")
    stop_daily = service_dates.merge(stop_service, on="service_id",
how="inner").merge(stop_names, on="stop_id", how="left")
    stop_daily = stop_daily.groupby(["service_date", "stop_id", "stop_name"],
as_index=False, dropna=False)["scheduled_departures_count"].sum()
    stop_daily =
stop_daily[OUTPUT_COLUMNS["stop_daily_departures"]].sort_values(["service_date",
"stop_id"]).reset_index(drop=True)

    route_summary = route_daily.groupby("service_date",
as_index=False).agg(active_routes_count=("route_id", "nunique"),
scheduled_trips_count=("scheduled_trips_count", "sum"))
    stop_summary = stop_daily.groupby("service_date",
as_index=False).agg(scheduled_stop_departures_count=("scheduled_departures_count", "sum"),
active_stops_count=("stop_id", "nunique"))
    network_daily = route_summary.merge(stop_summary, on="service_date",
how="outer").fillna(0)
    for column in OUTPUT_COLUMNS["network_daily_summary"][1:]:
        network_daily[column] = network_daily[column].astype(int)
    network_daily =
network_daily[OUTPUT_COLUMNS["network_daily_summary"]].sort_values("service_date").reset_ind
ex(drop=True)

    route_period = route_daily.groupby(["route_id", "route_short_name", "route_long_name",
"route_type"], as_index=False, dropna=False).agg(
        active_service_days=("service_date", "nunique"),
        total_scheduled_trips=("scheduled_trips_count", "sum"),
        average_trips_per_active_day=("scheduled_trips_count", "mean"),
```

```
        max_daily_trips=("scheduled_trips_count", "max"),
        first_service_date=("service_date", "min"),
        last_service_date=("service_date", "max"),
    )
    route_period["average_trips_per_active_day"] =
route_period["average_trips_per_active_day"].round(2)
    for column in ("active_service_days", "total_scheduled_trips", "max_daily_trips"):
        route_period[column] = route_period[column].astype(int)
    route_period =
route_period[OUTPUT_COLUMNS["route_period_summary"]].sort_values(["total_scheduled_trips",
"route_id"], ascending=[False, True]).reset_index(drop=True)

    route_hourly_headway = route_hourly.copy()
    route_hourly_headway["planned_headway_minutes"] = 60 /
route_hourly_headway["scheduled_departures_count"].where(route_hourly_headway["scheduled_dep
artures_count"] > 0)
    route_hourly_headway["planned_headway_minutes"] =
route_hourly_headway["planned_headway_minutes"].round(2)
    route_hourly_headway =
route_hourly_headway[OUTPUT_COLUMNS["route_hourly_headway"]].sort_values(["service_date",
"route_id", "departure_hour"]).reset_index(drop=True)

    trip_departures = departures.merge(trip_base, on="trip_id",
how="inner").merge(route_info[["route_id", "route_type"]], on="route_id", how="left")
    route_type_stop_service = trip_departures.groupby(["service_id", "route_type"],
as_index=False, dropna=False).size().rename(columns={"size":
"scheduled_stop_departures_count"})
    route_type_stop_daily = service_dates.merge(route_type_stop_service, on="service_id",
how="inner")
    route_type_stop_daily = route_type_stop_daily.groupby(["service_date", "route_type"],
as_index=False, dropna=False)["scheduled_stop_departures_count"].sum()
    route_type_route_daily = route_daily.groupby(["service_date", "route_type"],
as_index=False, dropna=False).agg(
        active_routes_count=("route_id", "nunique"),
        scheduled_trips_count=("scheduled_trips_count", "sum"),
    )
    route_type_daily = route_type_route_daily.merge(route_type_stop_daily,
on=["service_date", "route_type"], how="left").fillna({"scheduled_stop_departures_count":
0})
    route_type_daily["route_type_label"] =
route_type_daily["route_type"].astype(str).map(ROUTE_TYPE_LABELS).fillna("Unknown")
    for column in ("active_routes_count", "scheduled_trips_count",
"scheduled_stop_departures_count"):
        route_type_daily[column] = route_type_daily[column].astype(int)
    route_type_daily =
route_type_daily[OUTPUT_COLUMNS["route_type_daily_summary"]].sort_values(["service_date",
"route_type"]).reset_index(drop=True)

    busiest_route_day = route_daily.sort_values(["scheduled_trips_count", "service_date",
"route_id"], ascending=[False, True, True]).head(50).copy()
    busiest_route_day["rank"] = range(1, len(busiest_route_day) + 1)
    busiest_route_day =
busiest_route_day[OUTPUT_COLUMNS["busiest_route_day"]].reset_index(drop=True)

    busiest_stop_day = stop_daily.sort_values(["scheduled_departures_count", "service_date",
"stop_id"], ascending=[False, True, True]).head(50).copy()
    busiest_stop_day["rank"] = range(1, len(busiest_stop_day) + 1)
    busiest_stop_day =
busiest_stop_day[OUTPUT_COLUMNS["busiest_stop_day"]].reset_index(drop=True)

    return {
        "route_daily_trips": route_daily,
        "route_hourly_departures": route_hourly,
        "stop_daily_departures": stop_daily,
        "network_daily_summary": network_daily,
        "route_period_summary": route_period,
        "route_hourly_headway": route_hourly_headway,
        "route_type_daily_summary": route_type_daily,
        "busiest_route_day": busiest_route_day,
```

```
        "busiest_stop_day": busiest_stop_day,
    }


def build_gold(silver_run: Path, gold_root: Path = Path("data/gold")) -> Path:
    if not silver_run.is_dir():
        raise FileNotFoundError(f"Silver run directory not found: {silver_run}")
    table_names = ("routes", "trips", "stop_times", "stops", "calendar", "calendar_dates")
    tables = {name: _read_csv(silver_run / f"{name}.csv") for name in table_names if
(silver_run / f"{name}.csv").is_file()}
    source_id = silver_run.parent.name
    output_dir = gold_root / source_id / silver_run.name
    output_dir.mkdir(parents=True, exist_ok=False)
    try:
        outputs = compute_gold_tables(tables)
        manifest_tables: dict[str, dict[str, Any]] = {}
        for name, frame in outputs.items():
            path = output_dir / f"{name}.csv"
            frame.to_csv(path, index=False)
            manifest_tables[name] = {"file": path.name, "row_count": len(frame), "columns":
list(frame.columns)}
        manifest = {
            "source_silver_run": str(silver_run),
            "generated_timestamp": datetime.now(timezone.utc).isoformat(),
            "tables_created": manifest_tables,
            "kpi_definitions": {
                "route_daily_trips": {"description": "Scheduled trips per service date and
route.", "static_planning_only": True},
                "route_hourly_departures": {"description": "Scheduled trip starts per
service date, route, and service-day hour.", "static_planning_only": True},
                "stop_daily_departures": {"description": "Scheduled departures per service
date and stop.", "static_planning_only": True},
                "network_daily_summary": {"description": "Daily active routes, scheduled
trips, scheduled stop departures, and active stops.", "static_planning_only": True},
                "route_period_summary": {"description": "Route totals and averages over the
GTFS service period.", "static_planning_only": True},
                "route_hourly_headway": {"description": "Approximate planned headway in
minutes from scheduled trip starts per route/hour.", "static_planning_only": True},
                "route_type_daily_summary": {"description": "Daily scheduled activity
summarized by GTFS route type.", "static_planning_only": True},
                "busiest_route_day": {"description": "Top 50 route/day combinations by
scheduled trips.", "static_planning_only": True},
                "busiest_stop_day": {"description": "Top 50 stop/day combinations by
scheduled departures.", "static_planning_only": True},
            },
            "important_assumptions": [
                "calendar weekday service is adjusted by calendar_dates additions and
removals.",
                "A trip start is the row with the minimum numeric stop_sequence for that
trip.",
                "Hours above 23 remain service-day hours rather than wrapping to the next
date.",
                "planned_headway_minutes is 60 divided by scheduled departures in that
route/hour.",
            ],
            "limitations": [
                "Metrics describe the published static schedule, not actual operations or
real-time reliability.",
                "Frequency-based service is not expanded into synthetic trips.",
                "Missing or invalid departure times are excluded from departure counts.",
                "Planned headway is an hourly approximation, not real passenger waiting
time.",
            ],
        }
        (output_dir / "gold_manifest.json").write_text(json.dumps(manifest, indent=2,
ensure_ascii=False) + "\n", encoding="utf-8")
    except Exception:
        shutil.rmtree(output_dir)
        raise
```

```
    return output_dir
'''

src/mobility_control_tower/profiling/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/profiling/__init__.py
FILE: src/mobility_control_tower/profiling/__init__.py
'''python
"""Data profiling."""
'''

src/mobility_control_tower/profiling/gtfs_profile.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/profiling/gtfs_profile
.py
FILE: src/mobility_control_tower/profiling/gtfs_profile.py
'''python
"""Create simple JSON and Markdown profiles of preserved GTFS ZIP files."""

from __future__ import annotations

import csv
import io
import json
import zipfile
from pathlib import Path, PurePosixPath
from typing import Any, Iterable


IMPORTANT_FILES = (
    "agency.txt",
    "stops.txt",
    "routes.txt",
    "trips.txt",
    "stop_times.txt",
    "calendar.txt",
    "calendar_dates.txt",
)
COUNT_FILES = {
    "routes.txt": "number_of_routes",
    "stops.txt": "number_of_stops",
    "trips.txt": "number_of_trips",
    "stop_times.txt": "number_of_stop_times",
}


def _txt_members(archive: zipfile.ZipFile) -> dict[str, str]:
    members: dict[str, str] = {}
    for name in archive.namelist():
        if not name.endswith("/") and name.lower().endswith(".txt"):
            members.setdefault(PurePosixPath(name).name.lower(), name)
    return members


def _read_rows(archive: zipfile.ZipFile, member: str) -> tuple[list[str], list[dict[str,
str]]]:
    with archive.open(member) as binary:
        text = io.TextIOWrapper(binary, encoding="utf-8-sig", newline="")
        reader = csv.DictReader(text)
        rows = list(reader)
        return list(reader.fieldnames or []), rows


def _date_range(values: Iterable[str]) -> dict[str, str | None]:
    dates = sorted(value for value in values if value and len(value) == 8 and
value.isdigit())
    return {"start_date": dates[0] if dates else None, "end_date": dates[-1] if dates else
None}
```

```
def build_profile(gtfs_zip: Path, run_id: str) -> dict[str, Any]:
    if not zipfile.is_zipfile(gtfs_zip):
        raise ValueError(f"Not a valid GTFS ZIP archive: {gtfs_zip}")
    with zipfile.ZipFile(gtfs_zip) as archive:
        all_files = sorted(name for name in archive.namelist() if not name.endswith("/"))
        members = _txt_members(archive)
        file_profiles: dict[str, dict[str, Any]] = {}
        rows_by_file: dict[str, list[dict[str, str]]] = {}
        for base_name, member in sorted(members.items()):
            columns, rows = _read_rows(archive, member)
            file_profiles[base_name] = {"archive_path": member, "row_count": len(rows),
"columns": columns}
            rows_by_file[base_name] = rows

    presence = {name: name in members for name in IMPORTANT_FILES}
    profile: dict[str, Any] = {
        "run_id": run_id,
        "gtfs_zip": gtfs_zip.name,
        "files_in_zip": all_files,
        "important_files_present": presence,
        "missing_important_files": [name for name, present in presence.items() if not
present],
        "files": file_profiles,
    }
    for filename, metric in COUNT_FILES.items():
        profile[metric] = file_profiles.get(filename, {}).get("row_count")

    date_values: list[str] = []
    for row in rows_by_file.get("calendar.txt", []):
        date_values.extend((row.get("start_date", ""), row.get("end_date", "")))
    date_values.extend(row.get("date", "") for row in rows_by_file.get("calendar_dates.txt",
[]))
    profile["service_date_range"] = _date_range(date_values)
    return profile


def _markdown(profile: dict[str, Any]) -> str:
    lines = [
        f"# GTFS profile: {profile['run_id']}", "",
        f"Archive: `{profile['gtfs_zip']}`", "",
        "## Summary", "",
        "| Metric | Value |", "|---|---:|",
    ]
    for metric in ("number_of_routes", "number_of_stops", "number_of_trips",
"number_of_stop_times"):
        value = profile[metric] if profile[metric] is not None else "not available"
        lines.append(f"| {metric.replace('_', ' ').title()} | {value} |")
    date_range = profile["service_date_range"]
    lines.extend([
        f"| Service start date | {date_range['start_date'] or 'not available'} |",
        f"| Service end date | {date_range['end_date'] or 'not available'} |", "",
        "## Important files", "", "| File | Present | Rows | Columns |",
"|---|---|---:|---|",
    ])
    for name, present in profile["important_files_present"].items():
        details = profile["files"].get(name, {})
        columns = ", ".join(details.get("columns", [])) or "?"
        rows = details.get("row_count", "?")
        lines.append(f"| `{name}` | {'yes' if present else 'no'} | {rows} | {columns} |")
    missing = profile["missing_important_files"]
    lines.extend(["", "## Missing important files", "", ", ".join(f"`{name}`" for name in
missing) if missing else "None.", ""])
    return "\n".join(lines)


def profile_raw_run(raw_run: Path, reports_dir: Path = Path("data/reports")) -> tuple[Path,
Path]:
    if not raw_run.is_dir():
```

```
        raise FileNotFoundError(f"Raw run directory not found: {raw_run}")
    zip_files = sorted(raw_run.glob("*.zip"))
    if len(zip_files) != 1:
        raise ValueError(f"Expected exactly one ZIP in {raw_run}, found {len(zip_files)}")
    profile = build_profile(zip_files[0], raw_run.name)
    reports_dir.mkdir(parents=True, exist_ok=True)
    json_path = reports_dir / f"gtfs_profile_{raw_run.name}.json"
    markdown_path = reports_dir / f"gtfs_profile_{raw_run.name}.md"
    json_path.write_text(json.dumps(profile, indent=2, ensure_ascii=False) + "\n",
encoding="utf-8")
    markdown_path.write_text(_markdown(profile), encoding="utf-8")
    return json_path, markdown_path
'''

src/mobility_control_tower/quality/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/quality/__init__.py
FILE: src/mobility_control_tower/quality/__init__.py
'''python
"""Basic GTFS data-quality checks."""
'''

src/mobility_control_tower/quality/gtfs_quality.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/quality/gtfs_quality.p
y
FILE: src/mobility_control_tower/quality/gtfs_quality.py
'''python
"""Understandable, intentionally limited validation of silver GTFS CSV tables."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterable

from mobility_control_tower.transformations.gtfs_silver import parse_gtfs_time,
read_csv_table


TABLES = ("agency", "stops", "routes", "trips", "stop_times", "calendar", "calendar_dates")
REQUIRED_COLUMNS = {
    "agency": ("agency_name", "agency_url", "agency_timezone"),
    "stops": ("stop_id", "stop_name", "stop_lat", "stop_lon"),
    "routes": ("route_id", "route_type"),
    "trips": ("route_id", "service_id", "trip_id"),
    "stop_times": ("trip_id", "arrival_time", "departure_time", "stop_id", "stop_sequence"),
    "calendar": ("service_id", "start_date", "end_date"),
    "calendar_dates": ("service_id", "date", "exception_type"),
}
KEY_COLUMNS = {
    "agency": ("agency_name", "agency_url", "agency_timezone"),
    "stops": ("stop_id", "stop_name", "stop_lat", "stop_lon"),
    "routes": ("route_id", "route_type"),
    "trips": ("route_id", "service_id", "trip_id"),
    "stop_times": ("trip_id", "stop_id", "stop_sequence"),
    "calendar": ("service_id", "start_date", "end_date"),
    "calendar_dates": ("service_id", "date", "exception_type"),
}
PRIMARY_IDS = {"stops": "stop_id", "routes": "route_id", "trips": "trip_id"}
VALID_ROUTE_TYPES = {str(value) for value in range(8)} | {"11", "12"}


def _check(name: str, status: str, table: str, count: int, explanation: str) -> dict[str,
Any]:
    return {"check_name": name, "status": status, "table": table, "problem_count": count,
"explanation": explanation}
```

```
def _status(count: int, problem_status: str = "FAIL") -> str:
    return problem_status if count else "PASS"


def _duplicate_count(rows: list[dict[str, str]], column: str) -> int:
    values = [row.get(column, "") for row in rows if row.get(column, "")]
    return len(values) - len(set(values))


def _missing_count(rows: list[dict[str, str]], columns: Iterable[str]) -> int:
    return sum(1 for row in rows for column in columns if not row.get(column, ""))


def _invalid_number(value: str, minimum: float, maximum: float) -> bool:
    try:
        number = float(value)
        return not minimum <= number <= maximum
    except (TypeError, ValueError):
        return bool(value)


def validate_tables(tables: dict[str, tuple[list[str], list[dict[str, str]]]]) -> dict[str,
Any]:
    checks: list[dict[str, Any]] = []
    for table in ("agency", "stops", "routes", "trips", "stop_times"):
        missing = int(table not in tables)
        checks.append(_check(f"{table}_table_present", _status(missing), table, missing,
f"Silver {table} table must exist."))
    schedules_missing = int("calendar" not in tables and "calendar_dates" not in tables)
    checks.append(_check("service_calendar_present", _status(schedules_missing),
"calendar/calendar_dates", schedules_missing, "At least one service calendar table must
exist."))

    row_counts = {table: len(rows) for table, (_, rows) in tables.items()}
    for table, (columns, rows) in tables.items():
        expected = REQUIRED_COLUMNS.get(table, ())
        missing_columns = [column for column in expected if column not in columns]
        if table == "routes" and "route_short_name" not in columns and "route_long_name" not
in columns:
            missing_columns.append("route_short_name or route_long_name")
        checks.append(_check(f"{table}_required_columns", _status(len(missing_columns)),
table, len(missing_columns), "Missing: " + ", ".join(missing_columns) if missing_columns
else "All required columns are present."))
        available_keys = [column for column in KEY_COLUMNS.get(table, ()) if column in
columns]
        missing_values = _missing_count(rows, available_keys)
        checks.append(_check(f"{table}_missing_key_values", _status(missing_values), table,
missing_values, "Counts empty values in available key columns."))

    for table, column in PRIMARY_IDS.items():
        if table in tables and column in tables[table][0]:
            count = _duplicate_count(tables[table][1], column)
            checks.append(_check(f"duplicate_{column}", _status(count), table, count,
f"Duplicate non-empty {column} values."))

    if "stops" in tables:
        columns, rows = tables["stops"]
        for column, minimum, maximum in (("stop_lat", -90, 90), ("stop_lon", -180, 180)):
            if column in columns:
                count = sum(_invalid_number(row.get(column, ""), minimum, maximum) for row
in rows)
                checks.append(_check(f"invalid_{column}", _status(count), "stops", count,
f"Values must be numeric and between {minimum} and {maximum}."))

    if "routes" in tables and "route_type" in tables["routes"][0]:
        count = sum(bool(row.get("route_type")) and row["route_type"] not in
VALID_ROUTE_TYPES for row in tables["routes"][1])
        checks.append(_check("invalid_route_type", _status(count, "WARN"), "routes", count,
```

```
"Expected common GTFS route_type values 0-7, 11, or 12."))

    if "stop_times" in tables:
        columns, rows = tables["stop_times"]
        for column in ("arrival_time", "departure_time"):
            if column in columns:
                count = sum(bool(row.get(column)) and parse_gtfs_time(row[column]) is None
for row in rows)
                checks.append(_check(f"invalid_{column}_format", _status(count),
"stop_times", count, "Expected H+:MM:SS with minutes and seconds from 00 to 59."))

    def references(child: str, child_column: str, parent: str, parent_column: str) -> None:
        if child not in tables or parent not in tables or child_column not in
tables[child][0] or parent_column not in tables[parent][0]:
            return
        parent_ids = {row.get(parent_column, "") for row in tables[parent][1] if
row.get(parent_column, "")}
        count = sum(bool(row.get(child_column)) and row[child_column] not in parent_ids for
row in tables[child][1])
        checks.append(_check(f"{child}_unknown_{child_column}", _status(count), child,
count, f"Values must reference {parent}.{parent_column}."))

    references("stop_times", "trip_id", "trips", "trip_id")
    references("stop_times", "stop_id", "stops", "stop_id")
    references("trips", "route_id", "routes", "route_id")

    if "trips" in tables and "service_id" in tables["trips"][0]:
        service_ids: set[str] = set()
        for table in ("calendar", "calendar_dates"):
            if table in tables and "service_id" in tables[table][0]:
                service_ids.update(row.get("service_id", "") for row in tables[table][1] if
row.get("service_id", ""))
        count = sum(bool(row.get("service_id")) and row["service_id"] not in service_ids for
row in tables["trips"][1])
        checks.append(_check("trips_unknown_service_id", _status(count), "trips", count,
"service_id must appear in calendar or calendar_dates."))

    overall = "FAIL" if any(check["status"] == "FAIL" for check in checks) else "WARN" if
any(check["status"] == "WARN" for check in checks) else "PASS"
    return {"overall_status": overall, "row_counts": row_counts, "checks": checks}


def _markdown(report: dict[str, Any]) -> str:
    lines = [f"# GTFS quality report: {report['run_id']}", "", f"Overall status:
**{report['overall_status']}**", "", "## Row counts", "", "| Table | Rows |", "|---|---:|"]
    lines.extend(f"| `{table}` | {count} |" for table, count in
sorted(report["row_counts"].items()))
    lines.extend(["", "## Checks", "", "| Status | Check | Table | Problems | Explanation
|", "|---|---|---|---:|---|"])
    for check in report["checks"]:
        explanation = check["explanation"].replace("|", "\\|")
        lines.append(f"| {check['status']} | `{check['check_name']}` | `{check['table']}` |
{check['problem_count']} | {explanation} |")
    return "\n".join(lines) + "\n"


def validate_silver_run(silver_run: Path, reports_dir: Path = Path("data/reports")) ->
tuple[Path, Path]:
    if not silver_run.is_dir():
        raise FileNotFoundError(f"Silver run directory not found: {silver_run}")
    tables = {table: read_csv_table(silver_run / f"{table}.csv") for table in TABLES if
(silver_run / f"{table}.csv").is_file()}
    report = validate_tables(tables)
    report.update({"run_id": silver_run.name, "silver_run": str(silver_run),
"generated_timestamp": datetime.now(timezone.utc).isoformat()})
    reports_dir.mkdir(parents=True, exist_ok=True)
    json_path = reports_dir / f"gtfs_quality_{silver_run.name}.json"
    markdown_path = reports_dir / f"gtfs_quality_{silver_run.name}.md"
    json_path.write_text(json.dumps(report, indent=2, ensure_ascii=False) + "\n",
```

```
encoding="utf-8")
    markdown_path.write_text(_markdown(report), encoding="utf-8")
    return json_path, markdown_path
'''

src/mobility_control_tower/realtime/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/__init__.py
FILE: src/mobility_control_tower/realtime/__init__.py
'''python
"""GTFS-Realtime snapshot exploration helpers."""

'''

src/mobility_control_tower/realtime/gtfs_rt_charts.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_chart
s.py
FILE: src/mobility_control_tower/realtime/gtfs_rt_charts.py
'''python
"""Generate static charts for GTFS-Realtime snapshot gold tables."""

from __future__ import annotations

from pathlib import Path

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import pandas as pd


RT_CHART_FILES = (
    "rt_top_routes_by_avg_delay.png",
    "rt_delay_distribution.png",
    "rt_top_stops_by_avg_delay.png",
    "rt_identifier_match_rates.png",
)


def _read_csv(path: Path) -> pd.DataFrame:
    if not path.is_file():
        raise ValueError(f"Required real-time gold table is missing: {path.name}")
    return pd.read_csv(path)


def _barh(frame: pd.DataFrame, label: str, value: str, title: str, xlabel: str, output:
Path) -> None:
    data = frame.sort_values(value, ascending=True)
    fig, ax = plt.subplots(figsize=(10, max(4, min(8, 0.45 * len(data) + 1.5))))
    ax.barh(data[label], data[value], color="#7a4f9f")
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.grid(axis="x", alpha=0.25)
    fig.tight_layout()
    fig.savefig(output, dpi=150)
    plt.close(fig)


def generate_rt_charts(rt_gold_run: Path, reports_dir: Path = Path("data/reports")) -> Path:
    if not rt_gold_run.is_dir():
        raise FileNotFoundError(f"Real-time gold run directory not found: {rt_gold_run}")
    figures_dir = reports_dir / "figures" / "realtime" / rt_gold_run.name
    figures_dir.mkdir(parents=True, exist_ok=True)

    routes = _read_csv(rt_gold_run / "rt_route_delay_snapshot.csv")
    route_plot = routes.dropna(subset=["avg_delay_seconds"]).copy()
```

```
    route_plot["route_label"] = route_plot["route_short_name"].fillna("").astype(str)
    route_plot.loc[route_plot["route_label"].str.strip().eq(""), "route_label"] =
route_plot["route_id"]
    _barh(
        route_plot.sort_values("avg_delay_seconds", ascending=False).head(10),
        "route_label",
        "avg_delay_seconds",
        "Top routes by average observed delay in one snapshot",
        "Average delay seconds",
        figures_dir / "rt_top_routes_by_avg_delay.png",
    )

    fig, ax = plt.subplots(figsize=(10, 4.8))
    route_plot["avg_delay_seconds"].dropna().plot(kind="hist", bins=20, ax=ax,
color="#4f8a5b")
    ax.set_title("Distribution of route average observed delays")
    ax.set_xlabel("Average delay seconds")
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(figures_dir / "rt_delay_distribution.png", dpi=150)
    plt.close(fig)

    stops = _read_csv(rt_gold_run / "rt_stop_delay_snapshot.csv")
    stop_plot = stops.dropna(subset=["avg_delay_seconds"]).copy()
    stop_plot["stop_label"] = stop_plot["stop_name"].fillna("").astype(str)
    stop_plot.loc[stop_plot["stop_label"].str.strip().eq(""), "stop_label"] =
stop_plot["stop_id"]
    _barh(
        stop_plot.sort_values("avg_delay_seconds", ascending=False).head(10),
        "stop_label",
        "avg_delay_seconds",
        "Top stops by average observed delay in one snapshot",
        "Average delay seconds",
        figures_dir / "rt_top_stops_by_avg_delay.png",
    )

    compatibility = _read_csv(rt_gold_run / "rt_identifier_compatibility_snapshot.csv")
    compat_plot = compatibility.copy()
    compat_plot["match_percentage"] = pd.to_numeric(compat_plot["match_percentage"],
errors="coerce").fillna(0)
    fig, ax = plt.subplots(figsize=(8, 4.8))
    ax.bar(compat_plot["identifier_type"], compat_plot["match_percentage"], color="#2f6f9f")
    ax.set_ylim(0, 100)
    ax.set_title("Static/live identifier match rates")
    ax.set_ylabel("Match percentage")
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(figures_dir / "rt_identifier_match_rates.png", dpi=150)
    plt.close(fig)
    return figures_dir
'''

src/mobility_control_tower/realtime/gtfs_rt_compatibility.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_compa
tibility.py
FILE: src/mobility_control_tower/realtime/gtfs_rt_compatibility.py
'''python
"""Compare parsed GTFS-Realtime identifiers with static silver GTFS tables."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import pandas as pd
```

```
def _read_csv(path: Path) -> pd.DataFrame:
    if not path.is_file():
        return pd.DataFrame()
    return pd.read_csv(path, dtype=str, keep_default_na=False)


def _feed_type(rt_run: Path) -> str:
    manifest_path = rt_run / "realtime_manifest.json"
    if manifest_path.is_file():
        manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
        return manifest.get("feed_type") or rt_run.parent.name
    return rt_run.parent.name


def _collect_ids(rt_run: Path, column: str) -> set[str]:
    values: set[str] = set()
    for csv_path in rt_run.glob("rt_*.csv"):
        frame = _read_csv(csv_path)
        if column in frame.columns:
            values.update(value.strip() for value in frame[column].dropna().astype(str) if
value.strip())
    return values


def _static_ids(silver_run: Path, table: str, column: str) -> set[str]:
    frame = _read_csv(silver_run / f"{table}.csv")
    if column not in frame.columns:
        return set()
    return set(value.strip() for value in frame[column].dropna().astype(str) if
value.strip())


def _id_check(name: str, label: str, rt_ids: set[str], static_ids: set[str], optional: bool
= False) -> dict[str, Any]:
    if not rt_ids:
        status = "NOT_APPLICABLE" if optional else "WARN"
        return {
            "check_name": name,
            "status": status,
            "realtime_id_count": 0,
            "unmatched_count": 0,
            "unmatched_percentage": 0.0,
            "sample_unmatched_values": [],
            "explanation": f"No {label} values were present in the parsed real-time
snapshot.",
        }
    if not static_ids:
        return {
            "check_name": name,
            "status": "FAIL",
            "realtime_id_count": len(rt_ids),
            "unmatched_count": len(rt_ids),
            "unmatched_percentage": 100.0,
            "sample_unmatched_values": sorted(rt_ids)[:10],
            "explanation": f"Static silver data does not provide {label} values for
comparison.",
        }
    unmatched = sorted(rt_ids - static_ids)
    percentage = round((len(unmatched) / len(rt_ids)) * 100, 2)
    status = "PASS" if not unmatched else "WARN"
    return {
        "check_name": name,
        "status": status,
        "realtime_id_count": len(rt_ids),
        "unmatched_count": len(unmatched),
        "unmatched_percentage": percentage,
        "sample_unmatched_values": unmatched[:10],
        "explanation": f"{len(unmatched)} of {len(rt_ids)} real-time {label} values were not
```

```
found in static silver data.",
    }


def _overall_status(checks: list[dict[str, Any]]) -> str:
    statuses = {check["status"] for check in checks}
    if "FAIL" in statuses:
        return "FAIL"
    if "WARN" in statuses:
        return "WARN"
    return "PASS"


def _markdown_report(report: dict[str, Any]) -> str:
    rows = []
    for check in report["checks"]:
        rows.append(
            f"| {check['check_name']} | {check['status']} | {check.get('realtime_id_count',
'')} | "
            f"{check.get('unmatched_count', '')} | {check.get('unmatched_percentage', '')} |
{check['explanation']} |"
        )
    table = "\n".join(["| Check | Status | RT IDs | Unmatched | Unmatched % | Explanation
|", "| --- | --- | ---: | ---: | ---: | --- |", *rows])
    return f"""# GTFS-Realtime Compatibility Report

- Feed type: `{report['feed_type']}`
- Real-time run: `{report['rt_run_id']}`
- Static silver run: `{report['static_run_id']}`
- Overall status: **{report['overall_status']}**

{table}

## Interpretation

This is an exploration check. It asks whether identifiers observed in one saved
GTFS-Realtime snapshot can be joined to the static silver GTFS tables. It does not require
perfect matching and it is not a production real-time pipeline.
"""


def check_realtime_compatibility(silver_run: Path, rt_run: Path, reports_dir: Path =
Path("data/reports")) -> tuple[Path, Path]:
    if not silver_run.is_dir():
        raise FileNotFoundError(f"Silver run directory not found: {silver_run}")
    if not rt_run.is_dir():
        raise FileNotFoundError(f"Parsed real-time run directory not found: {rt_run}")
    feed_type = _feed_type(rt_run)
    rt_route_ids = _collect_ids(rt_run, "route_id")
    rt_trip_ids = _collect_ids(rt_run, "trip_id")
    rt_stop_ids = _collect_ids(rt_run, "stop_id")
    checks = [
        {
            "check_name": "realtime_has_join_identifiers",
            "status": "PASS" if (rt_route_ids or rt_trip_ids or rt_stop_ids) else "WARN",
            "realtime_id_count": len(rt_route_ids | rt_trip_ids | rt_stop_ids),
            "unmatched_count": 0,
            "unmatched_percentage": 0.0,
            "sample_unmatched_values": [],
            "explanation": "Real-time snapshot has route_id, trip_id, or stop_id values
useful for joining." if (rt_route_ids or rt_trip_ids or rt_stop_ids) else "Real-time
snapshot did not expose route_id, trip_id, or stop_id values.",
        },
        _id_check("route_id_matches_static_routes", "route_id", rt_route_ids,
_static_ids(silver_run, "routes", "route_id")),
        _id_check("trip_id_matches_static_trips", "trip_id", rt_trip_ids,
_static_ids(silver_run, "trips", "trip_id"), optional=(feed_type == "service_alerts")),
        _id_check("stop_id_matches_static_stops", "stop_id", rt_stop_ids,
_static_ids(silver_run, "stops", "stop_id")),
```

```
    ]
    report = {
        "feed_type": feed_type,
        "rt_run": str(rt_run),
        "rt_run_id": rt_run.name,
        "silver_run": str(silver_run),
        "static_run_id": silver_run.name,
        "generated_timestamp": datetime.now(timezone.utc).isoformat(),
        "overall_status": _overall_status(checks),
        "checks": checks,
    }
    reports_dir.mkdir(parents=True, exist_ok=True)
    json_path = reports_dir /
f"rt_compatibility_{feed_type}_{rt_run.name}_vs_{silver_run.name}.json"
    markdown_path = reports_dir /
f"rt_compatibility_{feed_type}_{rt_run.name}_vs_{silver_run.name}.md"
    json_path.write_text(json.dumps(report, indent=2, ensure_ascii=False) + "\n",
encoding="utf-8")
    markdown_path.write_text(_markdown_report(report), encoding="utf-8")
    return json_path, markdown_path
'''

src/mobility_control_tower/realtime/gtfs_rt_kpis.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_kpis.
py
FILE: src/mobility_control_tower/realtime/gtfs_rt_kpis.py
'''python
"""Build snapshot-level GTFS-Realtime gold indicators."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import pandas as pd


def _read_csv(path: Path) -> pd.DataFrame:
    if not path.is_file():
        return pd.DataFrame()
    return pd.read_csv(path, dtype=str, keep_default_na=False)


def _to_numeric(series: pd.Series) -> pd.Series:
    return pd.to_numeric(series.replace("", pd.NA), errors="coerce")


def freshness_status(feed_age_seconds: Any) -> str:
    age = pd.to_numeric(pd.Series([feed_age_seconds]).replace("", pd.NA),
errors="coerce").iloc[0]
    if pd.isna(age):
        return "UNKNOWN"
    if age <= 90:
        return "PASS"
    if age <= 300:
        return "WARN"
    return "FAIL"


def compatibility_status(match_percentage: float | None) -> str:
    if match_percentage is None:
        return "NOT_APPLICABLE"
    if match_percentage >= 95:
        return "PASS"
    if match_percentage >= 50:
        return "WARN"
```

```
    return "FAIL"


def _set_from_column(frame: pd.DataFrame, column: str) -> set[str]:
    if column not in frame.columns:
        return set()
    return set(value.strip() for value in frame[column].dropna().astype(str) if
value.strip())


def _id_compatibility(identifier_type: str, rt_values: set[str], static_values: set[str]) ->
dict[str, Any]:
    if not rt_values:
        return {
            "identifier_type": identifier_type,
            "rt_distinct_count": 0,
            "matched_static_count": 0,
            "unmatched_static_count": 0,
            "match_percentage": "",
            "status": "NOT_APPLICABLE",
            "sample_unmatched_values": "",
        }
    matched = rt_values & static_values
    unmatched = sorted(rt_values - static_values)
    pct = round((len(matched) / len(rt_values)) * 100, 2)
    return {
        "identifier_type": identifier_type,
        "rt_distinct_count": len(rt_values),
        "matched_static_count": len(matched),
        "unmatched_static_count": len(unmatched),
        "match_percentage": pct,
        "status": compatibility_status(pct),
        "sample_unmatched_values": "|".join(unmatched[:10]),
    }


def _delay_seconds(stop_updates: pd.DataFrame) -> pd.Series:
    arrival = _to_numeric(stop_updates.get("arrival_delay", pd.Series(dtype=str)))
    departure = _to_numeric(stop_updates.get("departure_delay", pd.Series(dtype=str)))
    return arrival.combine_first(departure)


def _route_info(routes: pd.DataFrame) -> pd.DataFrame:
    frame = routes.copy()
    for column in ("route_short_name", "route_long_name"):
        if column not in frame.columns:
            frame[column] = ""
    return frame[["route_id", "route_short_name",
"route_long_name"]].drop_duplicates("route_id")


def _aggregate_route_delay(stop_updates: pd.DataFrame, routes: pd.DataFrame) ->
pd.DataFrame:
    columns = [
        "route_id",
        "route_short_name",
        "route_long_name",
        "stop_time_updates_count",
        "distinct_trip_updates_count",
        "distinct_stops_count",
        "avg_delay_seconds",
        "median_delay_seconds",
        "max_delay_seconds",
        "min_delay_seconds",
        "delayed_updates_5min_count",
        "delayed_updates_5min_pct",
        "early_updates_count",
        "no_delay_info_count",
    ]
```

```
    if stop_updates.empty or "route_id" not in stop_updates.columns:
        return pd.DataFrame(columns=columns)
    frame = stop_updates.copy()
    frame["delay_seconds"] = _delay_seconds(frame)
    rows: list[dict[str, Any]] = []
    for route_id, group in frame.groupby("route_id", dropna=False):
        total = len(group)
        delays = group["delay_seconds"].dropna()
        delayed_count = int((delays >= 300).sum())
        rows.append(
            {
                "route_id": route_id,
                "stop_time_updates_count": total,
                "distinct_trip_updates_count": group["trip_id"].replace("",
pd.NA).dropna().nunique() if "trip_id" in group else 0,
                "distinct_stops_count": group["stop_id"].replace("",
pd.NA).dropna().nunique() if "stop_id" in group else 0,
                "avg_delay_seconds": round(float(delays.mean()), 2) if not delays.empty else
"",
                "median_delay_seconds": round(float(delays.median()), 2) if not delays.empty
else "",
                "max_delay_seconds": int(delays.max()) if not delays.empty else "",
                "min_delay_seconds": int(delays.min()) if not delays.empty else "",
                "delayed_updates_5min_count": delayed_count,
                "delayed_updates_5min_pct": round((delayed_count / total) * 100, 2) if total
else 0.0,
                "early_updates_count": int((delays < 0).sum()),
                "no_delay_info_count": int(group["delay_seconds"].isna().sum()),
            }
        )
    result = pd.DataFrame(rows).merge(_route_info(routes), on="route_id", how="left")
    return result[columns].sort_values(["avg_delay_seconds", "stop_time_updates_count"],
ascending=[False, False], na_position="last").reset_index(drop=True)


def _aggregate_stop_delay(stop_updates: pd.DataFrame, stops: pd.DataFrame) -> pd.DataFrame:
    columns = [
        "stop_id",
        "stop_name",
        "stop_time_updates_count",
        "distinct_routes_count",
        "distinct_trip_updates_count",
        "avg_delay_seconds",
        "median_delay_seconds",
        "max_delay_seconds",
        "delayed_updates_5min_count",
        "delayed_updates_5min_pct",
        "no_delay_info_count",
        "stop_id_static_match",
    ]
    if stop_updates.empty or "stop_id" not in stop_updates.columns:
        return pd.DataFrame(columns=columns)
    frame = stop_updates.copy()
    frame["delay_seconds"] = _delay_seconds(frame)
    static_stop_ids = _set_from_column(stops, "stop_id")
    stop_names = stops[["stop_id", "stop_name"]].drop_duplicates("stop_id") if {"stop_id",
"stop_name"}.issubset(stops.columns) else pd.DataFrame(columns=["stop_id", "stop_name"])
    rows: list[dict[str, Any]] = []
    for stop_id, group in frame.groupby("stop_id", dropna=False):
        total = len(group)
        delays = group["delay_seconds"].dropna()
        delayed_count = int((delays >= 300).sum())
        rows.append(
            {
                "stop_id": stop_id,
                "stop_time_updates_count": total,
                "distinct_routes_count": group["route_id"].replace("",
pd.NA).dropna().nunique() if "route_id" in group else 0,
                "distinct_trip_updates_count": group["trip_id"].replace("",
```

```
pd.NA).dropna().nunique() if "trip_id" in group else 0,
                "avg_delay_seconds": round(float(delays.mean()), 2) if not delays.empty else
"",
                "median_delay_seconds": round(float(delays.median()), 2) if not delays.empty
else "",
                "max_delay_seconds": int(delays.max()) if not delays.empty else "",
                "delayed_updates_5min_count": delayed_count,
                "delayed_updates_5min_pct": round((delayed_count / total) * 100, 2) if total
else 0.0,
                "no_delay_info_count": int(group["delay_seconds"].isna().sum()),
                "stop_id_static_match": stop_id in static_stop_ids if stop_id else False,
            }
        )
    result = pd.DataFrame(rows).merge(stop_names, on="stop_id", how="left")
    return result[columns].sort_values(["avg_delay_seconds", "stop_time_updates_count"],
ascending=[False, False], na_position="last").reset_index(drop=True)


def _feed_health(summary: pd.DataFrame, trips: pd.DataFrame, stops: pd.DataFrame) ->
pd.DataFrame:
    row = summary.iloc[0].to_dict() if not summary.empty else {}
    delay = _delay_seconds(stops) if not stops.empty else pd.Series(dtype=float)
    feed_age = row.get("feed_age_seconds", "")
    return pd.DataFrame(
        [
            {
                "feed_type": row.get("feed_type", ""),
                "fetched_at": row.get("fetched_at", ""),
                "header_timestamp": row.get("header_timestamp", ""),
                "feed_age_seconds": feed_age,
                "freshness_status": freshness_status(feed_age),
                "entity_count": row.get("entity_count", ""),
                "parsed_entity_count": row.get("parsed_entity_count", ""),
                "skipped_entity_count": row.get("skipped_entity_count", ""),
                "distinct_route_ids": len(_set_from_column(pd.concat([trips, stops],
ignore_index=True), "route_id")),
                "distinct_trip_ids": len(_set_from_column(pd.concat([trips, stops],
ignore_index=True), "trip_id")),
                "distinct_stop_ids": len(_set_from_column(stops, "stop_id")),
                "has_delay_information": bool(delay.notna().any()),
                "notes": "Snapshot health indicator only; not a service-level guarantee.",
            }
        ]
    )


def _enrich_trip_updates(trips: pd.DataFrame, routes: pd.DataFrame, static_trips:
pd.DataFrame) -> pd.DataFrame:
    columns = [
        "entity_id",
        "trip_id",
        "route_id",
        "route_short_name",
        "route_long_name",
        "start_date",
        "start_time",
        "schedule_relationship",
        "trip_update_timestamp",
        "stop_time_update_count",
        "route_id_static_match",
        "trip_id_static_match",
        "compatibility_note",
    ]
    if trips.empty:
        return pd.DataFrame(columns=columns)
    route_ids = _set_from_column(routes, "route_id")
    trip_ids = _set_from_column(static_trips, "trip_id")
    enriched = trips.copy().merge(_route_info(routes), on="route_id", how="left")
    enriched["route_id_static_match"] = enriched["route_id"].apply(lambda value:
```

```
bool(str(value).strip()) and str(value).strip() in route_ids)
    enriched["trip_id_static_match"] = enriched["trip_id"].apply(lambda value:
bool(str(value).strip()) and str(value).strip() in trip_ids)

    def note(row: pd.Series) -> str:
        if not str(row.get("trip_id", "")).strip():
            return "trip_id missing in snapshot"
        if not row["trip_id_static_match"]:
            return "trip_id not found in static silver trips"
        if not row["route_id_static_match"]:
            return "route_id not found in static silver routes"
        return "matched static identifiers"

    enriched["compatibility_note"] = enriched.apply(note, axis=1)
    return enriched[columns]


def compute_rt_gold_tables(silver_tables: dict[str, pd.DataFrame], rt_tables: dict[str,
pd.DataFrame]) -> dict[str, pd.DataFrame]:
    summary = rt_tables.get("rt_feed_summary", pd.DataFrame())
    trips = rt_tables.get("rt_trip_updates", pd.DataFrame())
    stop_updates = rt_tables.get("rt_stop_time_updates", pd.DataFrame())
    routes = silver_tables.get("routes", pd.DataFrame())
    static_trips = silver_tables.get("trips", pd.DataFrame())
    stops = silver_tables.get("stops", pd.DataFrame())

    if not summary.empty and str(summary.iloc[0].get("feed_type", "")) != "trip_updates":
        raise ValueError("Real-time gold currently supports Trip Updates snapshots only")
    compatibility = pd.DataFrame(
        [
            _id_compatibility("route_id", _set_from_column(pd.concat([trips, stop_updates],
ignore_index=True), "route_id"), _set_from_column(routes, "route_id")),
            _id_compatibility("trip_id", _set_from_column(pd.concat([trips, stop_updates],
ignore_index=True), "trip_id"), _set_from_column(static_trips, "trip_id")),
            _id_compatibility("stop_id", _set_from_column(stop_updates, "stop_id"),
_set_from_column(stops, "stop_id")),
        ]
    )
    return {
        "rt_feed_health_snapshot": _feed_health(summary, trips, stop_updates),
        "rt_trip_update_enriched": _enrich_trip_updates(trips, routes, static_trips),
        "rt_route_delay_snapshot": _aggregate_route_delay(stop_updates, routes),
        "rt_stop_delay_snapshot": _aggregate_stop_delay(stop_updates, stops),
        "rt_identifier_compatibility_snapshot": compatibility,
    }


def build_rt_gold(silver_run: Path, rt_run: Path, output_root: Path =
Path("data/realtime_gold")) -> Path:
    if not silver_run.is_dir():
        raise FileNotFoundError(f"Silver run directory not found: {silver_run}")
    if not rt_run.is_dir():
        raise FileNotFoundError(f"Parsed real-time run directory not found: {rt_run}")
    silver_tables = {name: _read_csv(silver_run / f"{name}.csv") for name in ("routes",
"trips", "stops")}
    rt_tables = {path.stem: _read_csv(path) for path in rt_run.glob("rt_*.csv")}
    manifest_path = rt_run / "realtime_manifest.json"
    manifest = json.loads(manifest_path.read_text(encoding="utf-8")) if
manifest_path.is_file() else {}
    feed_type = manifest.get("feed_type") or rt_run.parent.name
    source_id = rt_run.parents[1].name
    output_dir = output_root / source_id / feed_type / rt_run.name
    output_dir.mkdir(parents=True, exist_ok=False)
    outputs = compute_rt_gold_tables(silver_tables, rt_tables)
    manifest_tables: dict[str, dict[str, Any]] = {}
    for name, frame in outputs.items():
        frame.to_csv(output_dir / f"{name}.csv", index=False)
        manifest_tables[name] = {"file": f"{name}.csv", "row_count": len(frame), "columns":
list(frame.columns)}
```

```
    health = outputs["rt_feed_health_snapshot"].iloc[0].to_dict()
    compatibility =
outputs["rt_identifier_compatibility_snapshot"].set_index("identifier_type")["status"].to_di
ct()
    rt_manifest = {
        "static_silver_run": str(silver_run),
        "realtime_parsed_run": str(rt_run),
        "generated_timestamp": datetime.now(timezone.utc).isoformat(),
        "tables_created": manifest_tables,
        "kpi_definitions": {
            "rt_feed_health_snapshot": "Freshness, entity counts, identifier counts, and
delay availability for one GTFS-Realtime snapshot.",
            "rt_trip_update_enriched": "Trip updates joined with static route/trip
identifiers where possible.",
            "rt_route_delay_snapshot": "Observed delay indicators by route in one
snapshot.",
            "rt_stop_delay_snapshot": "Observed delay indicators by stop in one snapshot.",
            "rt_identifier_compatibility_snapshot": "Identifier match rates between parsed
snapshot and static silver GTFS.",
        },
        "assumptions": [
            "Arrival delay is preferred over departure delay when both are available.",
            "Delay metrics describe only the saved GTFS-Realtime snapshot.",
            "Identifier matching uses exact string equality against silver routes, trips,
and stops.",
        ],
        "limitations": [
            "This is not continuous streaming and not route reliability.",
            "A single snapshot cannot prove recurring delay patterns.",
            "Trip IDs may be unmatched even when route and stop IDs match.",
        ],
        "compatibility_summary": compatibility,
        "feed_health_summary": health,
    }
    (output_dir / "rt_gold_manifest.json").write_text(json.dumps(rt_manifest, indent=2,
ensure_ascii=False) + "\n", encoding="utf-8")
    return output_dir
'''

src/mobility_control_tower/realtime/gtfs_rt_parser.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_parse
r.py
FILE: src/mobility_control_tower/realtime/gtfs_rt_parser.py
'''python
"""Parse GTFS-Realtime protobuf snapshots into simple CSV tables."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import pandas as pd
from google.transit import gtfs_realtime_pb2


def _enum_name(enum_type: Any, value: int | None) -> str:
    if value is None:
        return ""
    try:
        return enum_type.Name(value)
    except ValueError:
        return str(value)


def _translation_text(translated: Any) -> str:
    if not translated.translation:
```

```
        return ""
    for translation in translated.translation:
        if translation.text:
            return translation.text
    return ""


def _metadata(raw_run: Path) -> dict[str, Any]:
    path = raw_run / "metadata.json"
    if not path.is_file():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))


def _feed_type_from_raw_run(raw_run: Path, metadata: dict[str, Any]) -> str:
    return metadata.get("feed_type") or raw_run.parent.name


def _unix_to_iso(value: int | None) -> str:
    if not value:
        return ""
    return datetime.fromtimestamp(value, timezone.utc).isoformat()


def _feed_age_seconds(header_timestamp: int | None, fetched_at: str | None) -> int | None:
    if not header_timestamp or not fetched_at:
        return None
    try:
        fetched = datetime.fromisoformat(fetched_at.replace("Z", "+00:00"))
    except ValueError:
        return None
    return int(fetched.timestamp() - header_timestamp)


def _summary(feed: gtfs_realtime_pb2.FeedMessage, feed_type: str, fetched_at: str | None,
parsed_count: int, skipped_count: int) -> pd.DataFrame:
    header_timestamp = feed.header.timestamp if feed.header.HasField("timestamp") else None
    row = {
        "feed_type": feed_type,
        "gtfs_realtime_version": feed.header.gtfs_realtime_version,
        "header_timestamp": header_timestamp or "",
        "header_timestamp_iso": _unix_to_iso(header_timestamp),
        "fetched_at": fetched_at or "",
        "feed_age_seconds": _feed_age_seconds(header_timestamp, fetched_at),
        "entity_count": len(feed.entity),
        "parsed_entity_count": parsed_count,
        "skipped_entity_count": skipped_count,
    }
    return pd.DataFrame([row])


def _trip_updates(feed: gtfs_realtime_pb2.FeedMessage) -> tuple[pd.DataFrame, pd.DataFrame,
int, int]:
    trip_rows: list[dict[str, Any]] = []
    stop_rows: list[dict[str, Any]] = []
    skipped = 0
    for entity in feed.entity:
        if not entity.HasField("trip_update"):
            skipped += 1
            continue
        update = entity.trip_update
        trip = update.trip
        trip_rows.append(
            {
                "entity_id": entity.id,
                "trip_id": trip.trip_id,
                "route_id": trip.route_id,
                "start_date": trip.start_date,
                "start_time": trip.start_time,
```

```
                "schedule_relationship":
_enum_name(gtfs_realtime_pb2.TripDescriptor.ScheduleRelationship,
trip.schedule_relationship),
                "trip_update_timestamp": update.timestamp if update.HasField("timestamp")
else "",
                "stop_time_update_count": len(update.stop_time_update),
            }
        )
        for stop_update in update.stop_time_update:
            arrival = stop_update.arrival if stop_update.HasField("arrival") else None
            departure = stop_update.departure if stop_update.HasField("departure") else None
            stop_rows.append(
                {
                    "entity_id": entity.id,
                    "trip_id": trip.trip_id,
                    "route_id": trip.route_id,
                    "stop_sequence": stop_update.stop_sequence if
stop_update.HasField("stop_sequence") else "",
                    "stop_id": stop_update.stop_id,
                    "arrival_time": arrival.time if arrival and arrival.HasField("time")
else "",
                    "arrival_delay": arrival.delay if arrival and arrival.HasField("delay")
else "",
                    "departure_time": departure.time if departure and
departure.HasField("time") else "",
                    "departure_delay": departure.delay if departure and
departure.HasField("delay") else "",
                    "schedule_relationship":
_enum_name(gtfs_realtime_pb2.TripUpdate.StopTimeUpdate.ScheduleRelationship,
stop_update.schedule_relationship),
                }
            )
    return pd.DataFrame(trip_rows), pd.DataFrame(stop_rows), len(trip_rows), skipped


def _vehicle_positions(feed: gtfs_realtime_pb2.FeedMessage) -> tuple[pd.DataFrame, int,
int]:
    rows: list[dict[str, Any]] = []
    skipped = 0
    for entity in feed.entity:
        if not entity.HasField("vehicle"):
            skipped += 1
            continue
        vehicle = entity.vehicle
        trip = vehicle.trip
        descriptor = vehicle.vehicle
        position = vehicle.position
        rows.append(
            {
                "entity_id": entity.id,
                "trip_id": trip.trip_id,
                "route_id": trip.route_id,
                "vehicle_id": descriptor.id,
                "label": descriptor.label,
                "latitude": position.latitude if vehicle.HasField("position") else "",
                "longitude": position.longitude if vehicle.HasField("position") else "",
                "bearing": position.bearing if vehicle.HasField("position") and
position.HasField("bearing") else "",
                "speed": position.speed if vehicle.HasField("position") and
position.HasField("speed") else "",
                "current_stop_sequence": vehicle.current_stop_sequence if
vehicle.HasField("current_stop_sequence") else "",
                "stop_id": vehicle.stop_id,
                "current_status":
_enum_name(gtfs_realtime_pb2.VehiclePosition.VehicleStopStatus, vehicle.current_status),
                "timestamp": vehicle.timestamp if vehicle.HasField("timestamp") else "",
            }
        )
    return pd.DataFrame(rows), len(rows), skipped
```

```
def _alerts(feed: gtfs_realtime_pb2.FeedMessage) -> tuple[pd.DataFrame, pd.DataFrame, int,
int]:
    alert_rows: list[dict[str, Any]] = []
    entity_rows: list[dict[str, Any]] = []
    skipped = 0
    for entity in feed.entity:
        if not entity.HasField("alert"):
            skipped += 1
            continue
        alert = entity.alert
        periods = list(alert.active_period) or [None]
        for period in periods:
            alert_rows.append(
                {
                    "entity_id": entity.id,
                    "cause": _enum_name(gtfs_realtime_pb2.Alert.Cause, alert.cause),
                    "effect": _enum_name(gtfs_realtime_pb2.Alert.Effect, alert.effect),
                    "active_period_start": period.start if period and
period.HasField("start") else "",
                    "active_period_end": period.end if period and period.HasField("end")
else "",
                    "header_text": _translation_text(alert.header_text),
                    "description_text": _translation_text(alert.description_text),
                    "url": _translation_text(alert.url),
                }
            )
        for informed in alert.informed_entity:
            entity_rows.append(
                {
                    "entity_id": entity.id,
                    "agency_id": informed.agency_id,
                    "route_id": informed.route_id,
                    "route_type": informed.route_type if informed.HasField("route_type")
else "",
                    "trip_id": informed.trip.trip_id if informed.HasField("trip") else "",
                    "stop_id": informed.stop_id,
                }
            )
    return pd.DataFrame(alert_rows), pd.DataFrame(entity_rows), len(alert_rows), skipped


def parse_realtime_snapshot(raw_rt_run: Path, output_root: Path = Path("data/realtime")) ->
Path:
    feed_path = raw_rt_run / "feed.pb"
    if not feed_path.is_file():
        raise FileNotFoundError(f"GTFS-Realtime feed.pb not found: {feed_path}")
    metadata = _metadata(raw_rt_run)
    source_id = metadata.get("source_id") or raw_rt_run.parents[1].name
    feed_type = _feed_type_from_raw_run(raw_rt_run, metadata)
    output_dir = output_root / source_id / feed_type / raw_rt_run.name
    output_dir.mkdir(parents=True, exist_ok=False)

    feed = gtfs_realtime_pb2.FeedMessage()
    try:
        feed.ParseFromString(feed_path.read_bytes())
    except Exception as exc:
        raise ValueError(f"Unable to parse GTFS-Realtime protobuf snapshot: {exc}") from exc

    tables: dict[str, pd.DataFrame] = {}
    if feed_type == "trip_updates":
        trip_updates, stop_updates, parsed, skipped = _trip_updates(feed)
        tables["rt_trip_updates"] = trip_updates
        tables["rt_stop_time_updates"] = stop_updates
    elif feed_type == "vehicle_positions":
        vehicle_positions, parsed, skipped = _vehicle_positions(feed)
        tables["rt_vehicle_positions"] = vehicle_positions
    elif feed_type == "service_alerts":
```

```
        alerts, informed, parsed, skipped = _alerts(feed)
        tables["rt_alerts"] = alerts
        tables["rt_alert_informed_entities"] = informed
    else:
        raise ValueError(f"Unsupported GTFS-Realtime feed type in raw run: {feed_type}")

    tables["rt_feed_summary"] = _summary(feed, feed_type, metadata.get("fetched_at"),
parsed, skipped)
    manifest_tables: dict[str, dict[str, Any]] = {}
    for name, frame in tables.items():
        path = output_dir / f"{name}.csv"
        frame.to_csv(path, index=False)
        manifest_tables[name] = {"file": path.name, "row_count": len(frame), "columns":
list(frame.columns)}

    manifest = {
        "source_raw_realtime_run": str(raw_rt_run),
        "generated_timestamp": datetime.now(timezone.utc).isoformat(),
        "feed_type": feed_type,
        "tables_created": manifest_tables,
        "snapshot_note": "Parsed from a saved local feed.pb snapshot; this is not continuous
streaming.",
    }
    (output_dir / "realtime_manifest.json").write_text(json.dumps(manifest, indent=2,
ensure_ascii=False) + "\n", encoding="utf-8")
    return output_dir
'''

src/mobility_control_tower/realtime/gtfs_rt_raw.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_raw.p
y
FILE: src/mobility_control_tower/realtime/gtfs_rt_raw.py
'''python
"""Fetch and preserve one GTFS-Realtime protobuf snapshot."""

from __future__ import annotations

import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import requests


FEED_TYPES = {"trip_updates", "vehicle_positions", "service_alerts"}


def timestamp_run_id() -> str:
    return datetime.now().strftime("%Y-%m-%d_%H%M%S")


def configured_realtime_url(source: dict[str, Any], feed_type: str) -> str | None:
    feeds = source.get("gtfs_realtime") or {}
    return feeds.get(f"{feed_type}_url")


def preserve_realtime_snapshot(
    content: bytes,
    source_id: str,
    source: dict[str, Any],
    feed_type: str,
    url: str,
    raw_root: Path = Path("data/raw_realtime"),
    http_status: int | None = None,
    content_type: str | None = None,
) -> Path:
```

```
    if feed_type not in FEED_TYPES:
        raise ValueError(f"Unsupported GTFS-Realtime feed type '{feed_type}'. Expected one
of: {', '.join(sorted(FEED_TYPES))}")
    if not content:
        raise ValueError("GTFS-Realtime snapshot is empty")
    run_dir = raw_root / source_id / feed_type / timestamp_run_id()
    run_dir.mkdir(parents=True, exist_ok=False)
    feed_path = run_dir / "feed.pb"
    feed_path.write_bytes(content)
    metadata = {
        "source_id": source_id,
        "source_name": source.get("name"),
        "source_page_url": source.get("source_page_url"),
        "feed_type": feed_type,
        "url": url,
        "fetched_at": datetime.now(timezone.utc).isoformat(),
        "sha256": hashlib.sha256(content).hexdigest(),
        "file_size_bytes": len(content),
        "http_status": http_status,
        "content_type": content_type,
        "snapshot_note": "Saved feed.pb is an immutable local GTFS-Realtime snapshot for
replay, demos, parser tests, and compatibility checks.",
    }
    (run_dir / "metadata.json").write_text(json.dumps(metadata, indent=2,
ensure_ascii=False) + "\n", encoding="utf-8")
    return run_dir


def fetch_realtime_snapshot(
    source_id: str,
    source: dict[str, Any],
    feed_type: str,
    url: str | None = None,
    raw_root: Path = Path("data/raw_realtime"),
    timeout_seconds: int = 30,
) -> Path:
    resolved_url = url or configured_realtime_url(source, feed_type)
    if not resolved_url:
        raise ValueError(
            f"No configured GTFS-Realtime URL for feed type '{feed_type}'. "
            "Provide one with --url or add it under gtfs_realtime in config/sources.yml."
        )
    try:
        response = requests.get(resolved_url, timeout=timeout_seconds)
    except requests.RequestException as exc:
        raise RuntimeError(f"Failed to fetch GTFS-Realtime snapshot from {resolved_url}:
{exc}") from exc
    if response.status_code >= 400:
        raise RuntimeError(f"GTFS-Realtime request failed with HTTP {response.status_code}
for {resolved_url}")
    return preserve_realtime_snapshot(
        response.content,
        source_id,
        source,
        feed_type,
        resolved_url,
        raw_root,
        http_status=response.status_code,
        content_type=response.headers.get("content-type"),
    )
'''

src/mobility_control_tower/realtime/gtfs_rt_report.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_repor
t.py
FILE: src/mobility_control_tower/realtime/gtfs_rt_report.py
'''python
"""Generate Markdown reports for parsed GTFS-Realtime snapshots."""
```

```
from __future__ import annotations

import json
from pathlib import Path

import pandas as pd


def _read_csv(path: Path) -> pd.DataFrame:
    if not path.is_file():
        return pd.DataFrame()
    return pd.read_csv(path, dtype=str, keep_default_na=False)


def _manifest(rt_run: Path) -> dict:
    path = rt_run / "realtime_manifest.json"
    if not path.is_file():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))


def _count_distinct(rt_run: Path, column: str) -> int:
    values: set[str] = set()
    for csv_path in rt_run.glob("rt_*.csv"):
        frame = _read_csv(csv_path)
        if column in frame.columns:
            values.update(value.strip() for value in frame[column].astype(str) if
value.strip())
    return len(values)


def _table_counts(rt_run: Path) -> str:
    rows = []
    for csv_path in sorted(rt_run.glob("rt_*.csv")):
        frame = _read_csv(csv_path)
        rows.append(f"| {csv_path.name} | {len(frame)} |")
    if not rows:
        return "No parsed CSV tables found."
    return "\n".join(["| Table | Rows |", "| --- | ---: |", *rows])


def _observations(summary: pd.Series, route_count: int, trip_count: int, stop_count: int) ->
str:
    observations = []
    age = summary.get("feed_age_seconds", "")
    try:
        age_value = float(age)
    except (TypeError, ValueError):
        age_value = None
    if age_value is None:
        observations.append("- Feed age could not be computed from this snapshot.")
    elif age_value <= 300:
        observations.append("- Feed age was under five minutes at fetch time, so it looks
fresh enough for later exploration.")
    else:
        observations.append("- Feed age was over five minutes at fetch time; freshness
should be checked again before later phases.")
    if route_count or trip_count or stop_count:
        observations.append("- The snapshot exposes identifiers that may be useful for
joining with static GTFS.")
    else:
        observations.append("- The snapshot did not expose route_id, trip_id, or stop_id
values in parsed tables.")
    return "\n".join(observations)


def generate_realtime_report(rt_run: Path, reports_dir: Path = Path("data/reports")) ->
Path:
```

```
    if not rt_run.is_dir():
        raise FileNotFoundError(f"Parsed real-time run directory not found: {rt_run}")
    manifest = _manifest(rt_run)
    feed_type = manifest.get("feed_type") or rt_run.parent.name
    summary = _read_csv(rt_run / "rt_feed_summary.csv")
    if summary.empty:
        raise ValueError(f"Parsed real-time run is missing rt_feed_summary.csv: {rt_run}")
    row = summary.iloc[0]
    route_count = _count_distinct(rt_run, "route_id")
    trip_count = _count_distinct(rt_run, "trip_id")
    stop_count = _count_distinct(rt_run, "stop_id")
    feed_age = row.get("feed_age_seconds", "")
    usable = "candidate for later real-time phase" if (route_count or trip_count or
stop_count) else "limited candidate until identifiers are available"
    report = f"""# GTFS-Realtime Snapshot Report

## 1. Feed type

‘{feed_type}‘

## 2. Snapshot time

Fetched at: ‘{row.get(’fetched_at’, ’’)}‘

## 3. Header timestamp

- Raw timestamp: ‘{row.get(’header_timestamp’, ’’)}‘
- ISO timestamp: ‘{row.get(’header_timestamp_iso’, ’’)}‘

## 4. Feed age

‘{feed_age}‘ seconds, if computable from the header timestamp and fetch timestamp.

## 5. Number of entities

- Entities in protobuf snapshot: ‘{row.get(’entity_count’, ’’)}‘
- Parsed entities: ‘{row.get(’parsed_entity_count’, ’’)}‘
- Skipped entities: ‘{row.get(’skipped_entity_count’, ’’)}‘

## 6. Parsed table row counts

{_table_counts(rt_run)}

## 7. Main available identifiers

- Distinct route_id values: ‘{route_count}‘
- Distinct trip_id values: ‘{trip_count}‘
- Distinct stop_id values: ‘{stop_count}‘

## 8. Observations

{_observations(row, route_count, trip_count, stop_count)}

## 9. Limitations

- This report describes one saved GTFS-Realtime snapshot, not a stream.
- Values are observed at fetch time and may change seconds later.
- It is not a production real-time pipeline.

## 10. Later-phase usability

This feed is a ‘{usable}‘. Compatibility with static GTFS should be checked before building
any real-time metrics.
"""
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"gtfs_rt_report_{feed_type}_{rt_run.name}.md"
    output.write_text(report, encoding="utf-8")
    return output
’’’
```

src/mobility_control_tower/realtime/gtfs_rt_snapshot_report.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/realtime/gtfs_rt_snaps
hot_report.py
FILE: src/mobility_control_tower/realtime/gtfs_rt_snapshot_report.py
‘‘‘python
"""Generate a teacher-facing report from real-time snapshot gold tables."""

from __future__ import annotations

from pathlib import Path

import pandas as pd

from mobility_control_tower.realtime.gtfs_rt_charts import RT_CHART_FILES

def _read_csv(path: Path) -> pd.DataFrame:
    if not path.is_file():
        raise ValueError(f"Required real-time gold table is missing: {path.name}")
    return pd.read_csv(path)

def _markdown_table(frame: pd.DataFrame) -> str:
    if frame.empty:
        return "No rows available."
    headers = [str(column) for column in frame.columns]
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join("---" for _ in headers) +
" |"]
    for row in frame.itertuples(index=False, name=None):
        lines.append("| " + " | ".join(str(value).replace("|", "\\|") for value in row) + "
|")
    return "\n".join(lines)

def _chart_refs(run_id: str, reports_dir: Path) -> str:
    figures = reports_dir / "figures" / "realtime" / run_id
    existing = [name for name in RT_CHART_FILES if (figures / name).is_file()]
    if not existing:
        return "No real-time chart PNG files were found for this run. Run
‘generate-rt-charts‘ first."
    return "\n".join(f"- ‘{figures / name}‘" for name in existing)

def generate_rt_snapshot_report(rt_gold_run: Path, reports_dir: Path = Path("data/reports"))
-> Path:
    if not rt_gold_run.is_dir():
        raise FileNotFoundError(f"Real-time gold run directory not found: {rt_gold_run}")
    run_id = rt_gold_run.name
    health = _read_csv(rt_gold_run / "rt_feed_health_snapshot.csv")
    compatibility = _read_csv(rt_gold_run / "rt_identifier_compatibility_snapshot.csv")
    routes = _read_csv(rt_gold_run / "rt_route_delay_snapshot.csv")
    stops = _read_csv(rt_gold_run / "rt_stop_delay_snapshot.csv")
    health_row = health.iloc[0].to_dict() if not health.empty else {}
    compat_note = ""
    trip_rows = compatibility.loc[compatibility["identifier_type"] == "trip_id"]
    if not trip_rows.empty and str(trip_rows.iloc[0]["status"]) == "WARN":
        compat_note = "Trip ID compatibility is WARN: some trip identifiers observed at
fetch time were not found in the selected static silver GTFS run. Route and stop joins may
still be useful when their match rates are stronger."

    top_routes =
routes.dropna(subset=["avg_delay_seconds"]).sort_values("avg_delay_seconds",
ascending=False).head(10)
    top_stops = stops.dropna(subset=["avg_delay_seconds"]).sort_values("avg_delay_seconds",
ascending=False).head(10)
    report = f"""# Mobility Control Tower - GTFS-Realtime Snapshot Report

```
## 1. Dataset and snapshot information

- Feed type: ‘{health_row.get(’feed_type’, ’’)}‘
- Snapshot run ID: ‘{run_id}‘
- Fetched at: ‘{health_row.get(’fetched_at’, ’’)}‘
- This report describes one GTFS-Realtime snapshot observed at fetch time.

## 2. What the real-time snapshot pipeline did

The pipeline preserved a raw protobuf snapshot, parsed Trip Updates into CSV tables, joined
static GTFS identifiers where possible, computed snapshot delay indicators, and generated
static evidence artifacts.

## 3. Feed freshness and health

{_markdown_table(health)}

## 4. Identifier compatibility with static GTFS

{_markdown_table(compatibility)}

{compat_note}

## 5. Observed delay indicators

Delay uses ‘arrival_delay‘ when available, otherwise ‘departure_delay‘. These are snapshot
delay indicators, not continuous route reliability metrics.

## 6. Routes with highest average observed delay

{_markdown_table(top_routes[["route_id", "route_short_name", "route_long_name",
"stop_time_updates_count", "avg_delay_seconds", "delayed_updates_5min_count"]].head(10))}

## 7. Stops with highest average observed delay

{_markdown_table(top_stops[["stop_id", "stop_name", "stop_time_updates_count",
"avg_delay_seconds", "delayed_updates_5min_count"]].head(10))}

## 8. Chart references if generated

{_chart_refs(run_id, reports_dir)}

## 9. What this proves

The project can turn a saved GTFS-Realtime snapshot into enriched diagnostic tables, feed
health indicators, static/live compatibility evidence, delay indicators, and
teacher-friendly charts.

## 10. What it does not prove yet

- It does not prove route reliability from a single snapshot.
- It does not prove stable delay patterns over time.
- It does not prove production readiness.

## 11. Why this is still not streaming

The workflow processes one saved ‘feed.pb‘ file at a time. There is no scheduler, message
broker, database, API, dashboard, or continuous real-time monitoring system.

## 12. Next project step

Collect a few saved snapshots at different times and compare feed freshness, identifier
compatibility, and delay behavior before considering any streaming design.
"""
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"realtime_snapshot_report_{run_id}.md"
    output.write_text(report, encoding="utf-8")
    return output
’’’
```

src/mobility_control_tower/reporting/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/reporting/__init__.py
FILE: src/mobility_control_tower/reporting/__init__.py
```python
"""Local teacher-friendly reports."""
```

src/mobility_control_tower/reporting/charts.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/reporting/charts.py
FILE: src/mobility_control_tower/reporting/charts.py
```python
"""Generate simple static PNG charts from gold KPI CSV files."""

from __future__ import annotations

from pathlib import Path

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import pandas as pd


CHART_FILES = (
    "top_routes_by_total_scheduled_trips.png",
    "network_daily_scheduled_trips.png",
    "departures_by_hour_all_routes.png",
    "top_stops_by_total_departures.png",
)


def _read_gold(gold_run: Path, name: str) -> pd.DataFrame:
    path = gold_run / f"{name}.csv"
    if not path.is_file():
        raise ValueError(f"Chart generation requires missing gold file: {path.name}")
    return pd.read_csv(path)


def _route_label(row: pd.Series) -> str:
    short_name = str(row.get("route_short_name", "")).strip()
    long_name = str(row.get("route_long_name", "")).strip()
    return short_name or long_name or str(row.get("route_id", "unknown"))


def _save_barh(frame: pd.DataFrame, label_column: str, value_column: str, title: str,
xlabel: str, path: Path) -> None:
    plot_frame = frame.sort_values(value_column, ascending=True)
    height = max(4.0, min(8.0, 0.45 * len(plot_frame) + 1.5))
    fig, ax = plt.subplots(figsize=(10, height))
    ax.barh(plot_frame[label_column], plot_frame[value_column], color="#2f6f9f")
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.grid(axis="x", alpha=0.25)
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)


def generate_static_charts(gold_run: Path, reports_dir: Path = Path("data/reports")) ->
Path:
    """Create teacher-friendly static PNG charts for a gold run."""
    if not gold_run.is_dir():
        raise FileNotFoundError(f"Gold run directory not found: {gold_run}")
    run_id = gold_run.name
```

```
    figures_dir = reports_dir / "figures" / run_id
    figures_dir.mkdir(parents=True, exist_ok=True)

    route_period = _read_gold(gold_run, "route_period_summary")
    top_routes = route_period.sort_values("total_scheduled_trips",
ascending=False).head(10).copy()
    top_routes["route_label"] = top_routes.apply(_route_label, axis=1)
    _save_barh(
        top_routes,
        "route_label",
        "total_scheduled_trips",
        "Top routes by scheduled trips over the GTFS service period",
        "Scheduled trips",
        figures_dir / "top_routes_by_total_scheduled_trips.png",
    )

    network = _read_gold(gold_run, "network_daily_summary").sort_values("service_date")
    fig, ax = plt.subplots(figsize=(10, 4.8))
    ax.plot(network["service_date"], network["scheduled_trips_count"], marker="o",
linewidth=1.8, color="#2f6f9f")
    ax.set_title("Network scheduled trips by service date")
    ax.set_xlabel("Service date")
    ax.set_ylabel("Scheduled trips")
    ax.tick_params(axis="x", rotation=45)
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(figures_dir / "network_daily_scheduled_trips.png", dpi=150)
    plt.close(fig)

    hourly = _read_gold(gold_run, "route_hourly_departures")
    hourly_total = hourly.groupby("departure_hour",
as_index=False)["scheduled_departures_count"].sum().sort_values("departure_hour")
    fig, ax = plt.subplots(figsize=(10, 4.8))
    ax.bar(hourly_total["departure_hour"].astype(str),
hourly_total["scheduled_departures_count"], color="#4f8a5b")
    ax.set_title("Scheduled trip starts by service-day hour")
    ax.set_xlabel("Service-day hour")
    ax.set_ylabel("Scheduled departures")
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(figures_dir / "departures_by_hour_all_routes.png", dpi=150)
    plt.close(fig)

    stop_daily = _read_gold(gold_run, "stop_daily_departures")
    top_stops = stop_daily.groupby(["stop_id", "stop_name"], as_index=False,
dropna=False)["scheduled_departures_count"].sum()
    top_stops = top_stops.sort_values("scheduled_departures_count",
ascending=False).head(10)
    _save_barh(
        top_stops,
        "stop_name",
        "scheduled_departures_count",
        "Top stops by scheduled departures over the GTFS service period",
        "Scheduled departures",
        figures_dir / "top_stops_by_total_departures.png",
    )

    return figures_dir
'''

src/mobility_control_tower/reporting/demo_report.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/reporting/demo_report.
py
FILE: src/mobility_control_tower/reporting/demo_report.py
'''python
"""Generate a compact Markdown demonstration report from gold KPI tables."""

from __future__ import annotations
```

```
import json
from pathlib import Path

import pandas as pd

from mobility_control_tower.reporting.charts import CHART_FILES


def _markdown_table(frame: pd.DataFrame) -> str:
    if frame.empty:
        return "No rows available."
    headers = [str(column) for column in frame.columns]
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join("---" for _ in headers) +
" |"]
    for row in frame.itertuples(index=False, name=None):
        lines.append("| " + " | ".join(str(value).replace("|", "\\|") for value in row) + "
|")
    return "\n".join(lines)


def _read_csv(gold_run: Path, name: str) -> pd.DataFrame:
    path = gold_run / f"{name}.csv"
    if not path.is_file():
        raise ValueError(f"Report requires missing gold file: {path.name}")
    return pd.read_csv(path)


def _quality_summary(run_id: str, reports_dir: Path) -> tuple[str, dict[str, int] | None]:
    quality_path = reports_dir / f"gtfs_quality_{run_id}.json"
    if not quality_path.is_file():
        return "Quality summary unavailable for this run.", None
    quality = json.loads(quality_path.read_text(encoding="utf-8"))
    counts = {status: sum(check["status"] == status for check in quality.get("checks", []))
for status in ("PASS", "WARN", "FAIL")}
    text = f"Overall status: **{quality.get('overall_status', 'unknown')}**. Checks:
{counts['PASS']} PASS, {counts['WARN']} WARN, {counts['FAIL']} FAIL."
    return text, counts


def _chart_references(run_id: str, reports_dir: Path) -> str:
    figures_dir = reports_dir / "figures" / run_id
    if not figures_dir.is_dir():
        return "No chart PNG files were found for this run. Run `generate-static-charts` to
create them."
    existing = [name for name in CHART_FILES if (figures_dir / name).is_file()]
    if not existing:
        return "No chart PNG files were found for this run. Run `generate-static-charts` to
create them."
    return "\n".join(f"- `{figures_dir / name}`" for name in existing)


def generate_demo_report(gold_run: Path, reports_dir: Path = Path("data/reports")) -> Path:
    if not gold_run.is_dir():
        raise FileNotFoundError(f"Gold run directory not found: {gold_run}")
    required = (
        "route_daily_trips",
        "route_period_summary",
        "route_hourly_headway",
        "stop_daily_departures",
        "network_daily_summary",
        "busiest_route_day",
        "busiest_stop_day",
    )
    missing = [f"{name}.csv" for name in required if not (gold_run /
f"{name}.csv").is_file()]
    if missing:
        raise ValueError(f"Demo report requires missing gold files: {', '.join(missing)}")
    route_daily = _read_csv(gold_run, "route_daily_trips")
```

```
    route_period = _read_csv(gold_run, "route_period_summary")
    headway = _read_csv(gold_run, "route_hourly_headway")
    stop_daily = _read_csv(gold_run, "stop_daily_departures")
    network = _read_csv(gold_run, "network_daily_summary")
    busiest_route_day = _read_csv(gold_run, "busiest_route_day")
    busiest_stop_day = _read_csv(gold_run, "busiest_stop_day")
    run_id = gold_run.name

    top_routes = route_period[["route_id", "route_short_name", "route_long_name",
"active_service_days", "total_scheduled_trips", "average_trips_per_active_day",
"max_daily_trips"]].head(10)
    top_stops = stop_daily.groupby(["stop_id", "stop_name"], as_index=False,
dropna=False)["scheduled_departures_count"].sum().sort_values("scheduled_departures_count",
ascending=False).head(10)
    main_sample = route_daily.sort_values(["service_date", "scheduled_trips_count"],
ascending=[True, False]).head(10)
    service_period = {
        "first_service_date": network["service_date"].min() if not network.empty else None,
        "last_service_date": network["service_date"].max() if not network.empty else None,
        "service_days": int(network["service_date"].nunique()) if not network.empty else 0,
        "total_scheduled_trips": int(network["scheduled_trips_count"].sum()) if not
network.empty else 0,
        "total_scheduled_stop_departures":
int(network["scheduled_stop_departures_count"].sum()) if not network.empty else 0,
    }
    headway_sample = headway.sort_values(["service_date", "route_id",
"departure_hour"]).head(10)
    chart_refs = _chart_references(run_id, reports_dir)

    quality_text, _ = _quality_summary(run_id, reports_dir)

    report = f"""# Mobility Control Tower ? Static GTFS Demo Report

## 1. Dataset and run information

- Dataset: TissØo static GTFS schedule
- Run ID: ‘{run_id}‘
- Gold directory: ‘{gold_run}‘

## 2. What the pipeline did

The local pipeline preserved the source ZIP, profiled it, extracted bronze files, cleaned
canonical silver tables, validated them, expanded GTFS service dates, and aggregated static
planning KPIs.

## 3. Quality summary

{quality_text}

## 4. Service period summary

- First service date: ‘{service_period[’first_service_date’]}‘
- Last service date: ‘{service_period[’last_service_date’]}‘
- Service days represented: ‘{service_period[’service_days’]}‘
- Scheduled trips over the GTFS service period: ‘{service_period[’total_scheduled_trips’]}‘
- Scheduled stop departures over the GTFS service period:
‘{service_period[’total_scheduled_stop_departures’]}‘

## 5. Main KPI: scheduled trips by route and day

This KPI counts scheduled trips at the service-date and route grain. Sample:

{_markdown_table(main_sample)}

## 6. Top routes over the full service period

The totals below are aggregated over available GTFS service dates, not per day.

{_markdown_table(top_routes)}
```

```
## 7. Busiest route/day combinations

These are the highest individual route/day combinations by scheduled trips.

{_markdown_table(busiest_route_day.head(10))}

## 8. Planned hourly departures and approximate headway

Hourly departures count scheduled trip starts, using the first stop of each trip.
‘planned_headway_minutes‘ is a planned headway approximation: ‘60 /
scheduled_departures_count‘. It is not real passenger waiting time.

{_markdown_table(headway_sample)}

## 9. Busiest stops

The totals below are scheduled departures aggregated over the GTFS service period.

{_markdown_table(top_stops)}

## 10. Network daily summary

First seven service dates:

{_markdown_table(network.sort_values(’service_date’).head(7))}

## 11. Chart references if generated

{chart_refs}

## 12. What this proves technically

The project can reproducibly transform a published transport archive into validated,
explainable analytical tables with service-calendar logic, after-midnight GTFS semantics,
manifests, reports, and static PNG evidence.

## 13. What it does not prove yet

- These are static planning KPIs, not real-time reliability KPIs.
- They count scheduled trips and scheduled departures, not actual trips.
- Planned headway approximation is not real passenger waiting time.
- Frequency-based service is not expanded.
- There is intentionally no API, database, dashboard, streaming system, or orchestration
layer yet.

## 14. Next project step

Use these static planning KPIs as the baseline for the next bounded project phase. Real-time
data can later compare observed service against this planned schedule.
"""
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"mobility_demo_{run_id}.md"
    output.write_text(report, encoding="utf-8")
    return output


def generate_static_mvp_report(gold_run: Path, reports_dir: Path = Path("data/reports")) ->
Path:
    if not gold_run.is_dir():
        raise FileNotFoundError(f"Gold run directory not found: {gold_run}")
    run_id = gold_run.name
    route_period = _read_csv(gold_run, "route_period_summary")
    network = _read_csv(gold_run, "network_daily_summary")
    stop_daily = _read_csv(gold_run, "stop_daily_departures")
    manifest_path = gold_run / "gold_manifest.json"
    manifest = json.loads(manifest_path.read_text(encoding="utf-8")) if
manifest_path.is_file() else {"tables_created": {}}
    quality_text, quality_counts = _quality_summary(run_id, reports_dir)
```

```
    top_routes = route_period[["route_short_name", "route_long_name",
"total_scheduled_trips", "active_service_days", "average_trips_per_active_day"]].head(5)
    busiest_day = network.sort_values("scheduled_trips_count", ascending=False).head(1)
    busiest_stop = stop_daily.groupby(["stop_id", "stop_name"], as_index=False,
dropna=False)["scheduled_departures_count"].sum().sort_values("scheduled_departures_count",
ascending=False).head(1)
    table_names = pd.DataFrame(
        [{"table": name, "rows": details.get("row_count", 0)} for name, details in
manifest.get("tables_created", {}).items()]
    ).sort_values("table") if manifest.get("tables_created") else pd.DataFrame()
    checks_sentence = quality_text
    if quality_counts:
        checks_sentence = f"{quality_counts['PASS']} checks passed, {quality_counts['WARN']}
warnings, {quality_counts['FAIL']} failures."

    report = f"""# Mobility Control Tower ? Static MVP Evidence

## Dataset used

TissØo static GTFS for run '{run_id}'. The dataset describes the scheduled public-transport
offer, not observed real-time operations.

## What the pipeline did

The pipeline preserved the raw ZIP, captured metadata, profiled GTFS files, extracted bronze
files, created cleaned silver tables, validated data quality, and built gold static planning
KPIs.

## Data-quality evidence

{checks_sentence}

## Clean tables and KPI tables produced

{_markdown_table(table_names)}

## Main numerical results

- Service period: '{network['service_date'].min()}' to '{network['service_date'].max()}'
- Scheduled trips over the GTFS service period:
'{int(network['scheduled_trips_count'].sum())}'
- Scheduled stop departures over the GTFS service period:
'{int(network['scheduled_stop_departures_count'].sum())}'
- Active routes in gold route summary: '{int(route_period['route_id'].nunique())}'

Top routes over the GTFS service period:

{_markdown_table(top_routes)}

Busiest service day by scheduled trips:

{_markdown_table(busiest_day)}

Busiest stop over the GTFS service period:

{_markdown_table(busiest_stop)}

## Why this is Data Engineering

The work is not only a visualization. It implements repeatable ingestion, immutable raw
preservation, layered transformations, schema-aware validation, reproducible KPI tables,
manifests, tests, and generated evidence from source data.

## Next logical phase

Keep the static GTFS layer as the planned-service baseline. The next phase can add another
bounded capability while preserving the same raw-to-report discipline.
"""
```

```
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"static_mvp_evidence_{run_id}.md"
    output.write_text(report, encoding="utf-8")
    return output
'''

src/mobility_control_tower/reporting/final_report.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/reporting/final_report
.py
FILE: src/mobility_control_tower/reporting/final_report.py
'''python
"""Generate the final academic MVP report."""

from __future__ import annotations

import json
from pathlib import Path

import duckdb


def _table(rows: list[dict]) -> str:
    if not rows:
        return "No rows available."
    headers = list(rows[0])
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join("---" for _ in headers) +
" |"]
    for row in rows:
        lines.append("| " + " | ".join(str(row.get(header, "")).replace("|", "\\|") for
header in headers) + " |")
    return "\n".join(lines)


def _query(db_path: Path, sql: str) -> list[dict]:
    if not db_path.is_file():
        return []
    with duckdb.connect(str(db_path), read_only=True) as connection:
        try:
            return connection.execute(sql).fetchdf().where(lambda frame: frame.notna(),
None).to_dict("records")
        except duckdb.Error:
            return []


def generate_final_report(serving_run: Path, reports_dir: Path = Path("data/reports")) ->
Path:
    if not serving_run.is_dir():
        raise FileNotFoundError(f"Serving run directory not found: {serving_run}")
    manifest_path = serving_run / "serving_manifest.json"
    if not manifest_path.is_file():
        raise FileNotFoundError(f"Serving manifest not found: {manifest_path}")
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    db_path = Path(manifest["database_path"])
    static_tables = sorted(name for name in manifest.get("tables_loaded", {}) if not
name.startswith("rt_"))
    realtime_tables = sorted(name for name in manifest.get("tables_loaded", {}) if
name.startswith("rt_"))
    top_routes = _query(db_path, "SELECT route_short_name, route_long_name,
total_scheduled_trips FROM v_top_routes_static LIMIT 5")
    feed_health = _query(db_path, "SELECT feed_type, feed_age_seconds, freshness_status,
entity_count FROM v_rt_feed_health")
    compatibility = _query(db_path, "SELECT identifier_type, match_percentage, status FROM
v_rt_identifier_compatibility")
    report = f"""# Mobility Control Tower - Final Project Report

## 1. Project objective

Mobility Control Tower is a local Data Engineering platform that processes public transport
```

data and produces explainable indicators for an academic PFA demonstration.

2. Data source

The project uses Tisseo Toulouse static GTFS and bounded GTFS-Realtime snapshots from official public transport data sources.

3. Static pipeline

Static GTFS is preserved as raw data, extracted to bronze, cleaned into silver tables, validated, and aggregated into gold static planning KPIs.

4. Realtime snapshot pipeline

One GTFS-Realtime snapshot is preserved as ‘feed.pb‘, parsed into CSV tables, compared with static GTFS identifiers, and summarized as snapshot delay indicators.

5. Data quality

The project includes static GTFS quality checks and static/live compatibility checks. Reports are generated as Markdown and JSON artifacts.

6. Static KPIs

Loaded static gold tables: ‘{len(static_tables)}‘.

Top routes over the static service period:

{_table(top_routes)}

7. Realtime snapshot KPIs

Loaded realtime snapshot tables: ‘{len(realtime_tables)}‘.

Feed health:

{_table(feed_health)}

Identifier compatibility:

{_table(compatibility)}

8. Serving layer

The local DuckDB serving layer packages generated CSV data products into SQL tables and views for predefined queries.

9. API layer

The FastAPI layer exposes DuckDB views through a local read-only API. It serves DuckDB data products and does not expose arbitrary SQL.

10. Dashboard layer

The Streamlit dashboard consumes the local API and presents static planning KPIs plus GTFS-Realtime snapshot indicators for a teacher demo.

11. Technical skills demonstrated

- Data ingestion and raw preservation
- Metadata and checksums
- Layered transformations: raw, bronze, silver, gold
- Data-quality validation
- KPI design
- GTFS-Realtime protobuf parsing
- Static/live compatibility analysis
- DuckDB serving
- FastAPI read-only API
- Streamlit local dashboard

- Automated pytest coverage
- Teacher-facing documentation

12. Limitations

- This is a local academic MVP, not an enterprise platform.
- Realtime work is snapshot-based, not streaming.
- No production monitoring system is implemented.
- No authentication, deployment, cloud, Docker, Kafka, Spark, Airflow, PostgreSQL, ML, or RAG is included.

13. Future work

- Collect repeated GTFS-Realtime snapshots.
- Improve trip-level matching.
- Add a richer dashboard after the API contract is stable.
- Evaluate production storage only after academic MVP validation.

14. Final academic MVP status

The project is ready for a local academic demonstration.
"""
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"final_project_report_{serving_run.name}.md"
    output.write_text(report, encoding="utf-8")
    return output

'''

src/mobility_control_tower/serving/__init__.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/serving/__init__.py
FILE: src/mobility_control_tower/serving/__init__.py
'''python
"""Local DuckDB serving layer."""

'''

src/mobility_control_tower/serving/duckdb_loader.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/serving/duckdb_loader.py
FILE: src/mobility_control_tower/serving/duckdb_loader.py
'''python
"""Build and query a local DuckDB serving database from generated CSV data products."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import duckdb
import pandas as pd

from mobility_control_tower.serving.sql_views import QUERY_SQL, create_views

STATIC_TABLES = (
    "route_daily_trips",
    "route_hourly_departures",
    "stop_daily_departures",
    "network_daily_summary",
    "route_period_summary",
    "route_hourly_headway",
    "route_type_daily_summary",
    "busiest_route_day",
    "busiest_stop_day",

```
)
ESSENTIAL_STATIC_TABLES = ("route_daily_trips", "network_daily_summary",
"route_period_summary")
RT_TABLES = (
    "rt_feed_health_snapshot",
    "rt_trip_update_enriched",
    "rt_route_delay_snapshot",
    "rt_stop_delay_snapshot",
    "rt_identifier_compatibility_snapshot",
)


def _csv_path(run_dir: Path, table: str) -> Path:
    return run_dir / f"{table}.csv"


def _load_csv_table(connection: duckdb.DuckDBPyConnection, table: str, csv_path: Path) ->
dict[str, Any]:
    connection.execute(f"CREATE OR REPLACE TABLE {table} AS SELECT * FROM read_csv_auto(?,
header=true)", [str(csv_path)])
    columns = [row[1] for row in connection.execute(f"PRAGMA
table_info('{table}')").fetchall()]
    row_count = connection.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
    return {"file": str(csv_path), "row_count": int(row_count), "columns": columns}


def _validate_essential_static(gold_run: Path) -> None:
    missing = [f"{table}.csv" for table in ESSENTIAL_STATIC_TABLES if not
_csv_path(gold_run, table).is_file()]
    if missing:
        raise ValueError(f"Missing essential static gold files: {', '.join(missing)}")


def build_serving_database(gold_run: Path, rt_gold_run: Path | None = None, serving_root:
Path = Path("data/serving")) -> Path:
    if not gold_run.is_dir():
        raise FileNotFoundError(f"Static gold run directory not found: {gold_run}")
    if rt_gold_run is not None and not rt_gold_run.is_dir():
        raise FileNotFoundError(f"Real-time gold run directory not found: {rt_gold_run}")
    _validate_essential_static(gold_run)
    source_id = gold_run.parent.name
    output_dir = serving_root / source_id / gold_run.name
    output_dir.mkdir(parents=True, exist_ok=True)
    db_path = output_dir / "mobility_control_tower.duckdb"
    if db_path.exists():
        db_path.unlink()
    loaded_tables: dict[str, dict[str, Any]] = {}
    with duckdb.connect(str(db_path)) as connection:
        for table in STATIC_TABLES:
            path = _csv_path(gold_run, table)
            if path.is_file():
                loaded_tables[table] = _load_csv_table(connection, table, path)
        if rt_gold_run is not None:
            for table in RT_TABLES:
                path = _csv_path(rt_gold_run, table)
                if path.is_file():
                    loaded_tables[table] = _load_csv_table(connection, table, path)
        views_created = create_views(connection, set(loaded_tables))
    manifest = {
        "generated_timestamp": datetime.now(timezone.utc).isoformat(),
        "static_gold_run_path": str(gold_run),
        "realtime_gold_run_path": str(rt_gold_run) if rt_gold_run else None,
        "realtime_run_id": rt_gold_run.name if rt_gold_run else None,
        "database_path": str(db_path),
        "tables_loaded": loaded_tables,
        "views_created": views_created,
        "example_query_names": sorted(QUERY_SQL),
        "assumptions": [
            "CSV gold outputs are the source of truth for this local serving database.",
```

```
            "DuckDB is used as an embedded analytical database, not a server.",
            "Real-time tables remain snapshot-based when loaded.",
        ],
        "limitations": [
            "This is not a production database.",
            "No API, dashboard, scheduler, or streaming system is created.",
            "Rebuild the database after regenerating CSV data products.",
        ],
    }
    (output_dir / "serving_manifest.json").write_text(json.dumps(manifest, indent=2,
ensure_ascii=False) + "\n", encoding="utf-8")
    return output_dir


def query_serving_database(db_path: Path, query_name: str, limit: int = 10) -> pd.DataFrame:
    if query_name not in QUERY_SQL:
        available = ", ".join(sorted(QUERY_SQL))
        raise ValueError(f"Unknown query name '{query_name}'. Available query names:
{available}")
    if not db_path.is_file():
        raise FileNotFoundError(f"DuckDB database not found: {db_path}")
    safe_limit = max(1, min(int(limit), 1000))
    sql = QUERY_SQL[query_name].format(limit=safe_limit)
    with duckdb.connect(str(db_path), read_only=True) as connection:
        try:
            return connection.execute(sql).fetchdf()
        except duckdb.CatalogException as exc:
            raise ValueError(f"Query '{query_name}' is unavailable because its required
view/table is missing") from exc


def dataframe_to_text_table(frame: pd.DataFrame) -> str:
    if frame.empty:
        return "No rows returned."
    text_frame = frame.astype(str)
    columns = list(text_frame.columns)
    widths = {column: max(len(column), *(len(value) for value in
text_frame[column].tolist())) for column in columns}
    header = " | ".join(column.ljust(widths[column]) for column in columns)
    separator = "-+-".join("-" * widths[column] for column in columns)
    rows = [" | ".join(row[column].ljust(widths[column]) for column in columns) for _, row
in text_frame.iterrows()]
    return "\n".join([header, separator, *rows])
'''

src/mobility_control_tower/serving/serving_report.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/serving/serving_report
.py
FILE: src/mobility_control_tower/serving/serving_report.py
'''python
"""Generate a Markdown report for the local DuckDB serving layer."""

from __future__ import annotations

import json
from pathlib import Path

import pandas as pd

from mobility_control_tower.serving.duckdb_loader import query_serving_database


def _markdown_table(frame: pd.DataFrame) -> str:
    if frame.empty:
        return "No rows available."
    headers = [str(column) for column in frame.columns]
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join("---" for _ in headers) +
" |"]
```

```
    for row in frame.itertuples(index=False, name=None):
        lines.append("| " + " | ".join(str(value).replace("|", "\\|") for value in row) + "
|")
    return "\n".join(lines)


def _try_query(db_path: Path, query_name: str, limit: int = 10) -> str:
    try:
        return _markdown_table(query_serving_database(db_path, query_name, limit))
    except ValueError:
        return "Unavailable for this serving run."


def generate_serving_report(serving_run: Path, reports_dir: Path = Path("data/reports")) ->
Path:
    if not serving_run.is_dir():
        raise FileNotFoundError(f"Serving run directory not found: {serving_run}")
    manifest_path = serving_run / "serving_manifest.json"
    if not manifest_path.is_file():
        raise FileNotFoundError(f"Serving manifest not found: {manifest_path}")
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    db_path = Path(manifest["database_path"])
    tables = manifest.get("tables_loaded", {})
    static_tables = {name: details for name, details in tables.items() if not
name.startswith("rt_")}
    realtime_tables = {name: details for name, details in tables.items() if
name.startswith("rt_")}
    table_rows = pd.DataFrame(
        [{"table": name, "rows": details.get("row_count", 0)} for name, details in
sorted(tables.items())]
    )
    views = pd.DataFrame([{"view": name} for name in manifest.get("views_created", [])])
    report = f"""# Mobility Control Tower - Serving Layer Report

## 1. What the serving database contains

This local DuckDB serving layer stores queryable data products built from generated CSV
files. It is not a production database.

- Database: `{db_path}`
- Static gold run: `{manifest.get('static_gold_run_path')}`
- Real-time gold run: `{manifest.get('realtime_gold_run_path') or 'not loaded'}`

## 2. Static tables loaded

{_markdown_table(pd.DataFrame([{"table": name, "rows": details.get("row_count", 0)} for
name, details in sorted(static_tables.items())]))}

## 3. Real-time snapshot tables loaded

{_markdown_table(pd.DataFrame([{"table": name, "rows": details.get("row_count", 0)} for
name, details in sorted(realtime_tables.items())])) if realtime_tables else "No real-time
snapshot tables loaded."}

## 4. SQL views created

{_markdown_table(views)}

## 5. Example query results

### Network overview - first 7 days

{_try_query(db_path, "network-overview", 7)}

### Top 10 routes over the static service period

{_try_query(db_path, "top-routes", 10)}

### Route type summary sample
```

````
{_try_query(db_path, "route-types", 10)}

### Real-time feed health

{_try_query(db_path, "rt-feed-health", 10)}

### Real-time identifier compatibility

{_try_query(db_path, "rt-compatibility", 10)}

### Top delayed routes snapshot

{_try_query(db_path, "rt-top-delayed-routes", 10)}

## 6. What this proves technically

The project can package static and snapshot-based real-time outputs into a local DuckDB file
with SQL views and predefined query examples. This makes the generated data products easy to
inspect with SQL.

## 7. Current limitations

- This is a local DuckDB serving layer, not a production database.
- There is no API, dashboard, scheduler, or streaming system.
- Real-time outputs remain snapshot-based.
- Rebuild the database after regenerating CSV files.

## 8. Next project step

Use the local SQL views during demos and decide which views would be worth exposing later
through an API or dashboard.
"""
    reports_dir.mkdir(parents=True, exist_ok=True)
    output = reports_dir / f"serving_report_{serving_run.name}.md"
    output.write_text(report, encoding="utf-8")
    return output
'''

src/mobility_control_tower/serving/sql_views.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/serving/sql_views.py
FILE: src/mobility_control_tower/serving/sql_views.py
```python
"""SQL view definitions for the local DuckDB serving layer."""

from __future__ import annotations

import duckdb


STATIC_VIEW_SQL = {
    "v_network_overview": """
        CREATE OR REPLACE VIEW v_network_overview AS
        SELECT
            service_date,
            active_routes_count,
            scheduled_trips_count,
            scheduled_stop_departures_count,
            active_stops_count
        FROM network_daily_summary
        ORDER BY service_date
    """,
    "v_top_routes_static": """
        CREATE OR REPLACE VIEW v_top_routes_static AS
        SELECT
            route_id,
            route_short_name,
            route_long_name,
````
````

```
            route_type,
            active_service_days,
            total_scheduled_trips,
            average_trips_per_active_day,
            max_daily_trips
        FROM route_period_summary
        ORDER BY total_scheduled_trips DESC
    """,
    "v_route_hourly_headway": """
        CREATE OR REPLACE VIEW v_route_hourly_headway AS
        SELECT
            service_date,
            route_id,
            route_short_name,
            route_long_name,
            departure_hour,
            scheduled_departures_count,
            planned_headway_minutes
        FROM route_hourly_headway
    """,
    "v_route_type_daily_summary": """
        CREATE OR REPLACE VIEW v_route_type_daily_summary AS
        SELECT *
        FROM route_type_daily_summary
        ORDER BY service_date, route_type
    """,
}

STATIC_VIEW_REQUIREMENTS = {
    "v_network_overview": "network_daily_summary",
    "v_top_routes_static": "route_period_summary",
    "v_route_hourly_headway": "route_hourly_headway",
    "v_route_type_daily_summary": "route_type_daily_summary",
}


OPTIONAL_RT_VIEW_SQL = {
    "v_rt_feed_health": """
        CREATE OR REPLACE VIEW v_rt_feed_health AS
        SELECT *
        FROM rt_feed_health_snapshot
    """,
    "v_rt_identifier_compatibility": """
        CREATE OR REPLACE VIEW v_rt_identifier_compatibility AS
        SELECT *
        FROM rt_identifier_compatibility_snapshot
        ORDER BY identifier_type
    """,
    "v_rt_top_delayed_routes_snapshot": """
        CREATE OR REPLACE VIEW v_rt_top_delayed_routes_snapshot AS
        SELECT
            route_id,
            route_short_name,
            route_long_name,
            stop_time_updates_count,
            distinct_trip_updates_count,
            avg_delay_seconds,
            median_delay_seconds,
            max_delay_seconds,
            delayed_updates_5min_count,
            delayed_updates_5min_pct
        FROM rt_route_delay_snapshot
        ORDER BY avg_delay_seconds DESC NULLS LAST
    """,
    "v_rt_top_delayed_stops_snapshot": """
        CREATE OR REPLACE VIEW v_rt_top_delayed_stops_snapshot AS
        SELECT *
        FROM rt_stop_delay_snapshot
        ORDER BY avg_delay_seconds DESC NULLS LAST
```

```
    """,
}


RT_VIEW_REQUIREMENTS = {
    "v_rt_feed_health": "rt_feed_health_snapshot",
    "v_rt_identifier_compatibility": "rt_identifier_compatibility_snapshot",
    "v_rt_top_delayed_routes_snapshot": "rt_route_delay_snapshot",
    "v_rt_top_delayed_stops_snapshot": "rt_stop_delay_snapshot",
}


def create_views(connection: duckdb.DuckDBPyConnection, loaded_tables: set[str]) ->
list[str]:
    created: list[str] = []
    for view_name, sql in STATIC_VIEW_SQL.items():
        required_table = STATIC_VIEW_REQUIREMENTS[view_name]
        if required_table in loaded_tables:
            connection.execute(sql)
            created.append(view_name)
    for view_name, sql in OPTIONAL_RT_VIEW_SQL.items():
        required_table = RT_VIEW_REQUIREMENTS[view_name]
        if required_table in loaded_tables:
            connection.execute(sql)
            created.append(view_name)
    return created


QUERY_SQL = {
    "network-overview": "SELECT * FROM v_network_overview ORDER BY service_date LIMIT
{limit}",
    "top-routes": "SELECT * FROM v_top_routes_static LIMIT {limit}",
    "hourly-headway": "SELECT * FROM v_route_hourly_headway ORDER BY service_date, route_id,
departure_hour LIMIT {limit}",
    "route-types": "SELECT * FROM v_route_type_daily_summary LIMIT {limit}",
    "rt-feed-health": "SELECT * FROM v_rt_feed_health LIMIT {limit}",
    "rt-compatibility": "SELECT * FROM v_rt_identifier_compatibility LIMIT {limit}",
    "rt-top-delayed-routes": "SELECT * FROM v_rt_top_delayed_routes_snapshot LIMIT {limit}",
    "rt-top-delayed-stops": "SELECT * FROM v_rt_top_delayed_stops_snapshot LIMIT {limit}",
}
'''

src/mobility_control_tower/transformations/__init__.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/transformations/__init
__.py
FILE: src/mobility_control_tower/transformations/__init__.py
'''python
"""GTFS bronze and silver transformations."""
'''

src/mobility_control_tower/transformations/gtfs_bronze.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/transformations/gtfs_b
ronze.py
FILE: src/mobility_control_tower/transformations/gtfs_bronze.py
'''python
"""Extract preserved GTFS files into a faithful bronze layer."""

from __future__ import annotations

import csv
import io
import json
import shutil
import zipfile
from datetime import datetime, timezone
from pathlib import Path, PurePosixPath
from typing import Any
```

```
GTFS_FILES = (
    "agency.txt",
    "stops.txt",
    "routes.txt",
    "trips.txt",
    "stop_times.txt",
    "calendar.txt",
    "calendar_dates.txt",
    "shapes.txt",
    "frequencies.txt",
)


def _members_by_basename(archive: zipfile.ZipFile) -> dict[str, str]:
    members: dict[str, str] = {}
    for name in archive.namelist():
        if not name.endswith("/"):
            members.setdefault(PurePosixPath(name).name.lower(), name)
    return members


def _inspect_csv(content: bytes, filename: str) -> tuple[list[str], int]:
    try:
        reader = csv.reader(io.StringIO(content.decode("utf-8-sig"), newline=""))
        columns = next(reader, [])
        return columns, sum(1 for row in reader if row)
    except (UnicodeDecodeError, csv.Error) as exc:
        raise ValueError(f"Cannot read GTFS file '{filename}' as UTF-8 CSV: {exc}") from exc


def build_bronze(
    raw_run: Path,
    bronze_root: Path = Path("data/bronze"),
    generated_at: datetime | None = None,
) -> Path:
    """Extract selected files byte-for-byte and create a bronze manifest."""
    if not raw_run.is_dir():
        raise FileNotFoundError(f"Raw run directory not found: {raw_run}")
    zip_files = sorted(raw_run.glob("*.zip"))
    if len(zip_files) != 1:
        raise ValueError(f"Expected exactly one ZIP in {raw_run}, found {len(zip_files)}")
    if not zipfile.is_zipfile(zip_files[0]):
        raise ValueError(f"Raw archive is not a readable ZIP: {zip_files[0]}")

    source_id = raw_run.parent.name
    output_dir = bronze_root / source_id / raw_run.name
    output_dir.mkdir(parents=True, exist_ok=False)
    extracted: dict[str, dict[str, Any]] = {}
    try:
        with zipfile.ZipFile(zip_files[0]) as archive:
            members = _members_by_basename(archive)
            for filename in GTFS_FILES:
                member = members.get(filename)
                if member is None:
                    continue
                content = archive.read(member)
                columns, row_count = _inspect_csv(content, filename)
                (output_dir / filename).write_bytes(content)
                extracted[filename] = {"row_count": row_count, "columns": columns}

        raw_metadata_path = raw_run / "metadata.json"
        raw_metadata = json.loads(raw_metadata_path.read_text(encoding="utf-8")) if
raw_metadata_path.is_file() else {}
        timestamp = (generated_at or datetime.now(timezone.utc)).astimezone(timezone.utc)
        manifest = {
            "source_raw_run": str(raw_run),
            "source_zip": zip_files[0].name,
```

```
            "source_zip_sha256": raw_metadata.get("sha256"),
            "generated_timestamp": timestamp.isoformat(),
            "extracted_files": extracted,
            "missing_important_files": [name for name in GTFS_FILES if name not in
extracted],
        }
        (output_dir / "bronze_manifest.json").write_text(
            json.dumps(manifest, indent=2, ensure_ascii=False) + "\n", encoding="utf-8"
        )
    except Exception:
        shutil.rmtree(output_dir)
        raise
    return output_dir
'''

src/mobility_control_tower/transformations/gtfs_silver.py
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower/transformations/gtfs_s
ilver.py
FILE: src/mobility_control_tower/transformations/gtfs_silver.py
'''python
"""Create cleaned, canonical CSV tables from bronze GTFS files."""

from __future__ import annotations

import csv
import json
import re
import shutil
from datetime import datetime, timezone
from pathlib import Path
from typing import Any


SILVER_TABLES = (
    "agency",
    "stops",
    "routes",
    "trips",
    "stop_times",
    "calendar",
    "calendar_dates",
)
TIME_PATTERN = re.compile(r"^(\d{1,3}):([0-5]\d):([0-5]\d)$")


def parse_gtfs_time(value: str | None) -> int | None:
    """Return service-day seconds; GTFS hours may exceed 23."""
    if value is None:
        return None
    match = TIME_PATTERN.fullmatch(value.strip())
    if not match:
        return None
    hours, minutes, seconds = (int(part) for part in match.groups())
    return hours * 3600 + minutes * 60 + seconds


def parse_gtfs_date(value: str | None) -> str | None:
    """Convert a valid GTFS YYYYMMDD date to ISO text without changing the source field."""
    if not value:
        return None
    try:
        return datetime.strptime(value.strip(), "%Y%m%d").date().isoformat()
    except ValueError:
        return None


def read_csv_table(path: Path) -> tuple[list[str], list[dict[str, str]]]:
    try:
```

```
        with path.open(encoding="utf-8-sig", newline="") as handle:
            reader = csv.DictReader(handle)
            original_columns = list(reader.fieldnames or [])
            columns = [column.strip() for column in original_columns]
            if not columns:
                raise ValueError("header is empty")
            if len(columns) != len(set(columns)):
                raise ValueError("column names are duplicated after trimming")
            rows = [
                {clean: (row.get(original) or "").strip() for original, clean in
zip(original_columns, columns)}
                for row in reader
            ]
            return columns, rows
    except (UnicodeDecodeError, csv.Error) as exc:
        raise ValueError(f"Cannot read bronze table {path}: {exc}") from exc


def write_csv_table(path: Path, columns: list[str], rows: list[dict[str, Any]]) -> None:
    with path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=columns, extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)


def build_silver(
    bronze_run: Path,
    silver_root: Path = Path("data/silver"),
    generated_at: datetime | None = None,
) -> Path:
    if not bronze_run.is_dir():
        raise FileNotFoundError(f"Bronze run directory not found: {bronze_run}")
    source_id = bronze_run.parent.name
    output_dir = silver_root / source_id / bronze_run.name
    output_dir.mkdir(parents=True, exist_ok=False)
    tables: dict[str, dict[str, Any]] = {}
    try:
        for table in SILVER_TABLES:
            source_path = bronze_run / f"{table}.txt"
            if not source_path.is_file():
                continue
            columns, rows = read_csv_table(source_path)
            if table == "stop_times":
                columns.extend(["arrival_time_seconds", "departure_time_seconds"])
                for row in rows:
                    row["arrival_time_seconds"] = parse_gtfs_time(row.get("arrival_time"))
                    row["departure_time_seconds"] =
parse_gtfs_time(row.get("departure_time"))
            for date_column in ("start_date", "end_date", "date"):
                if date_column in columns:
                    iso_column = f"{date_column}_iso"
                    columns.append(iso_column)
                    for row in rows:
                        row[iso_column] = parse_gtfs_date(row.get(date_column))
            output_path = output_dir / f"{table}.csv"
            write_csv_table(output_path, columns, rows)
            tables[table] = {"file": output_path.name, "row_count": len(rows), "columns":
columns}

        timestamp = (generated_at or datetime.now(timezone.utc)).astimezone(timezone.utc)
        manifest = {
            "source_bronze_run": str(bronze_run),
            "generated_timestamp": timestamp.isoformat(),
            "tables_created": tables,
            "cleaning_notes": [
                "Column names and surrounding string whitespace were trimmed.",
                "Source identifiers were preserved; no identifiers were invented.",
                "Original GTFS dates and times remain strings.",
                "Valid YYYYMMDD dates also have non-destructive ISO date companion
```

```
columns.",
                "Valid stop_times values also have service-day seconds columns; hours above
23 are supported.",
            ],
        }
        (output_dir / "silver_manifest.json").write_text(
            json.dumps(manifest, indent=2, ensure_ascii=False) + "\n", encoding="utf-8"
        )
    except Exception:
        shutil.rmtree(output_dir)
        raise
    return output_dir
'''

src/mobility_control_tower.egg-info/dependency_links.txt
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower.egg-info/dependency_li
nks.txt
FILE: src/mobility_control_tower.egg-info/dependency_links.txt
'''text

'''

src/mobility_control_tower.egg-info/entry_points.txt
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower.egg-info/entry_points.
txt
FILE: src/mobility_control_tower.egg-info/entry_points.txt
'''text
[console_scripts]
mobility-control-tower = mobility_control_tower.cli:main
'''

src/mobility_control_tower.egg-info/PKG-INFO
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower.egg-info/PKG-INFO
FILE: src/mobility_control_tower.egg-info/PKG-INFO
'''text
Metadata-Version: 2.4
Name: mobility-control-tower
Version: 0.1.0
Summary: Local public transport data engineering platform
Requires-Python: >=3.10
Description-Content-Type: text/markdown
Requires-Dist: duckdb<2,>=1.0
Requires-Dist: fastapi<1,>=0.115
Requires-Dist: gtfs-realtime-bindings<2,>=1.0
Requires-Dist: matplotlib<4,>=3.8
Requires-Dist: pandas<3,>=2.0
Requires-Dist: PyYAML<7,>=6.0
Requires-Dist: requests<3,>=2.31
Requires-Dist: streamlit<2,>=1.40
Requires-Dist: uvicorn<1,>=0.30
Provides-Extra: dev
Requires-Dist: pytest<9,>=8; extra == "dev"

# Mobility Control Tower

Mobility Control Tower is a local data-engineering project that turns public-transport data
into explainable planning indicators. The current static MVP preserves and profiles TissØo
static GTFS, creates bronze and cleaned silver tables, validates them, builds gold static
planning KPIs, generates PNG charts, and writes teacher-facing Markdown evidence reports. It
also includes a bounded GTFS-Realtime snapshot exploration layer.

## Dataset

This phase uses the daily static GTFS for the TissØo urban network. It represents the
theoretical offer (stops, routes, trips, and schedules) for approximately the next three
weeks under the ODbL licence. See [docs/data_source.md](docs/data_source.md).
```

Install

Python 3.10 or newer is required.

```bash
python -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev]'
```

Run

For the dependable manual workflow, place `Tisseo_GTFS.zip` in `data/raw/manual/`, then run the commands below. Replace `<run_id>` with the timestamp printed by ingestion.

```bash
python -m mobility_control_tower.cli ingest-gtfs --source tisseo --local-zip data/raw/manual/Tisseo_GTFS.zip
python -m mobility_control_tower.cli profile-gtfs --raw-run data/raw/tisseo/<run_id>
python -m mobility_control_tower.cli build-bronze --raw-run data/raw/tisseo/<run_id>
python -m mobility_control_tower.cli build-silver --bronze-run data/bronze/tisseo/<run_id>
python -m mobility_control_tower.cli validate-gtfs --silver-run data/silver/tisseo/<run_id>
python -m mobility_control_tower.cli build-gold --silver-run data/silver/tisseo/<run_id>
python -m mobility_control_tower.cli generate-static-charts --gold-run data/gold/tisseo/<run_id>
python -m mobility_control_tower.cli generate-demo-report --gold-run data/gold/tisseo/<run_id>
python -m mobility_control_tower.cli generate-static-mvp-report --gold-run data/gold/tisseo/<run_id>
```

Automatic download from the source URL configured in `config/sources.yml` is also supported:

```bash
python -m mobility_control_tower.cli ingest-gtfs --source tisseo --download
```

Ingestion creates an immutable ZIP and metadata under `data/raw/`. Bronze preserves selected GTFS text files. Silver produces cleaned CSV tables. Gold produces schedule KPI tables under `data/gold/`. Each layer has a manifest. Profiling, validation, charts, the demo report, and the static MVP evidence report are generated under `data/reports/`.

The current MVP is intentionally static: it proves a reproducible Data Engineering flow from a public GTFS ZIP to validated analytical tables and evidence artifacts. It does not use GTFS-RT and does not measure actual delays, cancellations, or passenger waiting time. See docs/runbook.md, docs/data_quality.md, docs/kpi_definitions.md, and docs/static_mvp_explanation.md.

GTFS-Realtime snapshot exploration

This project can fetch one GTFS-Realtime protobuf snapshot, preserve it as `feed.pb`, parse it into CSV files, report feed freshness and identifiers, and compare live identifiers with static silver GTFS tables.

```bash
python -m mobility_control_tower.cli fetch-gtfs-rt --source tisseo --feed-type trip_updates
python -m mobility_control_tower.cli parse-gtfs-rt --raw-rt-run data/raw_realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli report-gtfs-rt --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli check-rt-compatibility \
  --silver-run data/silver/tisseo/<static_run_id> \
  --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli build-rt-gold \
  --silver-run data/silver/tisseo/<static_run_id> \
  --rt-run data/realtime/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-rt-charts --rt-gold-run
```

```
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
python -m mobility_control_tower.cli generate-rt-snapshot-report --rt-gold-run
data/realtime_gold/tisseo/trip_updates/<rt_run_id>
```

The saved `feed.pb` file is a deterministic local snapshot. It can be reused for parser tests, replay experiments, demos without live internet, static/live compatibility checks, and snapshot KPI reports. Continuous streaming is not implemented. See docs/realtime_exploration.md and docs/realtime_snapshot_kpis.md.

Local DuckDB serving layer

DuckDB is used as a local SQL serving layer because it does not require a server and can load the generated CSV data products. Build a queryable database from static gold outputs, optionally including one real-time snapshot gold run:

```bash
python -m mobility_control_tower.cli build-serving-db \
  --gold-run data/gold/tisseo/<static_run_id> \
  --rt-gold-run data/realtime_gold/tisseo/trip_updates/<rt_run_id>

python -m mobility_control_tower.cli query-serving-db \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb \
  --query-name top-routes \
  --limit 10

python -m mobility_control_tower.cli generate-serving-report \
  --serving-run data/serving/tisseo/<static_run_id>
```

See docs/serving_layer.md.

Local read-only API

After building the DuckDB serving database, start the local API:

```bash
python -m mobility_control_tower.cli serve-api \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb \
  --host 127.0.0.1 \
  --port 8000
```

Example URLs:

- `http://127.0.0.1:8000/health`
- `http://127.0.0.1:8000/metadata`
- `http://127.0.0.1:8000/static/top-routes?limit=5`
- `http://127.0.0.1:8000/realtime/feed-health`

FastAPI interactive documentation is available at `http://127.0.0.1:8000/docs`. The API is local and read-only; it serves DuckDB data products and is not a production API.

Generate an API report:

```bash
python -m mobility_control_tower.cli generate-api-report \
  --db data/serving/tisseo/<static_run_id>/mobility_control_tower.duckdb
```

See docs/api.md.

Run tests with:

```bash
pytest
```

Scope

Production APIs, dashboards, continuous streaming, Kafka, Spark, Airflow, database servers, containers, cloud services, ML, and RAG are intentionally not included yet.
‘‘‘

src/mobility_control_tower.egg-info/requires.txt
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower.egg-info/requires.txt
FILE: src/mobility_control_tower.egg-info/requires.txt
‘‘‘text
duckdb<2,>=1.0
fastapi<1,>=0.115
gtfs-realtime-bindings<2,>=1.0
matplotlib<4,>=3.8
pandas<3,>=2.0
PyYAML<7,>=6.0
requests<3,>=2.31
streamlit<2,>=1.40
uvicorn<1,>=0.30

[dev]
pytest<9,>=8
‘‘‘

src/mobility_control_tower.egg-info/SOURCES.txt
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower.egg-info/SOURCES.txt
FILE: src/mobility_control_tower.egg-info/SOURCES.txt
‘‘‘text
README.md
pyproject.toml
src/mobility_control_tower/__init__.py
src/mobility_control_tower/cli.py
src/mobility_control_tower/config.py
src/mobility_control_tower.egg-info/PKG-INFO
src/mobility_control_tower.egg-info/SOURCES.txt
src/mobility_control_tower.egg-info/dependency_links.txt
src/mobility_control_tower.egg-info/entry_points.txt
src/mobility_control_tower.egg-info/requires.txt
src/mobility_control_tower.egg-info/top_level.txt
src/mobility_control_tower/api/__init__.py
src/mobility_control_tower/api/app.py
src/mobility_control_tower/api/db.py
src/mobility_control_tower/api/report.py
src/mobility_control_tower/api/routes.py
src/mobility_control_tower/dashboard/__init__.py
src/mobility_control_tower/dashboard/api_client.py
src/mobility_control_tower/dashboard/app.py
src/mobility_control_tower/ingestion/__init__.py
src/mobility_control_tower/ingestion/gtfs_raw.py
src/mobility_control_tower/metrics/__init__.py
src/mobility_control_tower/metrics/gtfs_kpis.py
src/mobility_control_tower/profiling/__init__.py
src/mobility_control_tower/profiling/gtfs_profile.py
src/mobility_control_tower/quality/__init__.py
src/mobility_control_tower/quality/gtfs_quality.py
src/mobility_control_tower/realtime/__init__.py
src/mobility_control_tower/realtime/gtfs_rt_charts.py
src/mobility_control_tower/realtime/gtfs_rt_compatibility.py
src/mobility_control_tower/realtime/gtfs_rt_kpis.py
src/mobility_control_tower/realtime/gtfs_rt_parser.py
src/mobility_control_tower/realtime/gtfs_rt_raw.py
src/mobility_control_tower/realtime/gtfs_rt_report.py
src/mobility_control_tower/realtime/gtfs_rt_snapshot_report.py
src/mobility_control_tower/reporting/__init__.py
src/mobility_control_tower/reporting/charts.py
src/mobility_control_tower/reporting/demo_report.py
src/mobility_control_tower/reporting/final_report.py

````
src/mobility_control_tower/serving/__init__.py
src/mobility_control_tower/serving/duckdb_loader.py
src/mobility_control_tower/serving/serving_report.py
src/mobility_control_tower/serving/sql_views.py
src/mobility_control_tower/transformations/__init__.py
src/mobility_control_tower/transformations/gtfs_bronze.py
src/mobility_control_tower/transformations/gtfs_silver.py
tests/test_api.py
tests/test_gtfs_kpis.py
tests/test_gtfs_pipeline.py
tests/test_gtfs_profile.py
tests/test_gtfs_realtime.py
tests/test_gtfs_rt_gold.py
tests/test_raw_metadata.py
tests/test_serving_layer.py
```

src/mobility_control_tower.egg-info/top_level.txt
Type: text | Source:
/mnt/d/wsl/projects/Mobility_Control_Tower/src/mobility_control_tower.egg-info/top_level.txt
FILE: src/mobility_control_tower.egg-info/top_level.txt
```text
mobility_control_tower
```

tests/test_api.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_api.py
FILE: tests/test_api.py
```python
from pathlib import Path

import duckdb
import pytest
from fastapi import HTTPException

from mobility_control_tower.api.app import create_app
from mobility_control_tower.api.report import generate_api_report
from mobility_control_tower.api.routes import (
    health,
    metadata,
    realtime_compatibility,
    realtime_feed_health,
    realtime_top_delayed_routes,
    realtime_top_delayed_stops,
    static_hourly_headway,
    static_top_routes,
)


def make_db(path: Path, include_realtime: bool = True) -> Path:
    with duckdb.connect(str(path)) as connection:
        connection.execute(
            """
            CREATE TABLE network_daily_summary AS
            SELECT '2026-01-01' AS service_date, 2 AS active_routes_count, 20 AS
scheduled_trips_count,
                   100 AS scheduled_stop_departures_count, 10 AS active_stops_count
            """
        )
        connection.execute("CREATE VIEW v_network_overview AS SELECT * FROM
network_daily_summary")
        connection.execute(
            """
            CREATE TABLE route_period_summary AS
            SELECT 'R1' AS route_id, 'A' AS route_short_name, 'Airport' AS route_long_name,
'3' AS route_type,
                   2 AS active_service_days, 20 AS total_scheduled_trips, 10.0 AS
average_trips_per_active_day,
                   12 AS max_daily_trips
```

```
            UNION ALL
            SELECT 'R2', 'B', 'Basso', '3', 1, 5, 5.0, 5
            """
        )
        connection.execute("CREATE VIEW v_top_routes_static AS SELECT * FROM
route_period_summary ORDER BY total_scheduled_trips DESC")
        connection.execute(
            """
            CREATE TABLE route_hourly_headway AS
            SELECT '2026-01-01' AS service_date, 'R1' AS route_id, 'A' AS route_short_name,
                   'Airport' AS route_long_name, 8 AS departure_hour, 4 AS
scheduled_departures_count,
                   15.0 AS planned_headway_minutes
            UNION ALL
            SELECT '2026-01-02', 'R2', 'B', 'Basso', 9, 2, 30.0
            """
        )
        connection.execute("CREATE VIEW v_route_hourly_headway AS SELECT * FROM
route_hourly_headway")
        connection.execute(
            """
            CREATE TABLE route_type_daily_summary AS
            SELECT '2026-01-01' AS service_date, '3' AS route_type, 'Bus' AS
route_type_label,
                   2 AS active_routes_count, 20 AS scheduled_trips_count, 100 AS
scheduled_stop_departures_count
            """
        )
        connection.execute("CREATE VIEW v_route_type_daily_summary AS SELECT * FROM
route_type_daily_summary")
        if include_realtime:
            connection.execute(
                """
                CREATE TABLE rt_feed_health_snapshot AS
                SELECT 'trip_updates' AS feed_type, 6 AS feed_age_seconds, 'PASS' AS
freshness_status,
                       1191 AS entity_count
                """
            )
            connection.execute("CREATE VIEW v_rt_feed_health AS SELECT * FROM
rt_feed_health_snapshot")
            connection.execute(
                """
                CREATE TABLE rt_identifier_compatibility_snapshot AS
                SELECT 'route_id' AS identifier_type, 100.0 AS match_percentage, 'PASS' AS
status
                UNION ALL
                SELECT 'trip_id', 88.08, 'WARN'
                """
            )
            connection.execute("CREATE VIEW v_rt_identifier_compatibility AS SELECT * FROM
rt_identifier_compatibility_snapshot")
            connection.execute(
                """
                CREATE TABLE rt_route_delay_snapshot AS
                SELECT 'R1' AS route_id, 'A' AS route_short_name, 'Airport' AS
route_long_name,
                       10 AS stop_time_updates_count, 2 AS distinct_trip_updates_count,
                       120.0 AS avg_delay_seconds, 100.0 AS median_delay_seconds, 300 AS
max_delay_seconds,
                       1 AS delayed_updates_5min_count, 10.0 AS delayed_updates_5min_pct
                """
            )
            connection.execute("CREATE VIEW v_rt_top_delayed_routes_snapshot AS SELECT *
FROM rt_route_delay_snapshot")
            connection.execute(
                """
                CREATE TABLE rt_stop_delay_snapshot AS
                SELECT 'S1' AS stop_id, 'Capitole' AS stop_name, 5 AS
```

```
stop_time_updates_count,
                        90.0 AS avg_delay_seconds
                """
            )
            connection.execute("CREATE VIEW v_rt_top_delayed_stops_snapshot AS SELECT * FROM
rt_stop_delay_snapshot")
    return path


class FakeRequest:
    def __init__(self, app):
        self.app = app


def test_app_factory_rejects_invalid_database(tmp_path: Path) -> None:
    with pytest.raises(FileNotFoundError):
        create_app(tmp_path / "missing.duckdb")


def test_health_metadata_static_endpoints_and_filters(tmp_path: Path) -> None:
    db = make_db(tmp_path / "api.duckdb")
    app = create_app(db)
    request = FakeRequest(app)

    health_response = health(request)
    metadata_response = metadata(request)
    top_routes = static_top_routes(request, limit=1)
    headway = static_hourly_headway(request, route_id="R1", service_date="2026-01-01",
limit=10)

    assert health_response["status"] == "ok"
    assert health_response["database_connected"] is True
    assert "network_daily_summary" in metadata_response["available_tables"]
    assert "v_top_routes_static" in metadata_response["available_views"]
    assert metadata_response["static_data_available"] is True
    assert metadata_response["realtime_snapshot_available"] is True
    assert not any(getattr(route, "path", None) == "/sql" for route in app.routes)
    assert top_routes["count"] == 1
    assert top_routes["data"][0]["route_id"] == "R1"
    assert headway["count"] == 1
    assert headway["data"][0]["route_id"] == "R1"


def test_realtime_endpoints_work_when_views_exist_and_report_is_generated(tmp_path: Path) ->
None:
    db = make_db(tmp_path / "api.duckdb")
    request = FakeRequest(create_app(db))
    report_path = generate_api_report(db, tmp_path / "reports")

    health_response = realtime_feed_health(request)
    compatibility = realtime_compatibility(request)
    delayed_routes = realtime_top_delayed_routes(request, limit=1)
    delayed_stops = realtime_top_delayed_stops(request, limit=1)

    assert health_response["data"][0]["freshness_status"] == "PASS"
    assert compatibility["count"] == 2
    assert delayed_routes["data"][0]["route_id"] == "R1"
    assert delayed_stops["data"][0]["stop_id"] == "S1"
    assert report_path.is_file()
    assert "local read-only API" in report_path.read_text(encoding="utf-8")


def test_realtime_endpoints_return_404_without_realtime_views(tmp_path: Path) -> None:
    db = make_db(tmp_path / "api_static_only.duckdb", include_realtime=False)
    request = FakeRequest(create_app(db))

    metadata_response = metadata(request)

    assert metadata_response["realtime_snapshot_available"] is False
```

```
    with pytest.raises(HTTPException) as excinfo:
        realtime_feed_health(request)
    assert excinfo.value.status_code == 404
    assert excinfo.value.detail == "Realtime snapshot data is not available in this serving
database."
'''

tests/test_final_demo.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_final_demo.py
FILE: tests/test_final_demo.py
'''python
import json
from pathlib import Path

import duckdb
import pytest
import requests

from mobility_control_tower.cli import build_parser
from mobility_control_tower.dashboard.api_client import fetch_dashboard_data, get_json
from mobility_control_tower.reporting.final_report import generate_final_report


class FakeResponse:
    def __init__(self, status_code: int, payload: dict):
        self.status_code = status_code
        self._payload = payload

    def json(self) -> dict:
        return self._payload


def make_serving_run(tmp_path: Path) -> Path:
    serving = tmp_path / "serving" / "tisseo" / "run-1"
    serving.mkdir(parents=True)
    db = serving / "mobility_control_tower.duckdb"
    with duckdb.connect(str(db)) as connection:
        connection.execute(
            """
            CREATE TABLE route_period_summary AS
            SELECT 'A' AS route_short_name, 'Airport' AS route_long_name, 10 AS
total_scheduled_trips
            """
        )
        connection.execute("CREATE VIEW v_top_routes_static AS SELECT * FROM
route_period_summary")
        connection.execute(
            """
            CREATE TABLE rt_feed_health_snapshot AS
            SELECT 'trip_updates' AS feed_type, 6 AS feed_age_seconds, 'PASS' AS
freshness_status, 10 AS entity_count
            """
        )
        connection.execute("CREATE VIEW v_rt_feed_health AS SELECT * FROM
rt_feed_health_snapshot")
        connection.execute(
            """
            CREATE TABLE rt_identifier_compatibility_snapshot AS
            SELECT 'route_id' AS identifier_type, 100.0 AS match_percentage, 'PASS' AS
status
            """
        )
        connection.execute("CREATE VIEW v_rt_identifier_compatibility AS SELECT * FROM
rt_identifier_compatibility_snapshot")
    manifest = {
        "database_path": str(db),
        "tables_loaded": {"route_period_summary": {"row_count": 1},
"rt_feed_health_snapshot": {"row_count": 1}},
    }
```

```
    (serving / "serving_manifest.json").write_text(json.dumps(manifest), encoding="utf-8")
    return serving


def test_dashboard_api_client_success_and_connection_error(monkeypatch: pytest.MonkeyPatch)
-> None:
    def fake_get(url: str, params=None, timeout=5):
        return FakeResponse(200, {"data": [{"value": 1}], "count": 1, "source": "test",
"notes": []})

    monkeypatch.setattr(requests, "get", fake_get)
    payload = get_json("http://api.test", "/health")
    assert payload["ok"] is True
    assert payload["count"] == 1

    def failing_get(url: str, params=None, timeout=5):
        raise requests.ConnectionError("down")

    monkeypatch.setattr(requests, "get", failing_get)
    error = get_json("http://api.test", "/health")
    assert error["ok"] is False
    assert "API is not reachable" in error["error"]


def test_dashboard_fetches_expected_endpoints(monkeypatch: pytest.MonkeyPatch) -> None:
    calls: list[str] = []

    def fake_get(url: str, params=None, timeout=5):
        calls.append(url)
        return FakeResponse(200, {"data": [], "count": 0, "source": url, "notes": []})

    monkeypatch.setattr(requests, "get", fake_get)
    result = fetch_dashboard_data("http://api.test")
    assert "health" in result
    assert "rt_top_delayed_routes" in result
    assert any("/static/top-routes" in call for call in calls)
    assert any("/realtime/feed-health" in call for call in calls)


def test_final_report_generation_and_cli_registration(tmp_path: Path) -> None:
    serving = make_serving_run(tmp_path)
    report = generate_final_report(serving, tmp_path / "reports")
    commands = build_parser()._subparsers._group_actions[0].choices

    assert report.is_file()
    text = report.read_text(encoding="utf-8")
    assert "Final academic MVP status" in text
    assert "Dashboard layer" in text
    assert "generate-final-report" in commands
    assert "serve-dashboard" in commands


def test_final_docs_readme_and_dashboard_are_presentation_ready() -> None:
    required_docs = [
        Path("docs/final_demo_guide.md"),
        Path("docs/teacher_presentation_notes.md"),
        Path("docs/project_summary.md"),
    ]
    for doc in required_docs:
        assert doc.is_file()

    readme = Path("README.md").read_text(encoding="utf-8")
    assert "Architecture Overview" in readme
    assert "Dashboard Usage" in readme
    assert "Academic Positioning" in readme

    dashboard =
Path("src/mobility_control_tower/dashboard/app.py").read_text(encoding="utf-8")
    forbidden_write_actions = ["st.file_uploader", "st.download_button", "requests.post",
```

```
"requests.delete", "requests.put"]
    assert not any(action in dashboard for action in forbidden_write_actions)
'''

tests/test_gtfs_kpis.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_gtfs_kpis.py
FILE: tests/test_gtfs_kpis.py
'''python
import json
from pathlib import Path

import pandas as pd

from mobility_control_tower.metrics.gtfs_kpis import build_gold, compute_gold_tables,
expand_service_dates
from mobility_control_tower.reporting.charts import CHART_FILES, generate_static_charts
from mobility_control_tower.reporting.demo_report import generate_demo_report,
generate_static_mvp_report


def fake_silver_tables() -> dict[str, pd.DataFrame]:
    return {
        "routes": pd.DataFrame(
            [
                {"route_id": "R1", "route_short_name": "A", "route_long_name": "Airport",
"route_type": "3"},
                {"route_id": "R2", "route_short_name": "B", "route_long_name": "Basso",
"route_type": "1"},
            ]
        ),
        "trips": pd.DataFrame(
            [
                {"route_id": "R1", "service_id": "WK", "trip_id": "T1"},
                {"route_id": "R1", "service_id": "WK", "trip_id": "T2"},
                {"route_id": "R1", "service_id": "WK", "trip_id": "T4"},
                {"route_id": "R1", "service_id": "EXTRA", "trip_id": "T5"},
                {"route_id": "R2", "service_id": "SPECIAL", "trip_id": "T3"},
            ]
        ),
        "stop_times": pd.DataFrame(
            [
                {"trip_id": "T1", "arrival_time": "08:00:00", "departure_time": "08:00:00",
"stop_id": "S1", "stop_sequence": "1", "departure_time_seconds": "28800"},
                {"trip_id": "T1", "arrival_time": "08:10:00", "departure_time": "08:10:00",
"stop_id": "S2", "stop_sequence": "2", "departure_time_seconds": "29400"},
                {"trip_id": "T2", "arrival_time": "24:10:00", "departure_time": "24:10:00",
"stop_id": "S1", "stop_sequence": "1", "departure_time_seconds": "87000"},
                {"trip_id": "T2", "arrival_time": "24:20:00", "departure_time": "24:20:00",
"stop_id": "S2", "stop_sequence": "2", "departure_time_seconds": "87600"},
                {"trip_id": "T4", "arrival_time": "08:30:00", "departure_time": "08:30:00",
"stop_id": "S1", "stop_sequence": "1", "departure_time_seconds": "30600"},
                {"trip_id": "T4", "arrival_time": "08:40:00", "departure_time": "08:40:00",
"stop_id": "S2", "stop_sequence": "2", "departure_time_seconds": "31200"},
                {"trip_id": "T5", "arrival_time": "08:45:00", "departure_time": "08:45:00",
"stop_id": "S1", "stop_sequence": "1", "departure_time_seconds": "31500"},
                {"trip_id": "T5", "arrival_time": "08:55:00", "departure_time": "08:55:00",
"stop_id": "S2", "stop_sequence": "2", "departure_time_seconds": "32100"},
                {"trip_id": "T3", "arrival_time": "25:30:00", "departure_time": "25:30:00",
"stop_id": "S2", "stop_sequence": "1", "departure_time_seconds": "91800"},
                {"trip_id": "T3", "arrival_time": "25:40:00", "departure_time": "25:40:00",
"stop_id": "S3", "stop_sequence": "2", "departure_time_seconds": "92400"},
            ]
        ),
        "stops": pd.DataFrame(
            [
                {"stop_id": "S1", "stop_name": "Capitole"},
                {"stop_id": "S2", "stop_name": "Matabiau"},
                {"stop_id": "S3", "stop_name": "Basso Cambo"},
            ]
```

```
        ),
        "calendar": pd.DataFrame(
            [
                {
                    "service_id": "WK",
                    "monday": "1",
                    "tuesday": "1",
                    "wednesday": "0",
                    "thursday": "0",
                    "friday": "0",
                    "saturday": "0",
                    "sunday": "0",
                    "start_date": "20260105",
                    "end_date": "20260106",
                }
            ]
        ),
        "calendar_dates": pd.DataFrame(
            [
                {"service_id": "WK", "date": "20260106", "exception_type": "2"},
                {"service_id": "WK", "date": "20260107", "exception_type": "1"},
                {"service_id": "EXTRA", "date": "20260105", "exception_type": "1"},
                {"service_id": "SPECIAL", "date": "20260108", "exception_type": "1"},
            ]
        ),
    }


def test_service_date_expansion_applies_regular_service_additions_and_removals() -> None:
    tables = fake_silver_tables()
    services = expand_service_dates(tables["calendar"], tables["calendar_dates"])

    assert set(map(tuple, services[["service_id", "service_date"]].to_records(index=False)))
== {
        ("WK", "2026-01-05"),
        ("WK", "2026-01-07"),
        ("EXTRA", "2026-01-05"),
        ("SPECIAL", "2026-01-08"),
    }


def test_service_date_expansion_works_with_calendar_dates_only() -> None:
    services = expand_service_dates(
        None,
        pd.DataFrame([{"service_id": "EXTRA", "date": "20260109", "exception_type": "1"}]),
    )

    assert services.to_dict("records") == [{"service_id": "EXTRA", "service_date":
"2026-01-09"}]


def test_gold_kpis_use_service_day_hours_and_correct_grains() -> None:
    outputs = compute_gold_tables(fake_silver_tables())
    route_daily = outputs["route_daily_trips"]
    hourly = outputs["route_hourly_departures"]
    stop_daily = outputs["stop_daily_departures"]
    network = outputs["network_daily_summary"]

    assert route_daily.to_dict("records") == [
        {"service_date": "2026-01-05", "route_id": "R1", "route_short_name": "A",
"route_long_name": "Airport", "route_type": "3", "scheduled_trips_count": 4},
        {"service_date": "2026-01-07", "route_id": "R1", "route_short_name": "A",
"route_long_name": "Airport", "route_type": "3", "scheduled_trips_count": 3},
        {"service_date": "2026-01-08", "route_id": "R2", "route_short_name": "B",
"route_long_name": "Basso", "route_type": "1", "scheduled_trips_count": 1},
    ]
    assert set(hourly["departure_hour"]) == {8, 24, 25}
    assert hourly["scheduled_departures_count"].sum() == 8
    assert stop_daily["scheduled_departures_count"].sum() == 16
```

```
    assert network.to_dict("records") == [
        {"service_date": "2026-01-05", "active_routes_count": 1, "scheduled_trips_count": 4,
"scheduled_stop_departures_count": 8, "active_stops_count": 2},
        {"service_date": "2026-01-07", "active_routes_count": 1, "scheduled_trips_count": 3,
"scheduled_stop_departures_count": 6, "active_stops_count": 2},
        {"service_date": "2026-01-08", "active_routes_count": 1, "scheduled_trips_count": 1,
"scheduled_stop_departures_count": 2, "active_stops_count": 2},
    ]


def test_richer_gold_kpis_period_headway_route_type_and_rankings() -> None:
    outputs = compute_gold_tables(fake_silver_tables())
    route_period = outputs["route_period_summary"]
    headway = outputs["route_hourly_headway"]
    route_type = outputs["route_type_daily_summary"]
    busiest_route = outputs["busiest_route_day"]
    busiest_stop = outputs["busiest_stop_day"]

    r1 = route_period.loc[route_period["route_id"] == "R1"].iloc[0]
    assert r1["active_service_days"] == 2
    assert r1["total_scheduled_trips"] == 7
    assert r1["average_trips_per_active_day"] == 3.5
    assert r1["max_daily_trips"] == 4

    r1_hour_8 = headway.query("service_date == '2026-01-05' and route_id == 'R1' and
departure_hour == 8").iloc[0]
    assert r1_hour_8["scheduled_departures_count"] == 3
    assert r1_hour_8["planned_headway_minutes"] == 20.0

    assert set(route_type["route_type_label"]) == {"Bus", "Subway/Metro"}
    assert busiest_route.iloc[0]["rank"] == 1
    assert busiest_route.iloc[0]["scheduled_trips_count"] == 4
    assert busiest_stop.iloc[0]["rank"] == 1
    assert busiest_stop.iloc[0]["scheduled_departures_count"] == 4


def test_gold_builder_writes_outputs_manifest_and_demo_report(tmp_path: Path) -> None:
    silver_run = tmp_path / "silver" / "tisseo" / "run-1"
    silver_run.mkdir(parents=True)
    for name, frame in fake_silver_tables().items():
        frame.to_csv(silver_run / f"{name}.csv", index=False)

    gold_run = build_gold(silver_run, tmp_path / "gold")
    manifest = json.loads((gold_run / "gold_manifest.json").read_text(encoding="utf-8"))

    assert (gold_run / "route_daily_trips.csv").is_file()
    assert (gold_run / "route_hourly_departures.csv").is_file()
    assert (gold_run / "stop_daily_departures.csv").is_file()
    assert (gold_run / "network_daily_summary.csv").is_file()
    assert (gold_run / "route_period_summary.csv").is_file()
    assert (gold_run / "route_hourly_headway.csv").is_file()
    assert (gold_run / "route_type_daily_summary.csv").is_file()
    assert (gold_run / "busiest_route_day.csv").is_file()
    assert (gold_run / "busiest_stop_day.csv").is_file()
    assert manifest["tables_created"]["route_daily_trips"]["row_count"] == 3
    assert manifest["kpi_definitions"]["route_period_summary"]["static_planning_only"] is
True
    assert "route_hourly_departures" in manifest["kpi_definitions"]

    reports_dir = tmp_path / "reports"
    quality_stub = {"overall_status": "PASS", "checks": [{"status": "PASS"}, {"status":
"WARN"}, {"status": "FAIL"}]}
    reports_dir.mkdir()
    (reports_dir / "gtfs_quality_run-1.json").write_text(json.dumps(quality_stub),
encoding="utf-8")
    figures_dir = generate_static_charts(gold_run, reports_dir)
    report_path = generate_demo_report(gold_run, reports_dir)
    evidence_path = generate_static_mvp_report(gold_run, reports_dir)
    report = report_path.read_text(encoding="utf-8")
```

```
    evidence = evidence_path.read_text(encoding="utf-8")

    for filename in CHART_FILES:
        assert (figures_dir / filename).is_file()
    assert report_path.name == "mobility_demo_run-1.md"
    assert "# Mobility Control Tower" in report
    assert "static planning KPIs" in report
    assert "over the GTFS service period" in report
    assert "1 PASS, 1 WARN, 1 FAIL" in report
    assert evidence_path.name == "static_mvp_evidence_run-1.md"
    assert "Why this is Data Engineering" in evidence
'''

tests/test_gtfs_pipeline.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_gtfs_pipeline.py
FILE: tests/test_gtfs_pipeline.py
'''python
import csv
import hashlib
import json
import zipfile
from pathlib import Path

from mobility_control_tower.quality.gtfs_quality import validate_silver_run
from mobility_control_tower.transformations.gtfs_bronze import build_bronze
from mobility_control_tower.transformations.gtfs_silver import build_silver,
parse_gtfs_date, parse_gtfs_time


def fake_gtfs_files() -> dict[str, str]:
    return {
        "agency.txt":
"agency_id,agency_name,agency_url,agency_timezone\nA1,Tisseo,https://example.test,Europe/Par
is\n",
        "stops.txt":
"stop_id,stop_name,stop_lat,stop_lon\nS1,Capitole,43.6045,1.4440\nS2,Invalid,95,200\nS2,Dupl
icate,43.6,1.4\n",
        "routes.txt": "route_id,route_short_name,route_type\nR1,A,1\n",
        "trips.txt": "route_id,service_id,trip_id\nR1,WK,T1\nUNKNOWN,WK,T2\n",
        "stop_times.txt":
"trip_id,arrival_time,departure_time,stop_id,stop_sequence\nT1,24:10:00,25:30:00,S1,1\nUNKNO
WN,invalid,08:00:00,UNKNOWN,2\n",
        "calendar.txt": "service_id,start_date,end_date\nWK,20260101,20260131\n",
    }


def make_raw_run(tmp_path: Path) -> tuple[Path, dict[str, str]]:
    raw_run = tmp_path / "raw" / "tisseo" / "2026-01-02_030405"
    raw_run.mkdir(parents=True)
    files = fake_gtfs_files()
    archive_path = raw_run / "Tisseo_GTFS.zip"
    with zipfile.ZipFile(archive_path, "w") as archive:
        for filename, content in files.items():
            archive.writestr(filename, content)
    metadata = {"sha256": hashlib.sha256(archive_path.read_bytes()).hexdigest()}
    (raw_run / "metadata.json").write_text(json.dumps(metadata), encoding="utf-8")
    return raw_run, files


def read_csv(path: Path) -> list[dict[str, str]]:
    with path.open(encoding="utf-8", newline="") as handle:
        return list(csv.DictReader(handle))


def check(report: dict, name: str) -> dict:
    return next(item for item in report["checks"] if item["check_name"] == name)


def test_bronze_extracts_original_content_and_manifest(tmp_path: Path) -> None:
```

```
    raw_run, files = make_raw_run(tmp_path)
    bronze_run = build_bronze(raw_run, tmp_path / "bronze")
    manifest = json.loads((bronze_run / "bronze_manifest.json").read_text(encoding="utf-8"))

    assert (bronze_run / "stops.txt").read_text(encoding="utf-8") == files["stops.txt"]
    assert manifest["source_zip_sha256"]
    assert manifest["extracted_files"]["stops.txt"]["row_count"] == 3
    assert manifest["extracted_files"]["routes.txt"]["columns"] == ["route_id",
"route_short_name", "route_type"]
    assert "calendar_dates.txt" in manifest["missing_important_files"]


def test_silver_tables_and_after_midnight_times(tmp_path: Path) -> None:
    raw_run, _ = make_raw_run(tmp_path)
    bronze_run = build_bronze(raw_run, tmp_path / "bronze")
    silver_run = build_silver(bronze_run, tmp_path / "silver")
    stop_times = read_csv(silver_run / "stop_times.csv")
    manifest = json.loads((silver_run / "silver_manifest.json").read_text(encoding="utf-8"))

    assert parse_gtfs_time("08:15:00") == 29_700
    assert parse_gtfs_time("24:10:00") == 87_000
    assert parse_gtfs_time("25:30:00") == 91_800
    assert parse_gtfs_time("invalid") is None
    assert parse_gtfs_time("") is None
    assert parse_gtfs_date("20260131") == "2026-01-31"
    assert parse_gtfs_date("20260231") is None
    assert stop_times[0]["arrival_time"] == "24:10:00"
    assert stop_times[0]["arrival_time_seconds"] == "87000"
    assert manifest["tables_created"]["trips"]["row_count"] == 2
    assert read_csv(silver_run / "calendar.csv")[0]["start_date_iso"] == "2026-01-01"


def test_quality_detects_duplicates_coordinates_and_unknown_references(tmp_path: Path) ->
None:
    raw_run, _ = make_raw_run(tmp_path)
    bronze_run = build_bronze(raw_run, tmp_path / "bronze")
    silver_run = build_silver(bronze_run, tmp_path / "silver")
    json_path, markdown_path = validate_silver_run(silver_run, tmp_path / "reports")
    report = json.loads(json_path.read_text(encoding="utf-8"))

    assert check(report, "duplicate_stop_id")["problem_count"] == 1
    assert check(report, "invalid_stop_lat")["problem_count"] == 1
    assert check(report, "invalid_stop_lon")["problem_count"] == 1
    assert check(report, "stop_times_unknown_trip_id")["problem_count"] == 1
    assert check(report, "stop_times_unknown_stop_id")["problem_count"] == 1
    assert check(report, "trips_unknown_route_id")["problem_count"] == 1
    assert check(report, "invalid_arrival_time_format")["problem_count"] == 1
    assert report["overall_status"] == "FAIL"
    assert markdown_path.is_file()


def test_quality_detects_missing_required_column(tmp_path: Path) -> None:
    silver_run = tmp_path / "silver" / "tisseo" / "run-1"
    silver_run.mkdir(parents=True)
    (silver_run / "stops.csv").write_text("stop_id,stop_name,stop_lat\nS1,Capitole,43.6\n",
encoding="utf-8")
    json_path, _ = validate_silver_run(silver_run, tmp_path / "reports")
    report = json.loads(json_path.read_text(encoding="utf-8"))

    required = check(report, "stops_required_columns")
    assert required["status"] == "FAIL"
    assert required["problem_count"] == 1
    assert "stop_lon" in required["explanation"]
'''

tests/test_gtfs_profile.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_gtfs_profile.py
FILE: tests/test_gtfs_profile.py
'''python
```

```
import json
import zipfile
from pathlib import Path

from mobility_control_tower.profiling.gtfs_profile import build_profile, profile_raw_run


def make_gtfs(path: Path, include_calendar: bool = True) -> None:
    contents = {
        "agency.txt": "agency_id,agency_name\n1,Tisseo\n",
        "stops.txt": "stop_id,stop_name\nS1,Capitole\nS2,Matabiau\n",
        "routes.txt": "route_id,route_short_name\nR1,A\n",
        "trips.txt": "route_id,service_id,trip_id\nR1,WK,T1\nR1,WK,T2\n",
        "stop_times.txt":
"trip_id,arrival_time,departure_time,stop_id,stop_sequence\nT1,08:00:00,08:00:00,S1,1\nT1,08
:05:00,08:05:00,S2,2\n",
    }
    if include_calendar:
        contents["calendar.txt"] =
"service_id,monday,start_date,end_date\nWK,1,20260101,20260131\n"
    with zipfile.ZipFile(path, "w") as archive:
        for filename, content in contents.items():
            archive.writestr(filename, content)


def test_profile_detects_files_counts_columns_and_dates(tmp_path: Path) -> None:
    gtfs_zip = tmp_path / "fake.zip"
    make_gtfs(gtfs_zip)
    profile = build_profile(gtfs_zip, "run-1")
    assert profile["important_files_present"]["routes.txt"] is True
    assert profile["files"]["stops.txt"]["row_count"] == 2
    assert profile["files"]["stops.txt"]["columns"] == ["stop_id", "stop_name"]
    assert profile["number_of_routes"] == 1
    assert profile["number_of_trips"] == 2
    assert profile["service_date_range"] == {"start_date": "20260101", "end_date":
"20260131"}


def test_profile_handles_missing_files_and_writes_reports(tmp_path: Path) -> None:
    run_dir = tmp_path / "raw" / "tisseo" / "2026-01-02_030405"
    run_dir.mkdir(parents=True)
    make_gtfs(run_dir / "Tisseo_GTFS.zip", include_calendar=False)
    json_path, markdown_path = profile_raw_run(run_dir, tmp_path / "reports")
    profile = json.loads(json_path.read_text(encoding="utf-8"))
    assert set(profile["missing_important_files"]) == {"calendar.txt", "calendar_dates.txt"}
    assert profile["important_files_present"]["calendar_dates.txt"] is False
    assert profile["service_date_range"] == {"start_date": None, "end_date": None}
    assert markdown_path.is_file()
'''

tests/test_gtfs_realtime.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_gtfs_realtime.py
FILE: tests/test_gtfs_realtime.py
'''python
import json
from pathlib import Path

import pandas as pd
from google.transit import gtfs_realtime_pb2

from mobility_control_tower.realtime.gtfs_rt_compatibility import
check_realtime_compatibility
from mobility_control_tower.realtime.gtfs_rt_parser import parse_realtime_snapshot
from mobility_control_tower.realtime.gtfs_rt_raw import preserve_realtime_snapshot
from mobility_control_tower.realtime.gtfs_rt_report import generate_realtime_report


SOURCE = {"name": "Tisseo", "source_page_url": "https://example.test"}
```

```
def trip_updates_feed() -> bytes:
    feed = gtfs_realtime_pb2.FeedMessage()
    feed.header.gtfs_realtime_version = "2.0"
    feed.header.timestamp = 1_700_000_000
    entity = feed.entity.add()
    entity.id = "tu-1"
    update = entity.trip_update
    update.trip.trip_id = "T1"
    update.trip.route_id = "R1"
    update.trip.start_date = "20260105"
    update.trip.start_time = "08:00:00"
    update.timestamp = 1_700_000_010
    stop_update = update.stop_time_update.add()
    stop_update.stop_sequence = 1
    stop_update.stop_id = "S1"
    stop_update.arrival.time = 1_700_000_100
    stop_update.arrival.delay = 60
    stop_update.departure.time = 1_700_000_120
    stop_update.departure.delay = 30
    return feed.SerializeToString()


def vehicle_positions_feed() -> bytes:
    feed = gtfs_realtime_pb2.FeedMessage()
    feed.header.gtfs_realtime_version = "2.0"
    feed.header.timestamp = 1_700_000_000
    entity = feed.entity.add()
    entity.id = "veh-1"
    vehicle = entity.vehicle
    vehicle.trip.trip_id = "T1"
    vehicle.trip.route_id = "R1"
    vehicle.vehicle.id = "V1"
    vehicle.vehicle.label = "Bus 1"
    vehicle.position.latitude = 43.6
    vehicle.position.longitude = 1.44
    vehicle.position.bearing = 90
    vehicle.position.speed = 12.5
    vehicle.current_stop_sequence = 1
    vehicle.stop_id = "S1"
    vehicle.current_status = gtfs_realtime_pb2.VehiclePosition.IN_TRANSIT_TO
    vehicle.timestamp = 1_700_000_030
    return feed.SerializeToString()


def service_alerts_feed() -> bytes:
    feed = gtfs_realtime_pb2.FeedMessage()
    feed.header.gtfs_realtime_version = "2.0"
    feed.header.timestamp = 1_700_000_000
    entity = feed.entity.add()
    entity.id = "alert-1"
    alert = entity.alert
    alert.cause = gtfs_realtime_pb2.Alert.CONSTRUCTION
    alert.effect = gtfs_realtime_pb2.Alert.DETOUR
    period = alert.active_period.add()
    period.start = 1_700_000_000
    period.end = 1_700_003_600
    alert.header_text.translation.add(text="Works")
    alert.description_text.translation.add(text="Temporary detour")
    informed = alert.informed_entity.add()
    informed.route_id = "R1"
    informed.route_type = 3
    informed.stop_id = "S1"
    return feed.SerializeToString()


def read_csv(path: Path) -> list[dict[str, str]]:
    return pd.read_csv(path, dtype=str, keep_default_na=False).to_dict("records")
```

```
def make_raw_run(tmp_path: Path, feed_type: str, content: bytes) -> Path:
    return preserve_realtime_snapshot(
        content,
        "tisseo",
        SOURCE,
        feed_type,
        "https://example.test/feed.pb",
        tmp_path / "raw_realtime",
        http_status=200,
        content_type="application/x-protobuf",
    )


def test_raw_realtime_metadata_is_created(tmp_path: Path) -> None:
    raw_run = make_raw_run(tmp_path, "trip_updates", trip_updates_feed())
    metadata = json.loads((raw_run / "metadata.json").read_text(encoding="utf-8"))

    assert (raw_run / "feed.pb").is_file()
    assert metadata["feed_type"] == "trip_updates"
    assert metadata["sha256"]
    assert metadata["file_size_bytes"] > 0
    assert metadata["http_status"] == 200


def test_parse_trip_updates_and_stop_time_updates(tmp_path: Path) -> None:
    raw_run = make_raw_run(tmp_path, "trip_updates", trip_updates_feed())
    rt_run = parse_realtime_snapshot(raw_run, tmp_path / "realtime")

    summary = read_csv(rt_run / "rt_feed_summary.csv")[0]
    trips = read_csv(rt_run / "rt_trip_updates.csv")
    stops = read_csv(rt_run / "rt_stop_time_updates.csv")

    assert summary["feed_type"] == "trip_updates"
    assert summary["entity_count"] == "1"
    assert trips[0]["trip_id"] == "T1"
    assert trips[0]["route_id"] == "R1"
    assert stops[0]["stop_id"] == "S1"
    assert stops[0]["arrival_delay"] == "60"


def test_parse_vehicle_positions(tmp_path: Path) -> None:
    raw_run = make_raw_run(tmp_path, "vehicle_positions", vehicle_positions_feed())
    rt_run = parse_realtime_snapshot(raw_run, tmp_path / "realtime")
    vehicles = read_csv(rt_run / "rt_vehicle_positions.csv")

    assert vehicles[0]["vehicle_id"] == "V1"
    assert vehicles[0]["trip_id"] == "T1"
    assert vehicles[0]["stop_id"] == "S1"
    assert vehicles[0]["current_status"] == "IN_TRANSIT_TO"


def test_parse_service_alerts(tmp_path: Path) -> None:
    raw_run = make_raw_run(tmp_path, "service_alerts", service_alerts_feed())
    rt_run = parse_realtime_snapshot(raw_run, tmp_path / "realtime")
    alerts = read_csv(rt_run / "rt_alerts.csv")
    entities = read_csv(rt_run / "rt_alert_informed_entities.csv")

    assert alerts[0]["cause"] == "CONSTRUCTION"
    assert alerts[0]["effect"] == "DETOUR"
    assert alerts[0]["header_text"] == "Works"
    assert entities[0]["route_id"] == "R1"
    assert entities[0]["stop_id"] == "S1"


def test_realtime_report_and_static_compatibility(tmp_path: Path) -> None:
    raw_run = make_raw_run(tmp_path, "trip_updates", trip_updates_feed())
    rt_run = parse_realtime_snapshot(raw_run, tmp_path / "realtime")
    report_path = generate_realtime_report(rt_run, tmp_path / "reports")
```

```
    silver_run = tmp_path / "silver" / "tisseo" / "static-1"
    silver_run.mkdir(parents=True)
    (silver_run / "routes.csv").write_text("route_id\nR1\n", encoding="utf-8")
    (silver_run / "trips.csv").write_text("trip_id\nT1\n", encoding="utf-8")
    (silver_run / "stops.csv").write_text("stop_id\nS1\n", encoding="utf-8")
    json_path, markdown_path = check_realtime_compatibility(silver_run, rt_run, tmp_path /
"reports")
    compatibility = json.loads(json_path.read_text(encoding="utf-8"))

    assert report_path.is_file()
    assert "snapshot" in report_path.read_text(encoding="utf-8")
    assert compatibility["overall_status"] == "PASS"
    assert markdown_path.is_file()


def test_compatibility_reports_unmatched_ids(tmp_path: Path) -> None:
    raw_run = make_raw_run(tmp_path, "trip_updates", trip_updates_feed())
    rt_run = parse_realtime_snapshot(raw_run, tmp_path / "realtime")
    silver_run = tmp_path / "silver" / "tisseo" / "static-1"
    silver_run.mkdir(parents=True)
    (silver_run / "routes.csv").write_text("route_id\nOTHER\n", encoding="utf-8")
    (silver_run / "trips.csv").write_text("trip_id\nOTHER\n", encoding="utf-8")
    (silver_run / "stops.csv").write_text("stop_id\nOTHER\n", encoding="utf-8")
    json_path, _ = check_realtime_compatibility(silver_run, rt_run, tmp_path / "reports")
    compatibility = json.loads(json_path.read_text(encoding="utf-8"))

    assert compatibility["overall_status"] == "WARN"
    route_check = next(check for check in compatibility["checks"] if check["check_name"] ==
"route_id_matches_static_routes")
    assert route_check["unmatched_count"] == 1
    assert route_check["status"] == "WARN"
'''

tests/test_gtfs_rt_gold.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_gtfs_rt_gold.py
FILE: tests/test_gtfs_rt_gold.py
'''python
import json
from pathlib import Path

import pandas as pd

from mobility_control_tower.realtime.gtfs_rt_charts import RT_CHART_FILES,
generate_rt_charts
from mobility_control_tower.realtime.gtfs_rt_kpis import (
    build_rt_gold,
    compatibility_status,
    compute_rt_gold_tables,
    freshness_status,
)
from mobility_control_tower.realtime.gtfs_rt_snapshot_report import
generate_rt_snapshot_report


def static_tables() -> dict[str, pd.DataFrame]:
    return {
        "routes": pd.DataFrame(
            [
                {"route_id": "R1", "route_short_name": "A", "route_long_name": "Airport"},
                {"route_id": "R2", "route_short_name": "B", "route_long_name": "Basso"},
            ]
        ),
        "trips": pd.DataFrame([{"trip_id": "T1"}, {"trip_id": "T2"}]),
        "stops": pd.DataFrame([{"stop_id": "S1", "stop_name": "Capitole"}, {"stop_id": "S2",
"stop_name": "Matabiau"}]),
    }
```

```
def rt_tables() -> dict[str, pd.DataFrame]:
    return {
        "rt_feed_summary": pd.DataFrame(
            [
                {
                    "feed_type": "trip_updates",
                    "fetched_at": "2026-01-01T00:01:00+00:00",
                    "header_timestamp": "1767225600",
                    "feed_age_seconds": "60",
                    "entity_count": "4",
                    "parsed_entity_count": "4",
                    "skipped_entity_count": "0",
                }
            ]
        ),
        "rt_trip_updates": pd.DataFrame(
            [
                {"entity_id": "E1", "trip_id": "T1", "route_id": "R1", "start_date":
"20260101", "start_time": "08:00:00", "schedule_relationship": "SCHEDULED",
"trip_update_timestamp": "1", "stop_time_update_count": "2"},
                {"entity_id": "E2", "trip_id": "T2", "route_id": "R2", "start_date":
"20260101", "start_time": "09:00:00", "schedule_relationship": "SCHEDULED",
"trip_update_timestamp": "1", "stop_time_update_count": "2"},
                {"entity_id": "E3", "trip_id": "T3", "route_id": "R3", "start_date":
"20260101", "start_time": "10:00:00", "schedule_relationship": "SCHEDULED",
"trip_update_timestamp": "1", "stop_time_update_count": "2"},
                {"entity_id": "E4", "trip_id": "", "route_id": "R1", "start_date":
"20260101", "start_time": "11:00:00", "schedule_relationship": "SCHEDULED",
"trip_update_timestamp": "1", "stop_time_update_count": "1"},
            ]
        ),
        "rt_stop_time_updates": pd.DataFrame(
            [
                {"entity_id": "E1", "trip_id": "T1", "route_id": "R1", "stop_id": "S1",
"arrival_delay": "100", "departure_delay": "999"},
                {"entity_id": "E1", "trip_id": "T1", "route_id": "R1", "stop_id": "S2",
"arrival_delay": "", "departure_delay": "400"},
                {"entity_id": "E2", "trip_id": "T2", "route_id": "R2", "stop_id": "S1",
"arrival_delay": "-30", "departure_delay": ""},
                {"entity_id": "E2", "trip_id": "T2", "route_id": "R2", "stop_id": "S2",
"arrival_delay": "", "departure_delay": ""},
                {"entity_id": "E3", "trip_id": "T3", "route_id": "R3", "stop_id": "S3",
"arrival_delay": "600", "departure_delay": ""},
            ]
        ),
    }


def write_tables(root: Path, tables: dict[str, pd.DataFrame]) -> None:
    root.mkdir(parents=True)
    for name, frame in tables.items():
        frame.to_csv(root / f"{name}.csv", index=False)


def test_freshness_and_compatibility_status_thresholds() -> None:
    assert freshness_status("90") == "PASS"
    assert freshness_status("120") == "WARN"
    assert freshness_status("301") == "FAIL"
    assert freshness_status("") == "UNKNOWN"
    assert compatibility_status(95) == "PASS"
    assert compatibility_status(50) == "WARN"
    assert compatibility_status(49.9) == "FAIL"
    assert compatibility_status(None) == "NOT_APPLICABLE"


def test_rt_gold_delay_aggregation_compatibility_and_enrichment() -> None:
    outputs = compute_rt_gold_tables(static_tables(), rt_tables())
    health = outputs["rt_feed_health_snapshot"].iloc[0]
    enriched = outputs["rt_trip_update_enriched"]
```

```
    route_delay = outputs["rt_route_delay_snapshot"]
    stop_delay = outputs["rt_stop_delay_snapshot"]
    compatibility = outputs["rt_identifier_compatibility_snapshot"]

    assert health["freshness_status"] == "PASS"
    assert bool(health["has_delay_information"]) is True

    unmatched = enriched.loc[enriched["trip_id"] == "T3"].iloc[0]
    missing = enriched.loc[enriched["trip_id"] == ""].iloc[0]
    assert bool(unmatched["trip_id_static_match"]) is False
    assert "not found" in unmatched["compatibility_note"]
    assert "missing" in missing["compatibility_note"]

    r1 = route_delay.loc[route_delay["route_id"] == "R1"].iloc[0]
    assert r1["avg_delay_seconds"] == 250.0
    assert r1["delayed_updates_5min_count"] == 1
    assert r1["early_updates_count"] == 0

    r2 = route_delay.loc[route_delay["route_id"] == "R2"].iloc[0]
    assert r2["early_updates_count"] == 1
    assert r2["no_delay_info_count"] == 1

    s1 = stop_delay.loc[stop_delay["stop_id"] == "S1"].iloc[0]
    assert s1["avg_delay_seconds"] == 35.0
    assert bool(s1["stop_id_static_match"]) is True

    trip_compat = compatibility.loc[compatibility["identifier_type"] == "trip_id"].iloc[0]
    stop_compat = compatibility.loc[compatibility["identifier_type"] == "stop_id"].iloc[0]
    assert trip_compat["status"] == "WARN"
    assert stop_compat["status"] == "WARN"
    assert len(str(trip_compat["sample_unmatched_values"]).split("|")) <= 10


def test_identifier_compatibility_fail_and_not_applicable() -> None:
    outputs = compute_rt_gold_tables(
        {"routes": pd.DataFrame([{"route_id": "OTHER"}]), "trips": pd.DataFrame([{"trip_id":
"OTHER"}]), "stops": pd.DataFrame([{"stop_id": "OTHER"}])},
        rt_tables(),
    )
    compatibility = outputs["rt_identifier_compatibility_snapshot"]
    assert set(compatibility["status"]) == {"FAIL"}

    empty_outputs = compute_rt_gold_tables(static_tables(), {"rt_feed_summary":
rt_tables()["rt_feed_summary"], "rt_trip_updates": pd.DataFrame(), "rt_stop_time_updates":
pd.DataFrame()})
    assert set(empty_outputs["rt_identifier_compatibility_snapshot"]["status"]) ==
{"NOT_APPLICABLE"}


def test_build_rt_gold_charts_and_report(tmp_path: Path) -> None:
    silver_run = tmp_path / "silver" / "tisseo" / "static-1"
    rt_run = tmp_path / "realtime" / "tisseo" / "trip_updates" / "rt-1"
    write_tables(silver_run, static_tables())
    write_tables(rt_run, rt_tables())
    (rt_run / "realtime_manifest.json").write_text(json.dumps({"feed_type":
"trip_updates"}), encoding="utf-8")

    rt_gold_run = build_rt_gold(silver_run, rt_run, tmp_path / "realtime_gold")
    manifest = json.loads((rt_gold_run /
"rt_gold_manifest.json").read_text(encoding="utf-8"))
    figures_dir = generate_rt_charts(rt_gold_run, tmp_path / "reports")
    report_path = generate_rt_snapshot_report(rt_gold_run, tmp_path / "reports")
    report = report_path.read_text(encoding="utf-8")

    assert (rt_gold_run / "rt_feed_health_snapshot.csv").is_file()
    assert (rt_gold_run / "rt_trip_update_enriched.csv").is_file()
    assert (rt_gold_run / "rt_route_delay_snapshot.csv").is_file()
    assert (rt_gold_run / "rt_stop_delay_snapshot.csv").is_file()
    assert (rt_gold_run / "rt_identifier_compatibility_snapshot.csv").is_file()
```

```
    assert manifest["compatibility_summary"]["trip_id"] == "WARN"
    for filename in RT_CHART_FILES:
        assert (figures_dir / filename).is_file()
    assert "GTFS-Realtime snapshot" in report
    assert "continuous real-time monitoring system" in report
    assert "streaming" in report
'''

tests/test_raw_metadata.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_raw_metadata.py
FILE: tests/test_raw_metadata.py
'''python
import hashlib
import json
import zipfile
from datetime import datetime, timezone
from pathlib import Path

from mobility_control_tower.ingestion.gtfs_raw import preserve_gtfs_zip


SOURCE = {
    "name": "Test TissØo",
    "source_page_url": "https://example.test/dataset",
    "licence": "ODbL",
    "output_filename": "Tisseo_GTFS.zip",
}


def test_metadata_and_checksum_are_created(tmp_path: Path) -> None:
    input_zip = tmp_path / "input.zip"
    with zipfile.ZipFile(input_zip, "w") as archive:
        archive.writestr("agency.txt", "agency_id,agency_name\n1,Tisseo\n")
    original = input_zip.read_bytes()

    run_dir = preserve_gtfs_zip(
        input_zip,
        "tisseo",
        SOURCE,
        tmp_path / "raw",
        ingested_at=datetime(2026, 1, 2, 3, 4, 5, tzinfo=timezone.utc),
    )

    metadata_path = run_dir / "metadata.json"
    metadata = json.loads(metadata_path.read_text(encoding="utf-8"))
    copied = run_dir / "Tisseo_GTFS.zip"
    assert metadata_path.is_file()
    assert copied.read_bytes() == original
    assert metadata["sha256"] == hashlib.sha256(original).hexdigest()
    assert metadata["file_size_bytes"] == len(original)
    assert metadata["source_name"] == "Test TissØo"
    assert metadata["ingestion_timestamp"] == "2026-01-02T03:04:05+00:00"
'''

tests/test_serving_layer.py
Type: text | Source: /mnt/d/wsl/projects/Mobility_Control_Tower/tests/test_serving_layer.py
FILE: tests/test_serving_layer.py
'''python
import json
from pathlib import Path

import duckdb
import pandas as pd

from mobility_control_tower.serving.duckdb_loader import build_serving_database,
query_serving_database
from mobility_control_tower.serving.serving_report import generate_serving_report
```

```
def write_csv(path: Path, rows: list[dict]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    pd.DataFrame(rows).to_csv(path, index=False)


def make_gold_run(tmp_path: Path) -> Path:
    gold = tmp_path / "gold" / "tisseo" / "static-1"
    write_csv(
        gold / "route_daily_trips.csv",
        [{"service_date": "2026-01-01", "route_id": "R1", "route_short_name": "A",
"route_long_name": "Airport", "route_type": "3", "scheduled_trips_count": 10}],
    )
    write_csv(
        gold / "network_daily_summary.csv",
        [{"service_date": "2026-01-01", "active_routes_count": 1, "scheduled_trips_count":
10, "scheduled_stop_departures_count": 25, "active_stops_count": 2}],
    )
    write_csv(
        gold / "route_period_summary.csv",
        [{"route_id": "R1", "route_short_name": "A", "route_long_name": "Airport",
"route_type": "3", "active_service_days": 1, "total_scheduled_trips": 10,
"average_trips_per_active_day": 10, "max_daily_trips": 10, "first_service_date":
"2026-01-01", "last_service_date": "2026-01-01"}],
    )
    write_csv(
        gold / "route_hourly_headway.csv",
        [{"service_date": "2026-01-01", "route_id": "R1", "route_short_name": "A",
"route_long_name": "Airport", "departure_hour": 8, "scheduled_departures_count": 4,
"planned_headway_minutes": 15}],
    )
    write_csv(
        gold / "route_type_daily_summary.csv",
        [{"service_date": "2026-01-01", "route_type": "3", "route_type_label": "Bus",
"active_routes_count": 1, "scheduled_trips_count": 10, "scheduled_stop_departures_count":
25}],
    )
    return gold


def make_rt_gold_run(tmp_path: Path, include_optional_delay: bool = True) -> Path:
    rt = tmp_path / "realtime_gold" / "tisseo" / "trip_updates" / "rt-1"
    write_csv(
        rt / "rt_feed_health_snapshot.csv",
        [{"feed_type": "trip_updates", "fetched_at": "2026-01-01T00:00:00+00:00",
"feed_age_seconds": 30, "freshness_status": "PASS", "entity_count": 5}],
    )
    write_csv(
        rt / "rt_identifier_compatibility_snapshot.csv",
        [{"identifier_type": "route_id", "rt_distinct_count": 1, "matched_static_count": 1,
"unmatched_static_count": 0, "match_percentage": 100, "status": "PASS",
"sample_unmatched_values": ""}],
    )
    if include_optional_delay:
        write_csv(
            rt / "rt_route_delay_snapshot.csv",
            [{"route_id": "R1", "route_short_name": "A", "route_long_name": "Airport",
"stop_time_updates_count": 5, "distinct_trip_updates_count": 2, "avg_delay_seconds": 120,
"median_delay_seconds": 100, "max_delay_seconds": 300, "delayed_updates_5min_count": 1,
"delayed_updates_5min_pct": 20}],
        )
        write_csv(
            rt / "rt_stop_delay_snapshot.csv",
            [{"stop_id": "S1", "stop_name": "Capitole", "stop_time_updates_count": 3,
"avg_delay_seconds": 90}],
        )
    return rt


def table_names(db_path: Path) -> set[str]:
```

```
    with duckdb.connect(str(db_path), read_only=True) as connection:
        return {row[0] for row in connection.execute("SHOW TABLES").fetchall()}


def
test_build_serving_database_static_only_creates_tables_views_manifest_and_report(tmp_path:
Path) -> None:
    gold = make_gold_run(tmp_path)
    serving_run = build_serving_database(gold, serving_root=tmp_path / "serving")
    db_path = serving_run / "mobility_control_tower.duckdb"
    manifest = json.loads((serving_run /
"serving_manifest.json").read_text(encoding="utf-8"))
    names = table_names(db_path)
    result = query_serving_database(db_path, "top-routes", limit=5)
    report_path = generate_serving_report(serving_run, tmp_path / "reports")

    assert db_path.is_file()
    assert "route_daily_trips" in names
    assert "network_daily_summary" in names
    assert "v_network_overview" in names
    assert "v_top_routes_static" in names
    assert manifest["realtime_gold_run_path"] is None
    assert result.iloc[0]["route_id"] == "R1"
    assert report_path.is_file()


def test_build_serving_database_with_realtime_tables_and_optional_missing_files(tmp_path:
Path) -> None:
    gold = make_gold_run(tmp_path)
    rt = make_rt_gold_run(tmp_path, include_optional_delay=False)
    serving_run = build_serving_database(gold, rt, serving_root=tmp_path / "serving")
    db_path = serving_run / "mobility_control_tower.duckdb"
    manifest = json.loads((serving_run /
"serving_manifest.json").read_text(encoding="utf-8"))
    names = table_names(db_path)
    health = query_serving_database(db_path, "rt-feed-health", limit=5)
    compatibility = query_serving_database(db_path, "rt-compatibility", limit=5)

    assert "rt_feed_health_snapshot" in names
    assert "rt_identifier_compatibility_snapshot" in names
    assert "v_rt_feed_health" in names
    assert "v_rt_identifier_compatibility" in names
    assert "v_rt_top_delayed_routes_snapshot" not in names
    assert manifest["realtime_run_id"] == "rt-1"
    assert health.iloc[0]["freshness_status"] == "PASS"
    assert compatibility.iloc[0]["status"] == "PASS"


def test_build_serving_database_with_full_realtime_views(tmp_path: Path) -> None:
    gold = make_gold_run(tmp_path)
    rt = make_rt_gold_run(tmp_path)
    serving_run = build_serving_database(gold, rt, serving_root=tmp_path / "serving")
    db_path = serving_run / "mobility_control_tower.duckdb"
    names = table_names(db_path)
    delayed_routes = query_serving_database(db_path, "rt-top-delayed-routes", limit=5)

    assert "v_rt_top_delayed_routes_snapshot" in names
    assert "v_rt_top_delayed_stops_snapshot" in names
    assert delayed_routes.iloc[0]["route_id"] == "R1"
'''

3. Images
No images detected.

4. Skipped Binary Files
No skipped binary files were detected.

5. Export Summary
- Included files count: 72
```

- Skipped files count: 15461
- Skipped binary files: 0
- Skipped large files: 0
- Warning: no image embed failures